

ZeBu[®] Virtual Host Solution User Guide

Version W-2025.06, September 2025



Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

About This Book	9
Contents of This Book	9
Related Documentation	10
Typographical Conventions	11
Synopsys Statement on Inclusivity and Diversity	11

1. Overview to Virtual Host Solution	13
VHS Architecture	13
Features	14
Requirements	15
Hardware and Software	15
QEMU and Host Machine Dependencies	16
Supported and Unsupported VS Versions	16
ABI Support	17
Benefits	17

2. Compiling and Running Virtual Host Solution on ZeBu	19
Compiling a Top-level RTL Module	19
Creating a Virtualization Testbench	20
Running the Virtualization Testbench	22

3. Configuring a Virtual Machine	25
ProxyPing	25
Configuring QEMU	26
Create a New QEMU VM	27
Configure and Run the QEMU VM	27
Shared Folder Settings	28
Copy Files To and From Guest OS Using SSH	28
Useful QEMU Configurations	29
Configuring VirtualBox	29

Create a New Virtual Machine	30
Configure and Run VirtualBox	31
Remove Virtualization Configurations	31
Access the Virtual Machine Log	31
4. DMTCP	33
.....	33
Restoring the Run	35
Checkpointing without connecting to a VM	35
5. Creating a ProxyEnv XML File	37
XML File Overview	37
XML Configuration Structure	38
XML Schema Structure	38
Working With the ProxyEnv XML GUI	42
Launch ProxyEnv XML GUI	43
Configure an XML File	44
Generate the XML File	44
Modify an Existing XML File	44
6. Protocol Specific Configurations	46
PCIe	46
Active State Power Management	47
Address Translation Service	47
Advance Error Reporting	48
Cycle Accurate Mode Support	48
Configurable Delay in Cycle Accurate Mode	49
Controlling Dynamic Lane Width	50
Dynamic Link Speed Change	50
Getting Handle to Transactor API	50
In-Band Commands	51
In-Band IDE Commands	52
Memory Coalescing	53
Use Model	53
Example	54
PCIe Hot Plug Support	55

Contents

Starting the VM From an Unplugged State of a PCIe Device	55
ProxyEnv	56
Plug Out and Plug-In a PCIe Device After the Initial VM Boot-up	57
PCIe Reset Support	57
VM Reset/Reboot	57
Hot Reset	58
Link-Ctrl Reset	58
Sending TLPs to Traffic Class	58
Simulation Support for PCIe Transactor	59
Example	59
Enabling PCIe Protocol Analyzer from VHS	59
CXL	61
System Configuration	62
Memory Windows	62
Memory Decoder Base Addresses	63
CXL.mem Transactions	64
USB	64
Supporting USB3 Transactor Monitor	65
Enhanced USB Support for VHS	65
Enhanced APIs for U1/U2 Timer Feature in VHS	76
Enhanced Suspend and Resume APIs for USB Transactor in VHS	77
USB Device Hot Plug Support	78
Initiating USB Speed Change	78
Ethernet	80
Ethernet PHY Interfaces	80
Connecting Virtual Ethernet to ZeBu	83
Running Virtual Ethernet Connector	84
Configuring ENET Connector as Server	84
Configuring ENET Connector as Client	84
Virtual Ethernet TAP	85
Configuring the Virtualization Testbench Component	85
Opening the Virtual NIC Interface in the VM	86
Sending Messages to the VM	86
Sending Messages to the Virtualization Testbench	87
Virtual ZEMI-3 Ethernet Transactor Support	87
Interpreting Ethernet Packet From Log	88
UFS	90
Inserting UFS Kernel	90
Running the Linux UFS Driver	91

Formatting and Mounting the UFS Disk	91
UFS Utilities	91
PCIe End Point vNIC Solution	93

7. IDE Support	97
Pre-Shared Keys (PSK)	97
Link IDE Stream	98
PSK Selective IDE Stream	99
Specifying Pre-Shared Keys	99
Swapping Pre-Shared Keys	100
Unsecuring a Pre-Shared Key Stream	101
Using IDE_KM	101
Configuring the Host Side	102
IDE_KM Over SPDM	102
IDE_KM Over DOE	104

8. Performance Management	105
PCIe Performance Metrics Print	105
Enabling and Controlling the Prints	106
Real Time Issues in Performance Management	107
Measuring Performance	107
Example: NVMe	108

9. Troubleshooting	109
Setup Related Issues	109
Compatibility Check Between VSA Package and Transactors	109
ProxyEnv-Related Issues	110
ProxyEnv Testbench Compile Issues	110
ProxyEnv Testbench Runtime Issues	111
Virtual Machine Issues	112
Interpreting ProxyTB Log	113
Configuration Reads/Writes	113
Memory Reads/Writes	114
Interpreting QEMU Log	116
With Debug Level 1	116

Contents

Configuration Reads/Writes	116
Memory Reads/Writes	117
With Debug Levels 2 and 3	118
Interpreting NVMe Log	119
Enable NVMe Analyzer Log Generation	119
Main Sections of the NVMe Analyzer Log	119
NVMe Command Execution - Steps	121
10. Examples	128
Settings for Running an Example	128
PCIe Example	129
Compile	129
Runtime	130
VHS With End Point as IIP DUT	130
VHS Example With VDM NVMe Device	132
NVMe Analyzer GUI	132
DMTCP Save and Restore With VHS-VDM PCIe	134
USB Example	136
Ethernet Example	137
UFS Example	140
11. Hardware and Software Interface	142
ProxyEnv	142
Constructor	143
connect () Method	144
disconnect () Method	144
shutdown () Method	144
setDebugLevel () Method	144
startAllProxyXtors () Method	145
getProxy*Xtors () Methods	145
ProxyEnv XML Structure	145
ProxyEnv Schema Structure	146
ProxyPCleXtor Component	147
enableClockControl () Method	148
cfgRd () Method	148
cfgWr Method	148
memRd () Method	149

Contents

memWr () Method	149
setDebugLevel () Method	149
setPreTrainingFct () Method:	150
setInitFct () Method	150
setPreDownstreamFct () Method	150
setPreCompletionFct () Method	150
setHotResetFct () Method	151
setPostHotResetFct () Method	151
setLinkResetFct () Method	151
setHotPlugFct () Method	152
setMessageHandler () Method	152
sendMessage() Method	152
lock () / unlock () Method	152
ProxyUSBXtor Component	153
lock () Method	153
unlock () Method	153
setInitFct () Method	153
setDebugLevel () Method	154
ProxyEthXtor Component	154
setDebugLevel() Method	154
enableTx() Method	155
enableRx() Method	155
isTxEnabled() Method	155
isRxEnabled() Method	155
flushFrames() Method	155
getFrameCounts() Method	156
printStats() Method	156

Preface

This chapter has the following sections:

- [About This Book](#)
 - [Contents of This Book](#)
 - [Related Documentation](#)
 - [Typographical Conventions](#)
 - [Synopsys Statement on Inclusivity and Diversity](#)
-

About This Book

The *ZeBu® Virtual Host Solution User Guide* describes how to use the Virtual Machine Adaptor for VirtualBox PCIe, USB, Ethernet, SATA Host, UFS Host, and CXL.

Note:

VirtualBox provided by Oracle America, Inc. "VirtualBox" is a registered trademark of Oracle America, Inc. and "Oracle" is a registered trademark of Oracle International Corporation.

Contents of This Book

The *ZeBu® Virtual Host Solution User Guide* has the following chapters:

Chapter	Describes...
Virtual Host Solution	Describes the PCIe and USB Virtual Adaptor
Hardware and Software Interface	Lists software and hardware required for using the PCIe and USB Virtual Adaptor
Running Virtualization on ZeBu Host	Provides the configuration required for PCIe and USB Virtual Adaptor
QEMU Configuration	Provides brief information about QEMU configuration
Miscellaneous Useful Things	Describes information about important features of VHS

Real Time Issues	Describes information on the real time challenges with Virtual Host Solution
Example	Provides running PCIe and USB examples

Related Documentation

Document Name	Description
<i>ZeBu Virtual System Adaptors Installation Guide</i>	Provides information about prerequisites and steps to download and install the Virtual System Adaptor (VSA) products.
<i>ZeBu Virtual Host Solutions User Manual</i>	Provides the usage of Virtual Host Solution with various ZeBu transactors.
<i>ZeBu 5G Virtual Adaptor Solutions User Manual</i>	Provides the usage 5G Virtual Adaptor.
<i>ZeBu PCIe Transactor User Manual</i>	Provides the usage of ZeBu PCI Express (PCIe) Transactor with your design being emulated in ZeBu.
<i>ZeBu USB DRD Transactor User Manual</i>	Provides the usage of USB DRD 3.1 Transactor with your design being emulated in ZeBu.
<i>ZeBu USB Host Transactor User Manual</i>	Provides the usage of USB Host Transactor with your design being emulated in ZeBu.
<i>ZeBu USB Device Transactor User Manual</i>	Provides the usage of USB Device Transactor with your design being emulated in ZeBu.
<i>ZeBu ENET Transactor User Manual</i>	Provides the usage of ENET Transactor with your design being emulated in ZeBu.
<i>ZeBu Virtual System Adaptor LCA Features Guide</i>	Provides a list of Limited Customer Availability (LCA) features available with Virtual System Adaptors.
<i>ZeBu Virtual System Adaptor Release Notes</i>	Provides enhancements, new features, and limitation in the S-2023.03-SP1 release.

Typographical Conventions

This document uses the following typographical conventions:

To indicate	Convention Used
Program code	<i>OUT</i> <= <i>IN</i> ;
Object names	<i>OUT</i>
Variables representing objects names	< <i>sig-name</i> >
Message	Active low signal name '<sig-name>' must end with <i>_X</i> .
Message location	<i>OUT</i> <= <i>IN</i> ;
Reworked example with message removed	<i>OUT_X</i> <= <i>IN</i> ;
Important Information	NOTE: This rule...

The following table describes the syntax used in this document:

Syntax	Description
[] (Square brackets)	An optional entry
{ } (Curly braces)	An entry that can be specified once or multiple times
(Vertical bar)	A list of choices out of which you can choose one
... (Horizontal ellipsis)	Other options that you can specify

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our

software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Overview to Virtual Host Solution

Virtual Host Solution (VHS) is used to connect virtualized physical interfaces to the corresponding physical interfaces in a design emulated in ZeBu. VHS supports virtualized interfaces that include USB, PCIe, CXL, Ethernet, SATA Host, and UFS Host. The emulated design appears as if it was physically connected in a PC and can be accessed by application software running in the Virtual Machine.

This section explains the following subtopics:

- [VHS Architecture](#)
- [Features](#)
- [Requirements](#)
- [Benefits](#)

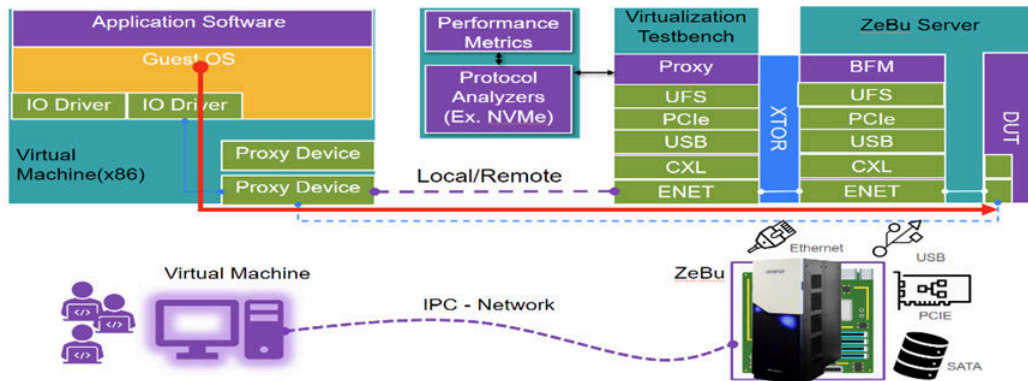
VHS Architecture

This technology allows early software development in the same environment where the software is to be used. Using a Virtual Machine allows any operating system to be used (as long as it is a supported guest OS), independently of the operating system supported by ZeBu. For example, the VM could be running Windows 10, even though the ZeBu software is running on Linux.

In addition, the Virtual Machine need not be running on the same host connected to the Zebu hardware. This allows multiple software engineers to work on their own computer hosting their Virtual Machines, all connecting to a shared pool of ZeBu machines.

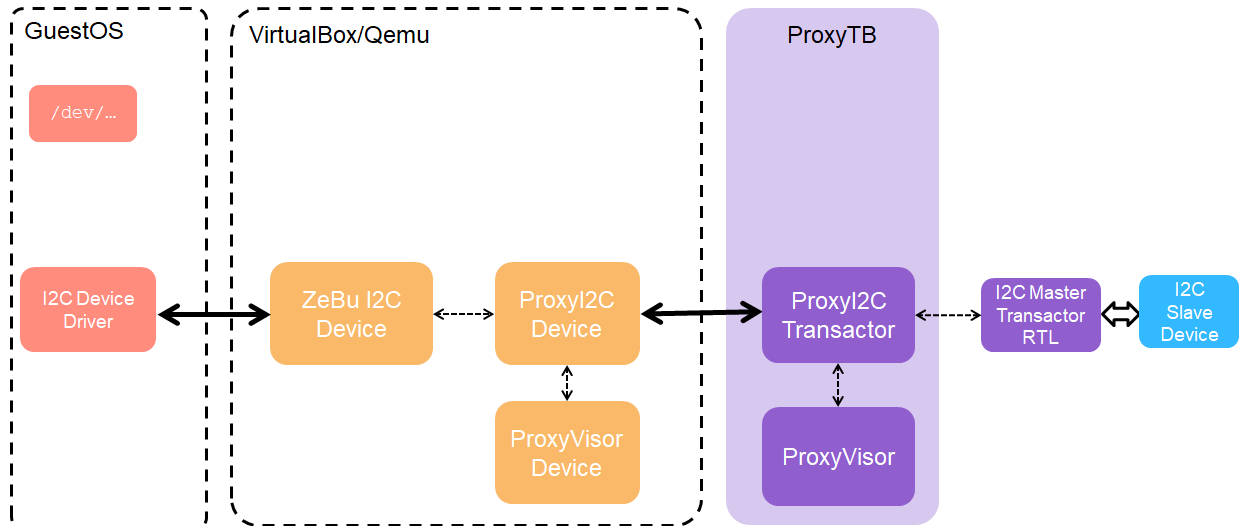
The following block diagram shows how a virtualized interface in the VM is connected to an emulated design of that interface.

Figure 1 ZeBu Virtualization



The following figure illustrates the architecture of VHS.

Figure 2 VHS Architecture



Features

VHS virtualized devices appear like any other device to the guest Operating System in the VM. It supports all the necessary features to properly operate under normal conditions in the guest Operating System. However, it supports only the following features:

- VirtualBox and QEMU
- Ubuntu as a Guest Operating System. However, other operating systems are successfully being used.

- PCIe, regardless of speed or generation, including hot-plug and reset, and Address Translation Service (ATS)
- CXL 2.0
- USB 2.0 and 3.X adaptors
- Ethernet speeds up to 800 Gb/s with PCS and MAC interface support
- SATA host
- UFS host (DPI)
- VM reset and reboot
- For accurate measurements of response times and performance (see Real Time Issues)
- Cycle accurate mode for physical interfaces such as PCIe and CXL

Note:

VHS does not support the following operations:

- Suspend
- Hibernate
- Resume

Requirements

The following topics describe the requirements to run VHS:

- [Hardware and Software](#)
- [QEMU and Host Machine Dependencies](#)
- [Supported and Unsupported VS Versions](#)
- [ABI Support](#)

Hardware and Software

VHS provides customized versions of VirtualBox and QEMU. The following table lists the versions of the Virtual Machine software included in VHS.

The following table lists the software and hardware versions for using VHS.

Table 1 Software Requirements

Hardware and Software	Versions
VirtualBox	5.2.20
QEMU	6.*

QEMU and Host Machine Dependencies

QEMU is a third-party tool that requires some mandatory software libraries to be present in the host machine for VM to startup. The hardware requirements of Virtual Host machines could also be different from that of Zebu Host machines. Contact Synopsys Support for any queries regarding hardware requirements for Qemu.

The following table lists host machine configurations on which the VSA package is validated.

	Host Operating System / Kernel Version	Processor
1	3.10.0-1160.31.1.el7.x86_64	Intel(R) Xeon(R) CPU E5-2670 0 @ 2.60GHz
2	3.10.0-1127.10.1.el7.x86_64	Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
3	34.18.0-553.el8_10.x86_64	Intel(R) Xeon(R) CPU E5-2640 v4 @ 2.40GHz

Each of these have their own underlying software and hardware requirements.

It is recommended to allocate two host CPU core per PCIe to QEMU process.

Note:

VirtualBox provided by Oracle America, Inc. "VirtualBox" is a registered trademark of Oracle America, Inc. and "Oracle" is a registered trademark of Oracle International Corporation.

Supported and Unsupported VS Versions

VSA solutions are compatible with the W-2025.06 release of ZeBu Vertical Solutions.

For every version of VSA, there are supported combinations of ZeBu and transactors. Any unsupported combinations of these toolsets cause unexpected behavior. The following table lists supported versions of ZeBu and Transactors for each release of VSA.

Table 2 Supported Versions for ZeBu and Transactors

VSA Release	ZeBu Version	Transactor Version
W-2025.06	W-2025.06	W-2025.06-BETA3
V-2024.03-SP1	V-2024.03-1-TZ	V-2024.03-SP1
V-2024.03	V-2024.03-1	V-2024.03
S-2023.03-SP1	S-2021.09-2	S-2023.03-SP1
	S-2021.09-1	S-2023.03
	Q-2020.03-SP1-4	Q-2021.06-SP3
S-2023.03	S-2021.09-1	S-2023.03
	Q-2020.03-SP1-4	Q-2021.06-SP3
	Q-2020.03-SP1-3	Q-2021.06-SP2 Q-2021.06-SP1
S-2023.03-SP2	S-2021.09-2	S-2023.03-SP2

Mixing “S” and “Q” streams of ZeBu and transactor versions is not supported and should not be used. For instance, you cannot use S-2021.09-2 version of ZeBu with Q-2021.06-SP3 version of transactors.

ABI Support

When compiling the ZeBu testbench, set `VSA_ABI` to indicate if the code should be compiled with Application Binary Interface (ABI) version 0 or 1 based on the following criteria:

For ZeBu Q-2020.03-SP1-3 or Q-2020.03-SP1-4, set `VSA_ABI` to 0.

For ZeBu S-2021.09-1 or later, set `VSA_ABI` to 1.

By default, `VSA_ABI` is set to 1.

Benefits

Benefits of VHS are as follows:

- Multiple software developers on physical/virtual machines
- OS-agnostic physical/virtual machines

- Real-time emulation at MHz speeds
- Vendor-provided virtual motherboards with timeouts
- Full-stack software/driver development
- Post pre-silicon hardware IP (Intellectual Property) integration
- Limited and complex advanced hardware debugging
- Sequential parallel hardware and software development cycle (TTM)
- Limited time for more software validation (software maturity)

2

Compiling and Running Virtual Host Solution on ZeBu

This chapter explains how to create and run VHS on ZeBu and connect to a Virtual Machine.

In the ZeBu testbench, a software component, called a *Proxy Environment*, is instantiated to integrate VHS. VHS handles most of the detailed transactor configuration and input and output to and from the transactors. You need to only do the following tasks in the testbench for integration:

1. Instantiate Proxy environment.
2. Provide an XML file describing the physical interfaces present in the design.
3. Start the environment.
4. Initiate a connection to the Virtual Machine after the DUT is properly initialized.

For more information, see the following topics:

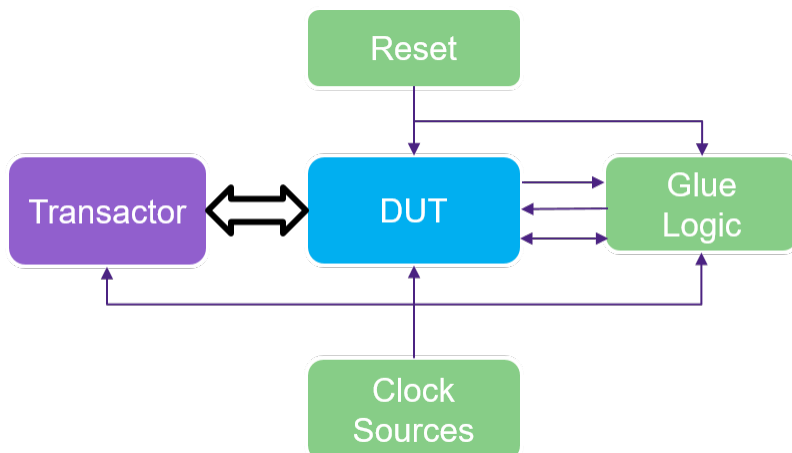
- [Compiling a Top-level RTL Module](#)
- [Creating a Virtualization Testbench](#)
- [Running the Virtualization Testbench](#)

Compiling a Top-level RTL Module

An existing emulation top-level module can be reused without any additional compile time changes for VHS as the virtualization components are built on top of existing transactors. The requirements of a top-level module for ZeBu with Virtual Host Solution are the same as those for regular emulation using transactors.

The following figure depicts the top-level RTL module communication for emulation.

Figure 3 Top-level Module Communication



For more information on the process of creating a top-level RTL module, the required clock and reset signals, any glue logic or RTL adapters, as well as the hardware compilation process, see the documentation and examples of each relevant transactors.

Note:

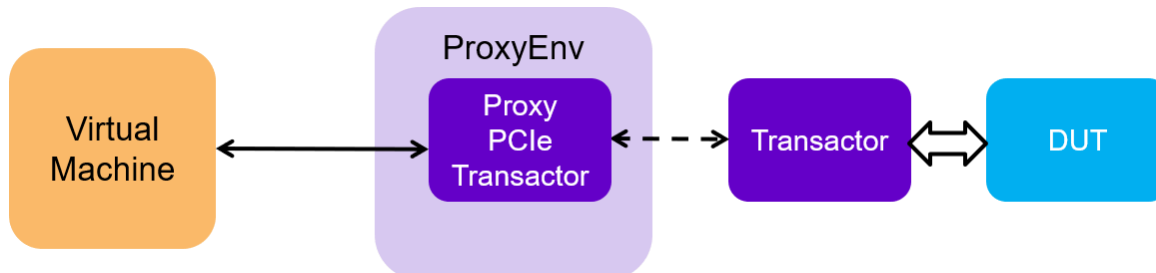
Each transactor has specific requirements, names, and options that are outside the scope of this document.

Creating a Virtualization Testbench

A virtualized testbench also known as a *Proxy testbench* is a C++ testbench that interfaces between the device drivers in a Virtual Machine and the DUT in the ZeBu emulator. The virtualization testbench virtualizes the corresponding interfaces on the DUT into corresponding devices in the Virtual Machine.

The following figure displays a virtualized testbench.

Figure 4 Virtualized Testbench



ProxyEnv is the virtualized testbench component that encapsulates Proxy transactor and takes all the configurations of the interfaces in an XML format. Proxy transactor encapsulates ZeBu transactors and provides translation between Virtual Host commands and transactor APIs. These Proxy transactors connect to the transactor RTL in ZeBu.

There are two formats in which you can create a Virtualization testbench. These are as follows:

- *zRci-Based Testbench*
- *C++ Based Testbench*

zRci-Based Testbench

This is the recommended format. To create a zRci testbench, you need to instantiate and configure a `ProxyEnv` component and compile the testbench so that it can be used during runtime.

You can start with the zRci files provided in the `Examples` directory because `ProxyEnv` is generic in nature.

If your emulation environment *does not* contain any additional transactors, start using the default testbench, `proxyTBEnv.cc`, without any modifications. This file is available under protocol-specific `Examples` folder. For example, for PCIe, it is available under the folder path `$VSA_VHS_ROOT/Examples/PCIe/zRci`.

To compile and run the testbench, use the following command:

```
%make run_zrci
```

However, if your emulation environment contains additional DPI transactors, modify the default file, `proxyTBEnv.cc`, to instantiate, initialize, and drive the additional transactors required by the DUT. Additionally, you can specify configurations of the design interfaces using `ProxyEnv` XML configuration file explained in `ProxyEnv XML Structure` section.

C++Based Testbench

To create a testbench in C++-based format, you can start with the C++ files provided in the `Examples` directory under `$VSA_VHS_ROOT/Examples/PCIe/C++` folder.

To compile the testbench, use the following command:

```
%make ProxyTB
```

To run the testbench use the following command:

```
%make ProxyTBRun
```

Note:

To instantiate, initialize, and drive these additional transactors, and the additional command-line arguments needed to compile the complete testbench, is beyond the scope of this document. You should refer to the user guide and examples of each relevant additional transactors.

Running the Virtualization Testbench

There are common elements in the execution of a Virtualization testbench that are important to understand whether it is a standalone testbench or a testbench with other transactors.

ProxyEnv Instantiation

In every Virtualization Testbench, the following line creates the ProxyEnv component, which in turn creates all required ancillary interface virtualization components according to the specified XML configuration file.

```
ProxyEnv env("env", "path/to/xmlfile", ...);
```

An exception is thrown if one of the interface virtualization components cannot be instantiated as configured. For details on the XML configuration specification, see [Creating a ProxyEnv XML File](#).

Starting the Virtualization Environment

The following line starts all transactors used by the Virtualization environment.

```
env.startAllProxyXtors();
```

This causes the clocks to start running, the transactors to initialize, interfaces to link-up, and in some cases device enumeration to happen.

All transactors and interface virtualization components execute their initialization sequence concurrently in separate threads. This method does not block and returning does not indicate that the virtualization environment has completed its initialization. The latter is indicated by the `ProxyEnv::Callbacks_t::readyFct()` callback function. This callback function is called by the virtualization environment when all of its interface virtualization components have completed their initialization sequences and are ready to connect to the Virtual Machine.

Note:

Implementation of `ProxyEnv::Callbacks_t::readyFct()` callback function, like of any other callback function, is entirely optional.

Connecting to the Virtual Machine

The following line waits for all the interface virtualization components in the virtualization environment to complete their initialization sequence (if not already done) and then connects to the Virtual Machine.

```
env.connect();
```

If the Virtual Machine is not already waiting for a connection, this method waits for the Virtual Machine to connect. Once ready to connect, it displays a “Ready to connect...” message indicating that Virtual Machine can be now connected as per instructions in [Configuring a Virtual Machine](#). This is called BLOCKING method. It does not return until the Virtual Machine is connected to the Virtualization Environment.

The virtualization environment or the Virtual Machine can be configured as a server or a client when establishing the connection. This configuration is specified in the XML configuration file. The choice of this configuration is arbitrary, but the virtualization environment is usually configured as the server. If multiple users plan to create servers on the same machine, they must use different port numbers. The server (which can be the Testbench or the VM) can select an unused port number and write that number into a Server Specification File. The Server Specification File is read by the client to identify the host and the port to connect to. In this case, the name of the Server Specification File is unique for each user.

Inside the `<vHost:ProxyVisor>` element, the following elements specify the Virtual Machine / Virtualization Environment connection relationship:

- `<vHost:IsServer>true</vHost:IsServer>`: Specifies that the Virtualization Environment is the server if `true`, or the client if `false`. If set to `true`, virtualization environment waits for the VM to connect.
- `<vHost:ServerSpecFile>${HOME}/myConnection</vHost:ServerSpecFile>`: Specifies the name of the Server Specification File that is created by the server ONLY when it is ready to connect and actively listening for incoming connections. It is automatically deleted after a connection is established to avoid a stale Server Specification File to remain and is accidentally used by a connecting client.
- `<vHost:MaxServerStartSecs>300</vHost:MaxServerStartSecs>`: Sets the timeout value when waiting for a `proxyXtor` to fully initialize and be ready to accept transactions from the VM. Default is 60 seconds. Timeout is disabled if set to 0 seconds.
- `<vHost:AutoShutdownMins>15</vHost:AutoShutdownMins>`: If no connection with a client is established within the specified number of minutes (default is 15), the server automatically shuts down with an error. This is used to avoid tying up an emulation resource for ever if, for some reason, the Virtual Machine was never able to start.

Shutting Down the Virtualization Environment

The following line waits for all the threads running in the Virtualization Environment to complete and terminate.

```
env.shutdown();
```

After this method returns, it is safe to destroy the Virtualization Environment object.

3

Configuring a Virtual Machine

A **hypervisor**, or **Virtual Machine Manager (VMM)**, facilitates the simultaneous operation of multiple virtual machines on a single hardware platform. It serves as a mediator, allocating resources such as CPU, memory, and GPU among guest operating systems, with the host OS being the operating system of the hypervisor itself.

For more information, see the following topics:

- [ProxyPing](#)
- [Configuring QEMU](#)
- [Configuring VirtualBox](#)

ProxyPing

`ProxyPing` is a utility that is shipped with the VHS for setting the VM configuration to work with the device in ZeBu. When the testbench is correctly configured a `ready to connect` message is displayed. At this stage, the `ProxyPing` utility needs to be run to probe the testbench process and set the VM configurations. It searches a running virtualization environment for all present virtualized interfaces, and automatically generates the necessary command line arguments to configure the necessary virtualization devices in the Virtual Machine.

To configure a QEMU Virtual Machine, use the following:

```
ProxyPing Options -Q {connectionSpec}
```

where,

- `connectionSpec` must be specified for each running Virtualization testbench and it can be one of following:
 - `-x <configFile>[:<pv>]` is the connection to the Virtualization testbench described by the specified XML configuration file.

Note:

If the Virtualization Testbench contains more than one `ProxyVisor`, the name of the instance name of the `ProxyVisor` to connect to must be specified.

- **Options** can be one of the following:
 - **-C vm**: Configures the specified VirtualBox VM to match Virtualization Testbench (Deletes all if no testbench port is specified)
 - **-d n**: Sets VM debug level.
 - **-D n**: Sets this application's debug level.
 - **-K**: Abruptly stops the Virtualization Testbench after pinging it.
 - **-n**: Displays VirtualBox configuration commands.

Note:

Do not execute the VirtualBox configuration commands because it requires the **-C** option)

- **-P <opts>**: It is a PCIe-specific option. It can be specified more than once for multiple PCIe options.
Bn: Assigns the bus number starting with a specified number (Default: 1).
- **-Q**: Displays QEMU configuration options required to match Virtualization Testbench.
- **-S n**: Tests the link speed with n PING.
- **-v**: Displays version.
- **-W n**: It is the maximum time for a client to wait to connect with a server, in minutes. Default is 5.

Configuring QEMU

The Virtual Machine is a software construct that must be instructed to add special devices to connect to a virtualized testbench and communicate with the DUT. This configuration must be performed when the VM is not running, like physical peripherals must be added to a PC while it is powered off.

See the following sections for details on how to create and run a QEMU VM:

- [Create a New QEMU VM](#)
- [Configure and Run the QEMU VM](#)
- [Useful QEMU Configurations](#)

Create a New QEMU VM

You can create a VM using any guest OS. To create a guest OS, download the bootable image of the OS you need from the resources available online.

You can create an Ubuntu 16.04 VM using the `QemuNewVM` target available in the PCIe Example Makefile: `% make QemuNewVM`.

This target executes the following commands:

1. Downloads a bootable image of the guest OS, `ubuntu-18.04.6-desktop-amd64.iso`.

```
% wget http://releases.ubuntu.com/16.04/ubuntu-16.04-desktop-amd64.iso
```

2. Create a new VM.

```
qemu-img create -f qcow2 $(QCOW2_IMG) 80G
qemu-system-x86_64 -M q35,accel=kvm -m 8192 -machine smm=off -drive
file=$(QCOW2_IMG),if=ide -cdrom ubuntu-18.04.6-desktop-amd64.iso
-boot d
```

Here `QCOW2_IMG` is the name of the qcow2 image to be created.

Note the following:

- QEMU on VHS has a limitation where the total VF bar size should not exceed 512 GB.
- For SRIOV VF having offset greater than or equals to 256 with QEMU 8.2, it is recommended to use Linux kernel guest OS version 4.3 or later.

Configure and Run the QEMU VM

The usual process is to tie the output of ProxyPing to a file and then add the content of that file to the QEMU command line. By default, the virtualization environment returns to waiting for a connection, so it is possible to run ProxyPing immediately followed by starting the Virtual Machine.

```
% ProxyPing -Q -x <config.xml> | tee Qemu.args
% qemu-system-x86 `cat Qemu.args` -drive file=$(QCOW2_IMG)
```

Here `Qemu.args` is nothing but the output of the ProxyPing and the drive file is the qcow2 image of the VM. It is not necessary to run ProxyPing before every VM connection. As long as the structure of the virtualized interfaces in the virtualization environment and the topology of the virtualized devices do not change, the VM does not need to be configured differently.

Shared Folder Settings

For sharing files between the host file system and the VM, specify the following QEMU command line arguments:

```
-virtfs local,id=SharedFolder,path=PATH_TO_SharedFolder,  
mount_tag=SharedFolder,security_model=none
```

where,

`PATH_TO_SharedFolder` is the host file system path that you want to access in the VM.

To access `PATH_TO_SharedFolder`, execute the following commands on the VM:

```
% mkdir /tmp/shared; #This is the mount point  
% sudo mount -t 9p -o trans=virtio,version=9p2000.L SharedFolder /tmp/  
Shared
```

The files in the `PATH_TO_SharedFolder` are accessible through `/tmp/shared` in the Virtual Machine.

Note:

The commands mentioned are only validated with the Ubuntu VM and might not work with all operating systems.

Copy Files To and From Guest OS Using SSH

The simplest way to transfer files to and from the Guest OS running on the VM is to copy them using SSH.

The following command options enable and tunnel an SSH connection from port 54321 (can be any unused port number between 16768 and 65534) on the local host to the guest. Normal SSH commands can then be used to copy files using `'scp'` or even remote-login using `'ssh'`.

```
% qemu-system-x64 <other QEMU options> \  
-device e1000,netdev=ssh \  
-netdev user,id=ssh,hostfwd=tcp::54321-:22
```

To copy a file from the local host to the guest under the guest userid `'root'`, use the `'scp'` command as follows:

```
% scp -P 54321 test.sh root@localhost:test.sh
```

Useful QEMU Configurations

Some of the useful QEMU command-line options or parameters to configure specific behaviors when launching QEMU are listed in the following table:

Configuration	Description
<code>-display none</code>	Opens QEMU in non-GUI mode
<code>-smp <no_of_cores></code>	Specifies number of cores the guest OS is to use
<code>-cpu <cpu></code>	Specifies the CPU architecture to emulate. such as, EPYC
<code>-vga virtio</code>	Resizes the VM to a smaller size
<code>-debugcon file:debug.log -global isa-debugcon.iobase=0x402</code>	Saves the BIOS output to 'debug.log'
<code>-device pcie-zebu-rp,id=pv0.pci0.bus1,...,cpl MaxSecs=600</code>	Specifies completion timeout in seconds for each non-posted PCIe transaction. Default is 10 seconds. If set to 0, no timeout value is set for completion timeout
<code>-M q35,accel=kvm,kernel-irqchip=sp lit -device intel-iommu,intremap=on,devi ce-iotlb=on</code>	Enables <code>iommu</code> in the guest OS. If the driver is expecting <code>iommu</code> to be enabled, this has to be added to the QEMU command line.
<code>-nic user,model=virtio-net-pci</code>	Enables Internet on the VM
<code>-usbdevice tablet</code>	Improves mouse pointer performance.
<code>-cpu host</code>	CPU specifications of the mirror's host. Yields better performance.

Configuring VirtualBox

QEMU is the recommended hypervisor for creating VMs and support for VirtualBox is limited. However, if you are an existing VirtualBox user, see the following sections for details on how to create and run a VirtualBox VM:

- [Create a New Virtual Machine](#)
- [Configure and Run VirtualBox](#)
- [Remove Virtualization Configurations](#)
- [Access the Virtual Machine Log](#)

Create a New Virtual Machine

You can create a VM using any guest OS. To create a guest OS, download the bootable image of the OS you need from the resources available online.

Before you create a new VM, you need to select a location where the runtime files for the VM are located. These files are typically huge as they contain the image for the hard drives used by the VM. In the following PCIe example, the runtime files are located in `~/VirtualBoxes`. You can create an Ubuntu 16.04 VM using the `VBoxNewVM` target available in the PCIe Example Makefile as follows

```
%make VM_NAME=myVM VM_BASE_DIR=$(HOME)/VirtualBoxes -f Examples/PCIe/
shared/Config.mk Makefile VBoxNewVM
```

This target executes the following commands:

1. Download a bootable image of the guest OS.

```
% wget http://releases.ubuntu.com/16.04/ubuntu-16.04-desktop-amd64.iso
```

2. Create a new VM using the ICH9 PCI bridge.

```
% VBoxManage createvm --name myVM --ostype Ubuntu_64 --basefolder ~/
VirtualBoxes --register
% VBoxManage modifyvm myVM --chipset ich9
```

3. Increase the memory and video RAM size.

```
% VBoxManage modifyvm myVM --memory 2048 --vram 16
```

4. Create and initialize a bootable SATA hard drive to store the guest OS installation, swap files, and subsequently work within the VM.

```
% VBoxManage storagectl myVM --name SATA --add sata --bootable on
% VBoxManage createhd --filename ~/VirtualBoxes/myVM/hdd --size 40960
% VBoxManage storageattach myVM --storagectl "SATA" --port 0 --device
0 --type hdd --medium ~/VirtualBoxes/myVM/hdd.vdi
```

5. Create a bootable DVD drive initialized with the guest OS bootable image.

```
% VBoxManage storagectl myVM --name IDE --add ide --bootable on
% VBoxManage storageattach myVM --storagectl "IDE" --port 1 --device 0
--type dvddrive --medium $PWD/ubuntu-16.04-desktop-amd64.iso
```

6. Create a folder that is accessible from both, the guest and host OS, to enable exchange of files.

```
% mkdir -p ~/VirtualBoxes/myVM/SharedFolder
% VBoxManage sharedfolder add myVM --name SharedFolder --hostpath
~/VirtualBoxes/myVM/SharedFolder -automount
```

7. Start the VM and perform the first time installation of the guest OS.

```
% VBoxManage startvm myVM --type sdl
```

When the guest OS is installed and the VM rebooted, install the VirtualBox guest additions to support shared folders. As a result, any files located in the `~/VirtualBoxes/myVM/SharedFolder` directory are visible in the `/media/sf_SharedFolder` directory in the VM.

The following commands must be executed on the VM:

```
% sudo apt-get install virtualbox-guest-dkms
% sudo apt-get install build-essential
% sudo apt-get -f install
% sudo adduser $USER vboxsf
% reboot
```

Configure and Run VirtualBox

Each VM instance in VirtualBox must be appropriately configured to connect to the proxy testbench running on the ZeBu host. The *ProxyPing* utility is used to automatically configure a VM to match the PCIe device topology found in the virtualization testbench. After the virtualization testbench is running and you see a *Ready to connect* message, use the following commands to configure VirtualBox and subsequently run the VirtualBox VM:

```
% ProxyPing -C myVM -x myenv.xml
% VBoxManage startvm myVM --type sdl
```

Remove Virtualization Configurations

If you want to subsequently make the VM usable either standalone or with a different virtualized testbench, you must remove the virtualization configuration items from the VM. Otherwise, the VM always waits to connect with a virtualization testbench before starting up (and eventually timeouts with an error). Therefore, no longer remains usable.

To remove all virtualization configuration items, use the following command:

```
% ProxyPing -C myVM
```

Access the Virtual Machine Log

If you want to see the transactions that occur between virtualized testbench and the VM, you can check the `VBox.log` file. All the events that occur during VM boot are also captured in this log file. To find the log file, you can execute the following command:

```
%VBoxManage showvminfo myVM | grep "Logs"
```

This command shows the path of the Logs folder on your VM. Inside this folder, you find the VBox.log file. Up to three old VBox.log files are stored in this location and are named as VBox.log.1, VBox.log.2, and VBox.log.3.

4

DMTCP

The DMTCP checkpointing feature allows you to save the status of the entire hardware DUT, software testbench environment and Virtual Machine during the emulation runtime. The saved state can be restored later saving on valuable emulation cycles.

The hardware state is captured by ZeBu Save and Restore functionality, software testbench environment is saved using an external open-source tool called Distributed MultiThreaded Checkpointing (DMTCP) and Virtual machine state is captured using Snapshot feature of the hypervisor.

You can perform checkpointing at different points in the run. The most common use case is saving both the VM and hardware states after the VM boots. This helps save on ZeBu load time and VM boot time. Another use case is saving just the hardware states before connecting to a VM. Both these scenarios and the changes required for these are described in the following sections:

Checkpointing after VM boot

To checkpoint or save the state of the run, the following changes need to be made to normal run environment:

1. Inside testbench, enable DMTCP before starting the virtualization environment using the following command:


```
env->DMTCP_VM.enable()
```
2. Inside the testbench, define a flag that will be used to indicate whether hardware save is completed. This flag is initialized to a false value in the beginning and inside the `zRci_post_save()` function, it is set to true indicating hardware save is completed. The value of this flag is then checked inside the `saveFct()` callback to exit the save process.

- Sample flag usage:

```
bool saveDone = false;
```

- Sample flag usage inside zRci post save function:

```
void* zRci_post_save(const ZRCI::TbOpts& tbopts)
```

Chapter 4: DMTCP

```

{
    fprintf (stderr, "Hardware Save Completed!\n");
    saveDone = true;
    return 0;
}

```

3. Inside the testbench, a `saveFct()` callback should be registered and defined. This callback is executed after VM save is completed and it initiates the hardware save by calling `checkpoint -fullsave zRci` command from the testbench using `zRci_UCLI_callback()`.

- Sample `saveFct()` callback:

```

bool saveFct(const char* saveTo, ProxyEnv *env, void* arg)
{
    printf("ProxyEnv is saving to \"%s\"...\n", saveTo);
    sleep(5);
    std::string strTclCmd;
    strTclCmd = "__save_dmtcp_cmd_from_pci_xtor ";
    strTclCmd += saveTo;
    printf("VM is checkpointing \"%s\"\n", saveTo);
    printf("TCL command callback is \"%s\"\n", strTclCmd.c_str());
    zRci_UCLI_callback(strTclCmd.c_str());
    while (true)
    {
        if (saveDone) break;
    }
    return true;
}

```

- Sample UCLI procedure defined in UCLI script:

```

proc __save_dmtcp_cmd_from_pci_xtor { name }
{
    puts "Start emulator saving process"
    checkpoint -fullsave $name
    puts "End emulator saving process"
    run
}

```

4. While launching the run, launch the testbench under DMTCP daemon

as: `$DMTCP_HOME/bin/dmtcp_launch - -modify-env testbench`

where,

`$DMTCP_HOME` is the directory where DMTCP is installed.

5. After DMTCP is enabled in the testbench, you can initiate a checkpoint at any point in the run.

For checkpointing once Qemu is launched, you can execute the following command:

```
echo "save <snapshotname>" | ncat localhost <monitor port>  
Here <monitor port> is the same port passed during Qemu launch with  
"-monitor .." argument
```

This saves both, ZeBu hardware states and Virtual Machine snapshots, inside the run directory in a folder called `zebuVMSnapshot` which is suffixed with an integer.

Restoring the Run

To restart the emulation from a specific checkpoint, you should launch the run using the script generated inside the checkpoint directory as follows:

```
./<run_db>/<zebuVMSnapshot>/dmtcp_restart_script_zebu.sh
```

Note:

Do not go to the `zebuVMSnapshot*` directory to execute the script. This causes errors when recovering data from hardware save directory. You should run the restart script from the same directory from where you executed the original emulation run.

After the ZeBu hardware states are restored, you see a message saying "Ready to connect". Now you can restore your Qemu VM state from the saved snapshot by running the Qemu launch command and appending the following to it:

```
qemu-system-x86_64 ... -loadvm <snapshotname>
```

This restores the VM state and connects to ZeBu.

Checkpointing without connecting to a VM

You can save the hardware states after PCIe enumeration and before connecting to a VM. After hardware states are restored, you can connect to VM and continue the run. In this case, some changes must be made to the default VHS testbench flow:

1. In the testbench, do not call `connect()` after `startAllProxyXtor()`. This ensures that after PCIe enumeration, socket file is not created and hardware save can be done without any open socket connection.
2. After PCIe enumeration is done, hardware save can be done by invoking `checkpoint -fullsave` command from `zRci` prompt.
3. After hardware save completes, call the `connect` call in a separate thread in testbench. This can be added to `zRci_param()` and invoked from `zRci` prompt.

Sample code for invoking `connect()` call :

```
pthread_t pbiThread;
:
void* pcieThread(void*)
{
    fprintf(stderr, "***Start ProxyEnv Connect***\n");
    env->connect();
    return NULL;
}
:
void* zRci_param(const std::string& key, const std::string& value) {
:
if (key == "connectcall")
{
    pthread_create(&pbiThread, NULL, pcieThread, NULL);
}
:
}
```

Note:

Qemu `savevm` command requires the disk image to be in QCOW2 format. If your disk image is in .img or raw format, make sure you convert to QCOW2 format before using DMTCP save.

VHS supports DMTCP and the Examples folder in the VHS package contains instructions to do this. See [Examples](#) for instructions to run a DMTCP example.

5

Creating a ProxyEnv XML File

XML configuration file serves as the foundation for the Virtualization Testbench. XML configuration file initiates the creation of the ProxyEnv component and is responsible for generating essential ancillary interface virtualization components and configuring them. This file is passed as an argument while creating ProxyEnv component in the testbench.

```
ProxyEnv env("env", "path/to/xmlfile", ...);
```

This section describes the configuration structure and schema of the XML configuration file. In addition, it describes the steps to create an XML file using the ProxyEnv GUI.

For more information, see the following topics:

- [XML File Overview](#)
- [XML Configuration Structure](#)
- [XML Schema Structure](#)
- [Working With the ProxyEnv XML GUI](#)

XML File Overview

There are three main sections to an XML configuration file:

1. **ZebuWork Path and DesignFeaturesPath**, which are specific to the design under test.
2. **ProxyVisor configurations** that are non-protocol specific configurations. These generally deal with Zebu and VM connectivity.
3. **ProxyXtor configurations** that are protocol specific configurations

You can use the existing XML configuration files inside VSA package `Example/<Protocol>/` area as a reference to start integrating it in your design.

Alternatively, you can also use the ProxyEnv XML GUI utility for generating the file. For details about using the ProxyEnv XML GUI utility, see [Working With the ProxyEnv XML GUI](#).

XML Configuration Structure

The XML configuration file encapsulates all of ZeBu-side configurations of VHS. Only a single XML file is needed for a ZeBu testbench, regardless of how many physical interfaces or VM connections are used. An example of an XML configuration file is as follows:

```
<vHost:Config xmlns:vHost='urn:synopsys.com:vHost'>
  <vHost:Version>1.0</vHost:Version>
  <vHost:CreationTime>2021-06-10T16:38:17Z</vHost:CreationTime>
  <vHost:ZebuWorkPath>${PWD}/../zebu/zcui.work.B2B/zebu.work</vHost:ZebuWorkPath>
  <vHost:DesignFeaturesPath>${PWD}/../zebu/designFeatures.B2B</vHost:DesignFeaturesPath>
  <vHost:ProxyVisor>
    <vHost:Name>pv0</vHost:Name>
    <vHost:IsServer>true</vHost:IsServer>
    <vHost:PortNumber>0</vHost:PortNumber>
    <vHost:ServerHostName></vHost:ServerHostName>
    <vHost:ServerSpecFile>${HOME}/VM.socket</vHost:ServerSpecFile>
    ...
    ...
    ...
  <vHost:ProxyXtor>
    <vHost:Name>pci0</vHost:Name>
    ...
    ...
  </vHost:ProxyXtor>
</vHost:ProxyVisor>
</vHost:Config>
```

In the configuration structure, the `ZebuWorkPath` and `DesignFeaturesPath` are mandatory. You need at least one `ProxyVisor` component to connect to a VM. Every VM has its specified `ProxyVisor` component. There can be one or many `ProxyXtor` components registered with `ProxyVisor`.

XML Schema Structure

Every component in the XML configuration file has a reference schema at `$VSA_VHS_ROOT/schemas/schema_<component>.xml`. This file contains details of all the

configuration options available and the possible values for them. Consider the following example of PCIe (ProxyPCIeXtor) component schema:

```
title: ProxyPCIeXtor
description: ProxyPCIeXtor Schema
type: object
properties:
  Name:
    label: Name
    description: ProxyPCIeXtor Instance Name
    type: string
    default: pcie
  HdPath:
    label: HDL Path
    description: HDL Path of the Driver
    type: string
    mandatory: True
  HdPathToDisableWhenActiveHigh:
    label: HDL Path to Disable Signal
    description: HDL Path of the Xtor Disable Signal
    type: string
  ClockControlled:
    label: Clock Controlled
    description: Enable Clock-Controlled Mode
    type: array
    items:
      type: string
      enum: ["false", "true"]
      default: ["false"]
  DebugLevel:
    label: Debug Level
    description: Set DebugLevel
    type: integer
    minimum: 0
    maximum: 4
    default: 0
```

In the schema, `properties` define the different components of the XML. Each property corresponds to a group of configuration parameters. Properties can also be nested. Every `label` in the schema translates to a configuration parameter component in the XML. `Description` explains the functionality of the parameter and `type` specifies the type of configuration parameter expected. Wherever applicable, `default` values and possible values of the configuration parameter are provided.

Note:

XML configuration options are also provided in a tabular format in the `vsa_vhs_config.html` file at the following location in the VHS package:
`$VSA_VHS_ROOT/UI/html/`

Some XML parameters that can be changed statically (according to specific requirements) are as follows:

Specifying Runtime Debug Verbosity Levels

The debug verbosity level in each virtualized Root Complex can be specified at instantiation time via the `<vHost:DebugLevel>` XML element. By default, the debug verbosity level is 0 (no debug messages). The larger the debug verbosity level value, the more detailed the debug messages issued by the PCIe Function. Maximum verbosity level is 4. It is recommended to set the debug level to "01" unless otherwise needed.

Specifying Clock Control Method

This method statically enables and disables the clock control mode.

`<vHost:ClockControlled>false</vHost:ClockControlled>` where, `enableClockControl(false)` switches to the free running mode.

Connecting to the Virtual Machine

The virtualization environment or the Virtual Machine can be configured as a server or a client when establishing the connection. This configuration is specified in the XML configuration file. The choice of this configuration is arbitrary, but the virtualization environment is usually configured as the server. If multiple users plan to create servers on the same machine, they must use different port numbers. The server (which can be the Testbench or the VM) can select an unused port number and write that number into a Server Specification File. The Server Specification File is read by the client to identify the host and the port to connect to. In this case, the name of the Server Specification File is unique for each user.

Inside the `<vHost:ProxyVisor>` element, the following elements specify the Virtual Machine / Virtualization Environment connection relationship:

- `<vHost:IsServer>true</vHost:IsServer>`: Specifies that the Virtualization Environment is the server if `true`, or the client if `false`.
- `<vHost:ServerSpecFile>${HOME}/myConnection</vHost:ServerSpecFile>`: Specifies the name of the Server Specification File that is created by the server only when it is ready to connect and actively listening for incoming connections. It is automatically deleted after a connection is established. This is done to avoid a stale server Specification File to remain and accidentally used by a connecting client.
- `<vHost:MaxServerStartSecs>300</vHost:MaxServerStartSecs>`: Sets the timeout value when waiting for a proxyXtor to fully initialize and be ready to accept transactions from the VM. Default is 60 seconds. Timeout is disabled if set to 0 seconds.
- `<vHost:AutoShutdownMins>15</vHost:AutoShutdownMins>`: If no connection with a client is established within the specified number of minutes (default is 15), the server automatically shuts down with an error. This is used to avoid tying up an emulation resource for long, if for some reason the VM was never able to start.

Enabling PCIe or USB Transactor in ProxyEnv

VHS supports enabling and disabling PCIe and USB transactors. To enable the PCIe or USB transactor, add the following XML configuration:

```
<vHost:HdlPathToDisableWhenActiveHigh>xmr_of_the_signal_used_to_disable</vHost:HdlPathToDisableWhenActiveHigh>.
```

To disable the transactor, ensure that it is done at time 0 before constructing the ProxyEnv.

PCIe Static Link Speed Change

VHS supports PCIe Static link speed change. Note that speed change is initiated only by RC. Valid values for changing speed are 0-6, where 0 implies that no speed change is done. Any value between 1 and 6 represents the corresponding Gen speed, that is, from Gen 1 to Gen 6. When a reset or hot plug is performed, then the speed is reset to maxRateChange (target speed) specified in the XML.

Cycle Accurate Mode Support

Cycle Accurate Mode is supported with Virtual Host PCIe and CXL to obtain the predictable response time for Upstream requests and Downstream requests.

By default, Cycle Accurate Mode is disabled. To enable Cycle Accurate Mode, add the following in the config.xml:

```
<vHost:PCIe><vHost:EnableCycleAccurateMode>true</vHost:EnableCycleAccurateMode></vHost:PCIe>
```

For more information, see [Cycle Accurate Mode Support](#).

Configurable Delay in Cycle Accurate Mode

In Cycle Accurate Mode, the clocks are interrupted (as soon as the transactors TX (xmit) buffer becomes empty), when an UP MemRd request is received. The clocks start again as soon as it receives the UP MemRd Response. In the real system, there is a clock delay. You can mimic this scenario by adding the required clock cycle delay to the Upstream Read response data path and the configurations to the ProxyEnv XML.

In Cycle Accurate Mode, there is no clock delay upstream read request. To mimic real system, you can add an additional clock delay to upstream read request by specifying the UpRespClkDelay in XML as following:

```
<vHost:EnableCycleAccurateMode>true</vHost:EnableCycleAccurateMode>  
<vHost:UpRespClkDelay>500</vHost:UpRespClkDelay>
```

For more information, see [Configurable Delay in Cycle Accurate Mode](#).

Enable Tx/Rx Payload Dumping

Tx and Rx dumping can be configured by setting the `<vHost:EnableTxPyldDump>` and `<vHost:EnableRxPyldDump>` parameters in the XML file as shown in the following example:

```
<vHost:ProxyXtor>
....
<vHost:Ethernet>
....
<vHost:EnableTxPyldDump>true</vHost:EnableTxPyldDump>
<vHost:EnableRxPyldDump>true</vHost:EnableRxPyldDump>
....
</vHost:Ethernet>
</vHost:ProxyXtor>
```

Enable Ethernet Loopback Mode

Loopback mode for VHS can be configured by setting the `<vHost:EnableLpbk>` parameter in the XML file as shown in the following example:

```
<vHost:ProxyXtor>
....
<vHost:Ethernet>
....
<vHost:EnableTxPyldDump>true</vHost:EnableTxPyldDump>
<vHost:EnableRxPyldDump>true</vHost:EnableRxPyldDump>
<vHost:EnableLpbk>true</vHost:EnableLpbk>
....
</vHost:Ethernet>
</vHost:ProxyXtor>
```

Working With the ProxyEnv XML GUI

ProxyEnv XML Graphical User Interface (GUI) is a utility created to streamline XML creation using a combination of the UI and ProxyVisor/Transactor schema.

The following sections explain the steps to configure a new XML file, generate the XML file, and modify an existing XML file using the ProxyEnv XML GUI.

- [Launch ProxyEnv XML GUI](#)
- [Configure an XML File](#)
- [Generate the XML File](#)
- [Modify an Existing XML File](#)

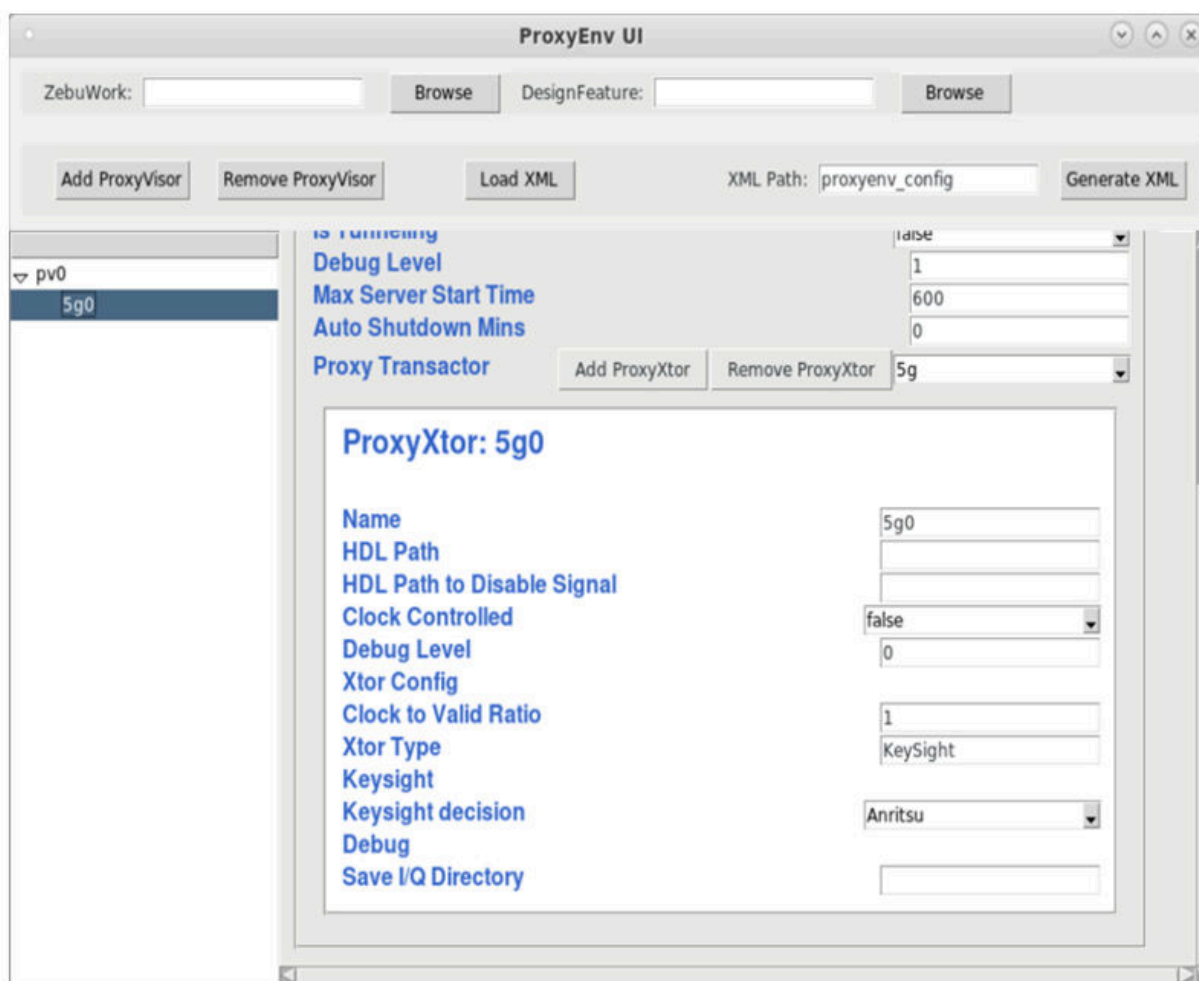
Launch ProxyEnv XML GUI

The environment variable required to launch the ProxyEnv XML utility is setup when you source the `$VSA_VHS_ROOT/setup.csh` file.

To launch the ProxyEnv XML UI, use the following command: `% zPython $VSA_VHS_ROOT/UI/bin/ProxyEnvUI.py.`

The UI is displayed as shown in the following figure:

Figure 5 ProxyEnv XML UI



This GUI enables you to do the following:

- Create XMLs via YAML schemas under `$VSA_VHS_ROOT/UI/schemas`
- Load a previously created XML, edit fields, and create a new XML

Configure an XML File

The ProxyEnv UI displays all required XML configuration fields. Some of these fields are auto filled; there are other mandatory fields like ZebuWorkPath and DesignFeaturesPath that you can update.

To add ProxyVisor, perform the following step:

1. Click the **Add ProxyVisor** button. The ProxyVisor name is displayed in the Tree view along with the card containing the ProxyVisor fields.

To add a ProxyXtor, perform the following step:

1. Click the **Add ProxyXtor** button. After selecting the preferred ProxyXtor from the drop-down list. Each ProxyVisor is required to have at least one ProxyXtor and each ProxyXtor is required to have an HDLPath.

Generate the XML File

After the required configuration fields are specified in the UI, the next step is to generate the XML file.

To generate the XML file, click the **Generate XML** button. Checks are performed to ensure all mandatory fields are filled out. Appropriate error messages are displayed if there are any missing fields.

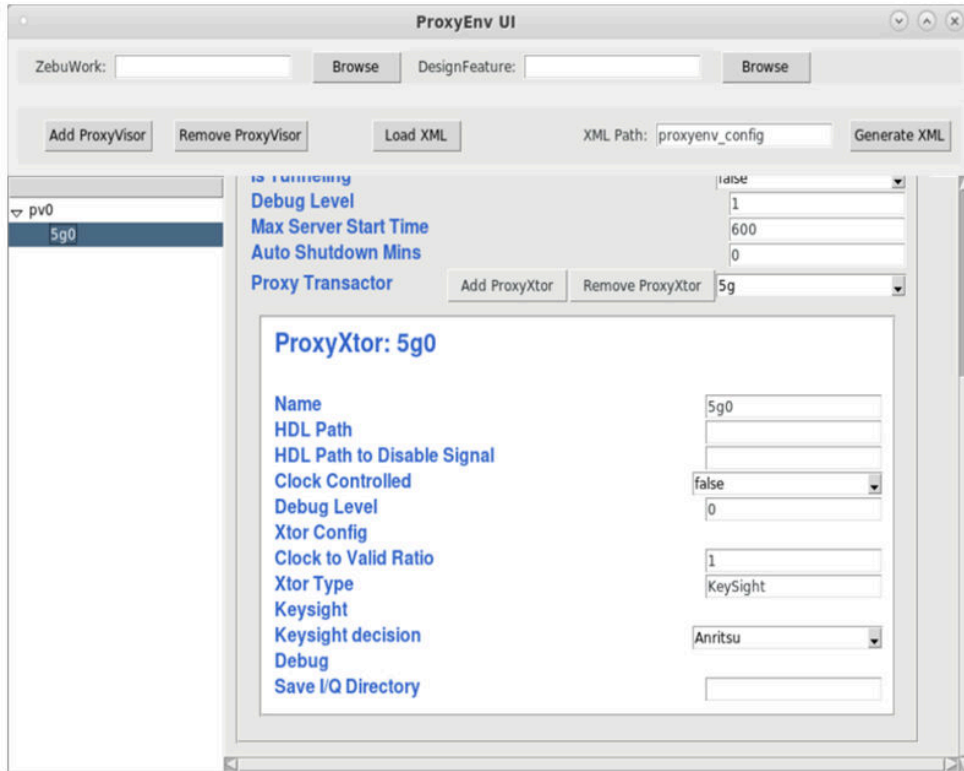
The XML is generated in the `$VSA_VHS_ROOT/UI/GenXML` directory with the standard/user provided name (by default). You have the option to provide a custom path from this given directory.

Modify an Existing XML File

The ProxyEnv UI can also load a previously developed XML. When loading, fields may be ignored or added based on fields present within the golden schemas. This loaded XML can be altered and updated through the UI.

To import an existing XML file, click the **Load XML** button as shown in the following figure:

Figure 6 *Modifying an Existing XML File*



After the file is selected, it is loaded in the UI. Any extra fields are ignored and any missing fields are auto filled based on the default schema values. If an error is encountered when loading the XML file, the error message is displayed in a pop-up window.

6

Protocol Specific Configurations

This section describes features and configurations of the following virtualized interfaces:

- [PCIe](#)
- [CXL](#)
- [USB](#)
- [Ethernet](#)
- [UFS](#)
- [PCIe End Point vNIC Solution](#)

PCIe

For PCIe-related features, see the following topics:

- [Active State Power Management](#)
- [Address Translation Service](#)
- [Advance Error Reporting](#)
- [Cycle Accurate Mode Support](#)
- [Configurable Delay in Cycle Accurate Mode](#)
- [Controlling Dynamic Lane Width](#)
- [Dynamic Link Speed Change](#)
- [Getting Handle to Transactor API](#)
- [In-Band Commands](#)
- [In-Band IDE Commands](#)
- [Memory Coalescing](#)
- [PCIe Hot Plug Support](#)

- [Plug Out and Plug-In a PCIe Device After the Initial VM Boot-up](#)
- [PCIe Reset Support](#)
- [Sending TLPs to Traffic Class](#)
- [Simulation Support for PCIe Transactor](#)
- [Enabling PCIe Protocol Analyzer from VHS](#)

Active State Power Management

Active State Power Management (ASPM) is a power management mechanism for PCIe devices to garner power savings while otherwise in a fully active state. VHS supports ASPM and it is disabled by default. One or more low power states can be activated by modifying the XML configuration file:

```
<vHost:PCIe>
  <vHost:PowerStates>
    <vHost:L0>false</vHost:L0>
    <vHost:L1>false</vHost:L1>
    <vHost:L1_1>false</vHost:L1_1>
    <vHost:L1_2>false</vHost:L1_2>
  </vHost:PowerStates>
</vHost:PCIe>
```

Address Translation Service

The Virtual Host Solution supports a limited form of the PCI Express Address Translation Service (ATS). The following limitations are imposed by the Virtual Machine, not the transactor or the solution itself:

- ATS is only supported on QEMU. It requires the following command-line arguments:

```
qemu-system-x86_64 -m 8192 -M q35,accel=kvm,kernel-irqchip=off -device
intel-iommu,intremap=on,device-iotlb=on ...
```

If the command-line arguments are not present, or VirtualBox is used, the solution hides the ATS Capability that should be present in the DUT. Therefore, it prevents ATS to be enabled.

- Translation units are 4096 bytes
- Invalidation requests are not supported
- PASID prefixes are not supported

Advance Error Reporting

VHS supports Advanced Error Reporting (AER). This can be enabled using automatic error probing mechanism or triggered explicitly through an in-band command from a VM.

The following XML configuration option needs to be enabled for automatic error probing:

```
<vHost:PCIe>
<vHost:Type>DPI</vHost:Type>
    <vHost:EnableAER>true</vHost:EnableAER>
    <vHost:HandleErrorMessages>true</vHost:HandleErrorMessages>
</vHost:PCIe>
```

From the Virtual Machine, the following in-band command can be triggered for AER:

```
setpci -s 01:00.0 0x08.1=0x16c306FF
```

For more details on all the in-band command options available for AER, refer to the section on In-Band commands.

VHS also provides a command line interface utility called `vsa-vhs-aer-cli` to display the all the errors reported. To exercise this utility, it should be picked up from `$VSA_VHS_ROOT/binABI1` and transferred inside the Virtual Machine using SharedFolder. The following command can be executed from inside the Virtual Machine:

```
vsa-vhs-aer-cli 1:00.0 status
```

The following XML configuration displays AER capability and enables error probing:

```
<vHost:PCIe>
<vHost:EnableAER>true</vHost:EnableAER>
</vHost:PCIe>
```

When the following XML option is enabled, upon receiving an ERROR message, VHS probes for the cause and reports the AER status:

```
<vHost:AutoAER>true</vHost:AutoAER>
```

It is implied that `enableAER` is false in this case.

Cycle Accurate Mode Support

Cycle Accurate Mode is supported with Virtual Host PCIe and CXL to obtain the predictable response time for Upstream requests and Downstream requests.

When Cycle Accurate Mode is enabled, Virtual Machine stops the hardware clock for a specific duration to process all the Upstream requests that consume equal number of

clocks to get responses. Similarly, Virtual Machine issues the requests sequentially after receiving the previous responses from DUT to process the Downstream requests. The maximum pipelined downstream request is only one so that all downstream requests gets the same amount of clock to receive the responses.

By default, Cycle Accurate Mode is disabled. To enable Cycle Accurate Mode, add the following in the `config.xml`:

```
<vHost:PCIe><vHost:EnableCycleAccurateMode>true</vHost:EnableCycleAccurateMode></vHost:PCIe>
```

When Cycle Accurate Mode is enabled, there is a decrease in delay values and constant delay values are observed for upstream reads with debug level 1 or more.

Configurable Delay in Cycle Accurate Mode

In Cycle Accurate Mode, the clocks are interrupted (as soon as the transactors TX (*xmit*) buffer becomes empty), when an *UP MemRd* request is received. The clocks start again as soon as it receives the *UP MemRd* Response. In an ideal scenario, when the TX (*xmit*) buffer is empty (when there is no pending downstream activity), it is a zero delay response for the upstream read request.

However, in cycle accurate mode, there is no clock delay upstream read request. To mimic real system, you can add an additional clock delay to upstream read request by specifying the *UpRespClkDelay* in XML as following:

```
<vHost:EnableCycleAccurateMode>true</vHost:EnableCycleAccurateMode>  
<vHost:UpRespClkDelay>500</vHost:UpRespClkDelay>
```

Note:

The preceding setting ONLY guarantees a minimum user input clock delay and it does not guarantee the exact user input clock delay.

When Cycle Accurate Mode is enabled, the accurate time stamping feature is enabled and the log captures the added clock cycle delay between the request and response.

If the *UpMemRd Resp* cycle goes beyond the user input *UpRespClkDelay* cycles, the following warning is displayed.

```
"Clock cycles have advanced more than the user specified delay by xxx  
cycles in the upstream response path"
```

A dynamic control knob is provided for changing the *UpRespClkDelay* cycles. It can be used in the *ProxyEnv* testbench as follows:

```
ProxyPCIeXtor *pci0 = env->getProxyPCIeXtor("pci0"); //Name 'pci0' should  
match the name in config.xml
```

```
pci0->upRespClockDelay((int)500); //It will add 500 cycles to each  
upstream response
```

Controlling Dynamic Lane Width

VHS supports validating change in the lane width of a PCIe transactor. To do so, use the following API:

```
bool changeLink(uint32_t width, uint32_t speed);
```

If speed = 0, only link width is changed. Valid link width values are 0, 1, 2, 4, 8 and 16. width = 0 implies maximum width.

Dynamic Link Speed Change

VHS supports PCIe dynamic link speed change. Note that speed change is initiated only by RC.

To dynamically validate the change in speed, use the following API:

```
/bool changeLink(uint32_t width, uint32_t speed);/
```

This API is used to dynamically change the PCIe link width and speed. If width is set to 0, then only speed change is performed.

Valid values for changing speed are 0-6, where 0 implies that no speed change is done. Any value between 1 and 6 represents the corresponding Gen speed, that is, from Gen 1 to Gen 6. When a reset or hot plug is performed, then the speed is reset to maxRateChange (target speed) specified in the XML.

Getting Handle to Transactor API

VHS adds a proxy layer wrapper over native transactor APIs and therefore transactor APIs are not accessible when using VHS. However in rare cases when exercising transactor API becomes imperative, VHS provides two special APIs to do so. In order to use them, VHS team needs to be contacted for receiving the “password” to unlock the APIs. There are two such APIs called setMagicFlags and getMagicInfo. setMagicFlags API helps in dumping transactor configurations, enabling interrupt callbacks, generating fake memory I/O traffic and using native transactor API to call different Memory, Configuration and I/O transactions.

void setMagicFlags(uint64_t password); GetMagicInfo API helps in getting a handle of the transactor API

```
void* getMagicInfo(uint64_t passwd);
```

In-Band Commands

In-band commands use a regular PCIe transaction to modify or trigger some behavior in VHS. They can be issued by test programs, scripts or command line as required to help debug a problem or create the necessary situation to verify a design. Issuing these commands from inside a VM is the most common use-case. In VHS all in-band commands are configuration writes of a 4-byte value 0x16C3XXYY to a VHS root complex class code register (at address 0x008). The value "XX" determine the objective of the in-band command and the value "YY" is an objective-specific argument.

Example:

```
Example: % setpci -s 1:00.0 0x008.l=0x16C30001
```

Address	Value	To RC (VHS)
0x008	0x16C300YZ	Set ProxyPCIeXtor debug level to Z. Set XTOR debug level to Y.
	0x16C301YZ	
	0x16C302YY	Control collection of PCIe/CXL performance metrics, where "YY" is: • "AA": Enable collection. • "BB": Report metrics to date. • "EE": Report metrics to date and disable collection. • "FF": Report and reset metrics, then disable collection.
	0x16C303YY	N/A
	0x16C304YY	Set the Traffic Class for all subsequent downstream memory requests to "YY".
	0x16C30510	N/A

Address	Value	To RC (VHS)
	0x16C306YY	Control AER probing, where "YY" is: • "00": Turn off automatic probing on error message • "01": Disable DPC and turn on automatic probing on error message • "02": Disable DPC • "AA": Set the target function to manually probe. The next two "YY" values are bus and dev.fn numbers respectively. Default is function 00.0 on the immediate secondary bus. To probe the RC XTOR, use target function 00:00.0. The target function is ignored by the automatic probing as it probes the function that issued the message. • "BB": Probe the target function. Probing the same function again does not need setting the target function. • "FF": Probe the target function and clear all error status bits For example, to manually probe the RC itself then clear the errors, write the following sequence: 0x16C306AA, 0x16C30600, 0x16C30600, 0x16C306BB, 0x16C306FF
	0x16C307YY	N/A

In-Band IDE Commands

Whenever it becomes useful to offer some runtime user controllability on IDE features that are not directly controllable via a register in the IDE Extended Capability, in-band IDE commands can be executed by writing the value `0x16C3xxxx` to the IDE Capability Register at offset 04h in the IDE Extended Capability of the RC function. The value `xxxx` identifies the specific in-band command to execute.

The IDE Capability register should normally be located at address `0x0450` in the RC function's configuration space, but it is recommended that the actual address of the register be located by looking for the IDE Extended Capability starting at address `0x0100` and following the linked list until the Extended Capability ID `0x0030` is found.

Memory Coalescing

By default, individual memory write transactions sent by a Virtual Machine to the PCIe interface are executed atomically. In certain circumstances, multiple memory write transactions can be combined into a larger one.

By default, combining memory write transactions is disabled.

To enable Memory Write combining for a particular PCIe device, add the "*CombineMemWrites*" option to the *pcie-zebu-ep* instance specification on the command line, as follows:

```
% qemu-system-x86_64 ... -device  
pcie-zebu-ep ,...,CombineMemWrites=0xNNNN ...
```

Note:

Memory Write combining is disabled by default.

To enable ProxyPing to automatically combine Memory Writes, use *-c* as follows:

```
ProxyPing -Q [Options] -c n
```

where,

-c n: Enables *CombineMemWrites* with the given value of *n*, where, *n* is a HEX value. The option automatically combines Memory Write on all resources within a device. The utility automatically adds *CombineMemWrites=1* to the *QemuRun.args* file.

Use Model

To enable Memory Write combining on all resources within a device, set the *CombineMemWrites* option to *0x0001*. To enable Memory Write combining only on specific BAR regions, set bits 2..7 and 8..13 to '1'. This enables combining in BAR0..BAR5 and SRIOV BAR0..BAR5. For example, value *0x31* enables combining for BAR2 and BAR3 exclusively, and value *0x3FFD* is functionally equivalent to value *0x0001*.

To confirm if the Memory Write combining is enabled, check if the following messages are generated by the VM while it is booting:

```
pcie-zebu-ep: Mapping non-prefetchable 32bit MEM BAR0 region,  
size=0x00001000  
WARNING: pcie-zebu-ep: MemWrite Combining enabled on non-prefetchable  
BAR[0].  
pcie-zebu-ep: Mapping prefetchable 32bit MEM BAR1 region, size=0x00004000  
pcie-zebu-ep: MemWrite Combining enabled on BAR[1].  
pcie-zebu-ep: Mapping prefetchable 64bit MEM BAR3/2 region,  
size=0x00004000  
pcie-zebu-ep: MemWrite Combining enabled on BAR[2].
```

Note:

A warning is displayed if Memory Write combining is enabled on non-prefetchable resources.

After Memory Write combining is enabled, a sequence of Memory Write transactions are combined into a single larger Memory Write transaction (continue to be buffered instead of executed), only if the memory write transactions are:

- to increase consecutive addresses, without any gaps in the data
- within the same BAR region
- within the same 4K physical address page
- not interrupted by a Memory Read transaction, and
- the combined memory write does not exceed the configured maximum payload size of the PCIe link leading to the device.

If the aforementioned conditions hold true, Memory Write transactions are buffered for eventual execution as a single larger Memory Write transaction. These combined Memory Write transactions remain buffered and unexecuted as long as no memory transaction violating the rules aforementioned is requested. If one of the following requests are made, all the pending combined Memory Write requests are executed first, followed by the latest Memory transaction:

- Memory Write to a non-sequential address is requested
- Memory Write to a different BAR region is requested
- Memory Write to a different 4K page is requested
- Memory Read transaction is requested
- The amount of buffered data equals or exceeds the configured MPS

Example

For example, consider the following memory transaction sequence:

```
MemWr.L BAR[0]+0x0FF8
MemWr.L BAR[0]+0x0FFC
MemWr.L BAR[0]+0x1000
MemWr.L BAR[0]+0x1004
MemWr.L BAR[0]+0x0000
MemWr.L BAR[0]+0x0004
MemWr.L BAR[1]+0x0000
MemWr.L BAR[1]+0x0004
MemRd.L BAR[1]+0x0008
MemWr.L BAR[1]+0x0008
MemWr.L BAR[0]+0x0000
```

Enabling memory integration results in the following transactions executed on the PCIe device:

```
MemWr.Q BAR[0]+0x0FF8
MemWr.Q BAR[0]+0x1000 ;; New 4K page
MemWr.Q BAR[0]+0x0000 ;; Not consecutive
MemWr.Q BAR[1]+0x0000 ;; Different BAR
MemRd.L BAR[1]+0x0008 ;; MemRd
MemWr.L BAR[1]+0x0008
```

Note that the last transaction (**MemWr.L BAR[0]+0x0000**) is not executed because it is still in the combination buffer and a condition to flush it has not yet been encountered. At this time, the conditions outlined above are the only ones that can cause flushing of the combined Memory Write buffer.

If a VM is shutdown while there are pending Memory Write requests, a warning message is generated. If a snapshot of a VM is taken while there are pending Memory Write requests, an error occurs.

PCIe Hot Plug Support

PCIe Hot Plug allows an endpoint to be inserted or removed from a PCIe system without special consideration.

There are two use models supported and the support is available only with DPI transactor.

- Starting the VM from an unplugged state of a PCIe device in the design.
- Plug out and plug-in a PCIe device after the initial VM startup.

In the case of emulation, these events can typically be mimicked by putting the device under reset (Plug-out/unplugged state) and bringing the device out of reset (Plug-in/Plugged state).

Starting the VM From an Unplugged State of a PCIe Device

Starting the VM from an unplugged state in the Virtual Environment requires the user to populate `probedBAR[]` of the `UnpluggedCfg_s` structure in the user testbench.

Example

```
ProxyPCIeXtor::typeCfg_u::UnpluggedCfg_t
unPlug;
unPlug.probedBAR[0] = 0xFFFF0000C;
unPlug.probedBAR[1] = 0xFFFFFFFF;
unPlug.probedBAR[2] = 0xFFFF00000;
unPlug.probedBAR[3] = 0x000000000;
unPlug.probedBAR[4] = 0xFFFFFFFF01;
unPlug.probedBAR[5] = 0x000000000;
cfg.startUnplugged(0, cfg.unPlug);
```

ProxyEnv

Add `ProxyEnv` to `config.xml` to update it.

Example

An example of `config_lpcie_dpi.xml` is updated and shown as follows:

```
<vHost:Unplug>
<vHost:IsEnabled>true</vHost:IsEnabled>
<vHost:ProbedBAR0>FFF0000C</vHost:ProbedBAR0>
<vHost:ProbedBAR1>FFFFFFFF</vHost:ProbedBAR1>
<vHost:ProbedBAR2>FFF00000</vHost:ProbedBAR2>
<vHost:ProbedBAR3>00000000</vHost:ProbedBAR3>
<vHost:ProbedBAR4>FFFFFF01</vHost:ProbedBAR4>
<vHost:ProbedBAR5>00000000</vHost:ProbedBAR5>
<vHost:SRIOV>
<vHost:TotalVFs>0</vHost:TotalVFs>
<vHost:FirstVF>0</vHost:FirstVF>
<vHost:StrideVF>0</vHost:StrideVF>
<vHost:ProbedBAR0>0</vHost:ProbedBAR0>
<vHost:ProbedBAR1>0</vHost:ProbedBAR1>
<vHost:ProbedBAR2>0</vHost:ProbedBAR2>
<vHost:ProbedBAR3>0</vHost:ProbedBAR3>
<vHost:ProbedBAR4>0</vHost:ProbedBAR4>
```

```
<vHost:ProbedBAR5>0</vHost:ProbedBAR5>
</vHost:SRIOV>
</vHost:Unplug>
```

When you start the testbench, you can see the following as soon as ZeBu is loaded.

```
+-- 1:00.0: Endpoint UNPLUGGED -> Dev=0x5fff Frm=0x16c3

-Device ID: 0x5fff (dummy device ID)
-Vendor ID: 0x16c3 (Synopsys) for the dummy device
```

Note:

It does not expect the PCIe to link-train since it is started in the unplugged state.

The device ID and vendor ID are fixed for the dummy device, which represents the unplugged state. When the VM starts, *lspci* lists this dummy device but the BARs assigned to this device reflect the original PCIe design's BAR configuration.

Any access to the EP at this stage returns as not accessible but VM is completely functional.

When the link-training is established in the design, the transactor automatically completes the speed up, enumeration, and displays the correct *vendorID* and *deviceID* as per customers design.

To rediscover and load the device driver, use the following commands from the VM:

```
% echo 1 > /sys/bus/pci/devices/0000\:01\:00.0/remove
% echo 1 > /sys/bus/pci/rescan
% lspci -v -s 01:00.0
```

Note:

Assuming the PCIe device is mapped at 01:00.0 (B:D:F).

The *lspci* output should match the customer PCIe EP design specification.

Plug Out and Plug-In a PCIe Device After the Initial VM Boot-up

In this use model, start the VM from PCIe Plugged state. After the VM/guest machine boots up, you can mimic the plug-out event by applying device reset on the DUT side. Any Cfg/Mem/I/O access to the EP device is controlled and you should clearly see the message “EP is unplugged”.

Note:

You can access Root-Port/Bridge/RC.

When the user brings the EP from reset, link-training, speed up and device enumeration happens automatically and you can access the EP device.

PCIe Reset Support

The following different resets are supported:

- VM Reset/Reboot
- Hot Reset
- Link-Ctrl Reset

In the case of multiple VM use model, all the 3 resets are supported from each VMs.

VM Reset/Reboot

VM reset/reboot can typically be issued for any software related issue. This is similar to restarting a host machine in the event of any system freeze, and so on. To initiate the startup process, specify the following from the VM/Guest machine:

```
% sudo reboot
```

After the command is specified, the link-training, speed-up and re-enumeration happens on the DUT side and VM must reboot after performing the OS re-enumeration and driver load.

Hot Reset

Hot-Reset is a mechanism to reset the PCIe device by programming the Root port's reset bit of the bridge *ctrl* register. To achieve hot reset, use the following programming sequence from the VM/Guest machine:

```
% setpci -s 00:10.0 0x3e.w=0x40:0x40 (Reset the PCIe)
% setpci -s 00:10.0 0x3e.w=0x0:0x40 (Bring the PCIe out of reset. Re-link
  training, speed and enumeration happens automatically)
```

Note:

VM remains active with hot reset and there is no change in the machine state.

Link-Ctrl Reset

Link-Ctrl reset is another mechanism to reset the PCIe device by programming the Root port's reset bit of the link-ctrl register. To achieve link-ctrl reset, use the following programming sequence from the VM/Guest machine.

```
% setpci -s 00:10.0 0x80.w=0x10:0x10 (Reset the PCIe)
% setpci -s 00:10.0 0x80.w=0x0:0x10 (Bring the PCIe out of reset. Re-link
  training, speed and enumeration happens automatically)
% echo 1 > /sys/bus/pci/devices/0000\:01\:00.0/remove
% echo 1 > /sys/bus/pci/rescan
```

Note:

VM remains active with link-ctrl reset and there is no change in the machine state.

Sending TLPs to Traffic Class

By default, VHS sends all request TLPs using Traffic Class 0. If the IDE stream uses a different Traffic Class, you must change the default traffic class using an in-band IDE command, writing the value 0x16C304cc where “cc” is the required Traffic Class value (0..7) to the Class Code register (offset 0x08) in the configuration space of the PCIe RC function that corresponds to the PCIe transactor. All subsequent memory requests through that RC are sent (and responses expected) on that Traffic Class.

For example, if the PCIe transactor is mapped to the RC in slot 0:03.0, use the command `setpci-s 0:03.0 0x08.l=0x16C30407` to send all subsequent memory requests on Traffic Class 7.

Simulation Support for PCIe Transactor

VHS supports simulation mode for PCIe DPI transactor. To invoke ProxyEnv testbench API during the simulation, use the import calls from DUT top.

For example, following are the general import calls, which can be invoked from DUT and to be implemented in ProxyEnv testbench:

```
import "DPI-C" context task run_init(); // To initialize the testbench
import "DPI-C" context task run_rc(); // To run RC setup (ProxyEnv)
import "DPI-C" context task run_ep(); // To run EP setup if needed
```

Note:

To avoid abnormal VM behavior or VM time out issues, you might need to increase the timeout of the software driver running inside the VM because the simulation setup is slow.

Example

To understand how to invoke the import calls from DUT top, use the example available at `Examples/PCIe/simulation/src/dut/wrapper.v`.

```
Examples/PCIe/simulation/
Makefile
src
    bench
        proxyTBEnv.cc
    dut
        ep_dut
            ep_dut.sv
            rc_xtor_ep_dut_tbench.v
            wrapper.v
env
    example_wrapper.utf
rc_xtor_ep_dut.utf
vcs
    Makefile
```

To implement the import calls in ProxyEnv, use the example available at `Examples/PCIe/simulation/src/bench/proxyTBEnv.cc`.

Enabling PCIe Protocol Analyzer from VHS

VHS solution supports enabling the ZDPI-based PCIe monitor. To enable and compile the ZDPI-based PCIe monitor in ZeBu compilation before you enable it from VHS, see the *ZeBu PCIe Transactor User Guide*.

Enable the Protocol analyzer in XML by adding following lines in the XML:

```
<vHost:ProxyXtor>
  .
  .
  <vHost:PCIe>
    .
    .
    <vHost:Debug>
      <vHost:ProtocolTrace>
        <vHost:IsEnabled>true</vHost:IsEnabled>
        <vHost:HdlInst>hdl path of pcie monitor</vHost:HdlInst>
        <vHost:ManualMonitorStart>>false</vHost:ManualMonitorStart>
      </vHost:ProtocolTrace>
    </vHost:Debug>
    .
    .
  </vHost:PCIe>
  .
  .
</vHost:ProxyXtor>
```

where,

the `HdlInst` represents the hierarchical path of PCIe monitor in ZeBu compilation.

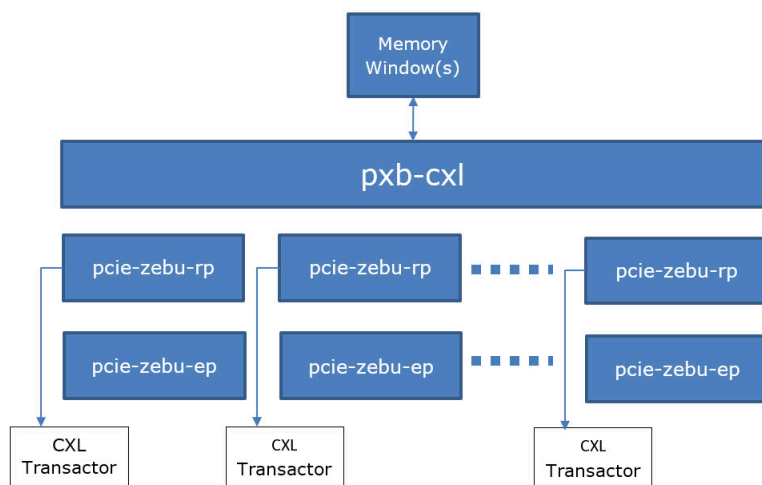
After enabling the protocol analyzer in XML, during the emulation run, a ZTDB file gets generated with `post_procdumpfile_0.ztdb` in the run area.

For more information on converting the ZTDB file into FSDB to view in Verdi protocol analyzer or a text file to view the traces, see the post-processing mechanism defined in the *ZeBu PCIe Transactor User Guide*.

CXL

When *ProxyPing* identifies the presence of a CXL device on a CXL link, it configures QEMU with the following device structure for each CXL link:

Figure 7 Device Structure of Each CXL Link



By default, there is one non-interleaved memory window per link large enough to contain both memory ranges and all 10 HDM decoder ranges in the CXL device on that link. You can specify up to eight memory windows located at different base addresses and sizes. All accesses in those memory windows are translated into `CXL.mem` transactions, executed on a CXL transactor. The `CXL.mem` accesses are claimed by the CXL device if the address maps to either of the memory ranges or to one of the HDM decoder, whichever is enabled. If the address is not claimed by the device, no response is provided.

The base address of the default memory window can be specified on the `ProxyPing` command line. The size of the window is automatically determined to contain all possible decoder ranges. By default, corresponding base addresses are specified for each range (as well as size and address skip in HDM decoders) are automatically written by QEMU to the appropriate range or HDM registers, before the UEFI or BIOS code is executed in the Linux startup sequence.

For more information, see the following topics:

- [System Configuration](#)
- [Memory Windows](#)
- [Memory Decoder Base Addresses](#)
- [CXL.mem Transactions](#)

System Configuration

The location and size of the CXL memory windows are critical configuration elements to ensure functional correctness of the VHS integration. All memory accesses into those windows are forwarded to the CXL link for completion by the CXL device. It is important that the location and size of those windows match with the guest OS or application software. How to specify the location and size of the CXL memory windows to the guest OS or application is outside the scope of the VHS solution, and must be accomplished using whatever mechanism each provides. Each programmed address decoder range must lie entirely within a window.

ProxyPing provides a mechanism for creating a default system configuration that uses a single window and assigns memory decoder ranges using a simple-minded continuous address allocation mechanism. No attempt is made to minimize the size of the window or the addresses within a window that fall outside of decoder ranges. That configuration is only a default. A different configuration can be specified using different command-line arguments.

Memory Windows

The location and size of memory windows is specified using pairs of 'window-<i>-base[=0x<baseAddr>' and 'window-<i>-size=<size>' properties on the 'pxb-cxl' port expander device. Up to 8 windows can be specified, where <i> is one of 0 through 7. For example, the following command-line options configures a port expander device with two windows. The windows can then be mapped to different ports and different memory decoder ranges.

```
-device
  pxb-cxl,id=cxl.io.0,bus=pcie.0,bus_nr=0x80,uid=0,window-0-base=0x10000000
  0000,window-0-size=512M,window-1-base=0x200000000000,window-1-size=4G
```

ProxyPing always specifies only one window, large enough to contain all `CXL.mem` resources (Range1 and Range2 decoder sizes, and DSMAS entries in CDAT) found in the devices. The base address of that window is specified using the '-X A<address>' option to *ProxyPing*. This is shown using the following command:

```
% ProxyPing ... -X A0x100000000000 ...
May generate a CXL memory window configuration like this one:
-device pxb-cxl,...,window-0-base=0x100000000000,window-0-size=8G
```

It is important that the CXL memory windows do not overlap each other, or with the PCIe memory resources assigned via the BAR registers.

Note:

The code to make the OS aware of the memory windows is not yet available. However, even though the range is not visible to the OS, the memory device works if memory accesses within the physical address range are performed.

Memory Decoder Base Addresses

The memory decoder base addresses (as well as size and address skip in the case of the HDM decoders) can be programmed at runtime by the BIOS, guest OS, or application software. Each memory decoder range must be configured to lie entirely within one of the memory windows.

If the BIOS, guest OS, or application do not yet support programming of the memory decoders, it is possible to have the Range 1 and Range 2 base addresses or the HDM decoders base, size, skip, and control registers automatically programmed by the VHS integration during initialization. The memory range base addresses are optionally specified using the '*cxl-range-1-base=0x<baseAddr>*', the '*cxl-range-2-base=0x<baseAddr>*' properties on the CXL device. If at least one base address is provided, *CXL.mem* is also automatically enabled.

If the CXL 2.0 device supports the HDM decoder capability, values for the base, size, skip, and control registers can be similarly optionally specified using the and the '*cxl-hdm-<i>-base=0x<baseAddr>*', '*cxl-hdm-<i>-size=0x<size>*', '*cxl-hdm-<i>-skip=0x<size>*' and '*cxl-hdm-<i>-ctrl=0x<IWIG>*' properties on the device and automatically programmed and committed by the VHS integration during initialization.

If at least one HDM decoder configuration property is specified, the HDM decoders are enabled, superseding the Range 1 and Range 2 decoders. It is up to the user to ensure that the specified base, size, skip, and interleaving control values fall within the CXL port expander memory windows, are respectively coherent as described in the CXL 3.0 Specification. Further, if DSMAS regions are specified in the CDAT table for the device, the decoded physical address as described in section 8.2.5.12.21 must fall within one of those physical regions.

ProxyPing always specifies proper base addresses for the Range 1 and Range 2 decoders aligned for their respective sizes. If DSMAS CDAT entries are reported by the device, they take precedence over Range decoders, and *ProxyPing* maps one HDM decoder to an entire DSMAS entry without any interleaving. These automatically-generated addresses and size can be changed at runtime or deleted from the command-line option generated by *ProxyPing*. For example, the following command:

```
% ProxyPing ... -X A0x10000000000 ...
```

may generate these automatic base addresses located within the default generated CXL memory window, based on the size of each decoder and the absence of DSMAS CDAT entries.

```
-device  
pcie-zebu-ep,...,cxl-range-1-base=0x100000000000,cxl-range-2-base=0x10100  
000000
```

If a single DSMAS CDAT entry is reported by the device, it may generate this automatic base address and size located within the default generated CXL memory window, based on the physical address and size of the DSMAS entry.

```
-device  
pcie-zebu-ep,...,cxl-hdm-0-base=0x100000000000,cxl-hdm-0-size=0x4000000000  
0,cxl-hdm-0-skip=0x000000000000
```

CXL.mem Transactions

QEMU does not model the host cache so all MMIO transactions are 8 bytes or less. This translates to partial *CXL.mem* read and writes. To avoid always performing a *CXL.mem* operation for every MMIO operation, a single 64-byte cache line model is included in each QEMU CXL memory window.

The entire line is read the first time a non-valid data byte is read. The entire line is written if it is replaced by a different line and it contains written bytes, or when the last byte in the line is written and all of the bytes have been written. This allows full *CXL.mem* data transactions to be performed even with partial MMIO transactions. To create partial *CXL.mem* transactions, you only need to ensure that a transaction in a different cache line in the same memory region is performed next to flush the content of the cache line in the memory region.

USB

For USB-related features, see the following topics:

- [Supporting USB3 Transactor Monitor](#)
- [Enhanced USB Support for VHS](#)
- [Enhanced APIs for U1/U2 Timer Feature in VHS](#)
- [Enhanced Suspend and Resume APIs for USB Transactor in VHS](#)
- [USB Device Hot Plug Support](#)
- [Initiating USB Speed Change](#)

Supporting USB3 Transactor Monitor

A USB transactor monitor analyzes the USB physical channel traffic and reports it in the form of log and PA FSDB (graphical packet format) to easily understand the UTMI/PIPE traffic.

With VSA S-2023.03-SP1, you can generate the USB3 monitor data for VHS through `monitor_usb_svs` transactor. To do this, add the following options in the XML configuration file as follows:

```
<vHost:USBMonitorXtor>
  //Set to true to enable the monitor and to false to disable it
  <vHost:IsEnabled>true</vHost:IsEnabled>
  //Specify the name of the monitor file
  <vHost:FileName>USBXtorMonitor</vHost:FileName>
  //Specify the name of the instance in the DUT
  <vHost:HdlInst>hw_top.top1.mon</vHost:HdlInst>
  //Specify whether the fsdb file is required or not
  <vHost:CreateFSDB>true</vHost:CreateFSDB>
  //The following options are specific to USB3 transactor
  <vHost:ProgName>NULL</vHost:ProgName>
  <vHost:AutoInc>false</vHost:AutoInc>
  <vHost:Debug>false</vHost:Debug>
  <vHost:MonitorType>true</vHost:MonitorType>
  <vHost:Descramble>false</vHost:Descramble>
  <vHost:Lane1>false</vHost:Lane1>
</vHost:USBMonitorXtor>
```

Note:

The current release supports limited functionality of USB3, which includes validating multiple instances of USB3XDPI for USB31 DPI and bulk transfer for USB31 and USB32 with various speeds including SSP, SS, HS, and FS.

For additional details about USB3 transactor, refer to the *ZeBu USB Transactor User Manual*.

Enhanced USB Support for VHS

The following USB features are available in VHS:

- **Link Up and Enumeration at Different Speeds**

USB link up and enumeration is available at different speeds: HS, SS, SSP in VHS. To use this feature, perform the following steps:

1. Configure speed on the host and device.
2. Start ProxyXtor.

3. Start the VM.
4. Check for the device connect status.
5. Check the device descriptors and information using the `lsusb` utility on the VM.

To check the device enumeration, run the “`lsusb`” utility.

- **Control and Bulk Transfers**

To use this feature, perform the following steps:

1. Configure the host and device.
2. Start ProxyXtor.
3. Start the VM.
4. Check the device descriptors and device information.
5. Initiate the control and bulk transfers.

`Interrupt` and `Isoc` are not supported through QEMU because of its implementation.

To check the implementation of this feature:

- Run the “`lsusb`” utility to check device enumeration.
- Check the bulk and control transfers printed in the log file. For each transfer, there is a unique ID. Use this ID to check ACK packet to confirm successful transfer.

- **Repeated Connect and Disconnect Cycles**

To initiate repeated connect and disconnect cycles, perform the following steps:

1. Configure Host and the device.
2. Run the test on ZeBu.
3. Start the VM.
4. Check for basic enumeration.
5. Initiate the device disconnect.
6. Initiate the device connect.
7. Restart the VM to load the new device configuration.

8. Initiate control and bulk transfers after startup.

Note:

This sequence is repeated multiple times using the same speed or with different speeds.

To check successful transfers, after each disconnect or connect cycle, use the common checks for enumeration and bulk transfers. For speed change, check the configuration printed in the log file followed by the bus ID and device ID that is printed after enumeration.

In repeated connect/disconnect cycles, perform the following steps to avoid restarting QEMU:

1. Disable the device from the VM after the device is disconnected. Use the command:
`cd /sys/bus/usb/drivers/hub`. Here, you see two entries:

- 1-0:1.0 - This is for the ehci controller.
- 2-0:1.0 - This is the XHCI controller.

2. Unbind the device under XHCI:

```
echo "2-0:1.0" > unbind ;
```

3. After the device is reconnected:

```
echo "2-0:1.0" > bind ;
```

Note:

** This works only when the device is reconnected with the same speed, because the internal data structure are remains the same because VM is still open.

• **Enumerate Multiple Times During Startup (POR)**

To enumerate multiple times during startup (POR), perform the following steps:

1. Configure Host and the device.
2. Run the test on ZeBu.
3. Start the VM.
4. Check for basic enumeration.
5. Initiate the bulk transfer.
6. Initiate the device disconnect.
7. Initiate the device connect.

8. Restart the VM to load the new device configuration.
9. Initiate control and bulk transfers after startup.

Note:

Sequence can be repeated multiple times using the same speed while disconnecting & reconnect the device again..

To check the implementation of this feature:

- Use common checks for enumeration and bulk transfer after each disconnect or connect cycle.
- For speed change, check the configuration printed in the log file. After enumeration, check the bus ID and the device ID.

- **Enumerate Multiple Times During Startup (Protocol Reset)**

To enumerate multiple times during startup (protocol reset), perform the following steps:

1. Configure Host and the device.
2. Run the test on ZeBu.
3. Start the VM.
4. Check for basic enumeration.
5. Initiate the bulk transfer.
6. Enable warm/hot reset by calling the `enableDeviceReset()` API.
7. Initiate reset from VM (through `do_reset_dev` binary).
8. After successful reset, initiate control and bulk transfers after startup.

To check the implementation of this feature:

- Use common checks for enumeration and bulk transfer after each disconnect or connect cycle.

- **Suspend and Resume**

There are two ways to resume:

Resume via remote-wakeup: To do this, perform the following steps:

1. Initiate U3 suspend from host by calling the `suspend()` API. Wait for the link to reach U3 state.
2. Remote wakeup by writing into GCTL reg. Wait for link to reach U0 state.
3. Initiate bulk transfer.

Resume via host: To do this, perform the following steps:

1. Initiate U3 suspend from host by calling the `suspend()` API. Wait for link to reach U3 state.
2. Resume via host by calling the `resume()` API. Wait for the link to reach U0 state.
3. Initiate bulk transfer.

To check the implementation of this feature:

- Use the common checks for enumeration and bulk transfer.
- For suspend, check the link state transition to U3 in the log file. Also, check for the `Suspend Event`.
- For resume via host, check the device state transition from U3 to U0 in the log file. Also, check for the `Wakeup Event`.
- For resume via remote wakeup, check the device state transition from U3 to U0 in the log file. Also, check for the GCTL register access in the log file.

Note:

Suspend and resume is supported using a transactor handle in the VHS environment.

- **L1 Suspend**

There are two ways:

Resume via remote-wakeup: To do this, perform the following steps:

1. Initiate L1 suspend from host. Wait for the link to reach L1 state.
2. Resume L1 from host. Wait for link to reach L0 state.
3. Initiate bulk transfer.

Resume via host: To do this, perform the following steps:

1. Initiate L1 suspend from host. Wait for link to reach L1 state.
2. Initiate remote wakeup by writing to GCTL reg. Wait for the link to reach L0 state.
3. Initiate bulk transfer.

To check the implementation of this feature:

- For L1 suspend, check the link state transition to L1 in the log file. Also, check for the `Suspend Event`.
- For resume via host, check the device state transition from L1 to U0 in the log file. Also, check for the `Wakeup Event`.
- For resume via remote wakeup, check the device state transition from L1 to U0 in the log file. Also, check for the GCTL register access in the log file.

- **U1/U2 Timers**

To enable only the `U1` timer, perform the following steps:

1. Configure the timeout value. Wait for the link to reach U1 state.
2. Initiate the bulk transfer to exit U1 state to active U0 state.

To enable only the `U2` timer, perform the following steps:

1. Configure the timeout value. Wait for the link to reach U2 state.
2. Initiate the bulk transfer to exit U2 state to active U0 state.

To enable both `U1` and `U2` timer, perform the following steps:

1. Configure the timeout value. Link should transition as { `U0 ->U1->U2`}.
2. Initiate the bulk transfer to exit `U2` state to active `U0` state.

To check the implementation of this feature:

- Use the common checks for enumeration and bulk transfer.
- For `U1` or `U2` timers, check the link state transition to `U1/ U2` in the log file depending on the configuration set. Also, check the `SET_FEATURE` request initiated along with the timeout values in the log file.

- **Speed Change or Speed Negotiation**

To initiate speed change, speed negotiation, or fallback from SS to HS speeds, perform the following steps:

1. Configure host and the device.
2. Run the test on ZeBu.
3. Start the VM.
4. Check for basic enumeration.
5. Initiate the bulk transfer.
6. Initiate device connect with the required speed.
7. Restart the VM to load the new device configuration.
8. Initiate control and bulk transfers after startup.

To check the implementation of this feature:

- Run the “lsusb” utility to check device enumeration.
- Check the bulk and control transfer prints in the log file. For each transfer there is a unique ID, use these IDs to track ACK packet for each transfer. After VM restarts, check if the warm reset is getting initiated and the device is reset. ****Note:** Need more testing.

- **Use the restart_zebu Command with VHS**

To use the restart_zebu command, perform the following steps:

1. Configure host and the device.
2. Run the test on ZeBu.
3. Start the VM.
4. Check the device descriptors and basic enumeration.
5. Initiate `restart_zebu` command on zRci.
6. Initiate run on zRci.
7. Start the VM.
8. Initiate control and bulk transfers.

To check the implementation of this feature:

- Use the common checks for enumeration and bulk transfers before and after ZeBu restart.
- Check if VM is gets automatically disconnected.

• USB 3.2 Monitor support – [Aug B4]

To enable USB 3.2 Monitor support, add the macro “+define+USB32_MONITOR_2LANE” during hardware compile.

Added a new parameter (bool lane1) to “startBusMonitor” API. Set this to 1 when they intend to enable both the lanes in monitor as shown in the following code. `mon->startBusMonitor(ZEBU_IP::MONITOR_USB_SVS::MonitorData,1,1/* lane1*/);`

An example of the XML configuration file for settings to enable monitor are as follows:

```
<vHost:USB>
/Already existing configuration/
    <vHost:USBMonitorXtor>
        <vHost:IsEnabled>true</vHost:IsEnabled>
        <vHost:FileName>USBXtorMonitor</vHost:FileName>
        <vHost:HdlInst>hw_top.top1.mon</vHost:HdlInst>
        <vHost>CreateFSDB>true</vHost>CreateFSDB>
        <vHost:ProgName>NULL</vHost:ProgName>
        <vHost:AutoInc>false</vHost:AutoInc>
        <vHost:Debug>false</vHost:Debug>
        <vHost:MonitorType>true</vHost:MonitorType>
        <vHost:Descramble>false</vHost:Descramble>
        <vHost:Lane1>false</vHost:Lane1>
    </vHost:USBMonitorXtor>
/Already existing configuration/
</vHost:USB>
```

Set Lane1 to `true` for USB32 Monitor in the configuration XML.

• Protocol Reset feature in VHS – [Aug B4]

Protocol resets can be enabled using the `enableResteddevice` API or by setting the XML configuration.

Using the API

- For hot reset, use the command `enableDeviceReset(1,0,1)`
- For warm reset, use the command `enableDeviceReset(1,1,1)`

Setting the XML Configuration

```
<vHost:USB>
/Already existing configuration/
```



```

        <vHost:ResetDevice>
        <vHost:IsEnabled>true</vHost:IsEnabled>
        <vHost:IsWarmReset>>false</vHost:IsWarmReset>
        <vHost:XtorEnumerate>true</vHost:XtorEnumerate>
        </vHost:ResetDevice>
/Already existing configuration/
<vHost:USB>

```

Set “WarmReset” to “false” for Hot Reset and set it to “true” for Warm Reset in the configuration XML. Set “XtorEnumerate” to “true” always.

• **VSA USB XML Configuration Properties With Description**

Property	Description	Type	Items for Array Type Only	Default
Name	ProxyUSBXtor Instance Name	string		usb
HdlPath	HDL path of the driver	string		True
HdlPathToDisableWhenActiveHigh	HDL path of the transactor Disable Signal	string		
ClockControlled	Enable Clock-Controlled Mode	array	enum string	“false”
DebugLevel	Set DebugLevel	integer		Min:0,Max:55 default:0 **The LSB digit is used for VHS debug level, and MSB digit is used for Xtor debug level.**
USB	USB transactor configuration	object		
Type	Type of ProxyXtor	array	enum string	Default: [“USB3X_DPI “ **Always set to USB3X_DPI**
LowSpeed	If transactor is running in low speed or not	array	string	“false”
IsUTMI16	IsUTMI16	array	string	“false”

Property	Description	Type	Items for Array Type Only	Default
XtorPlugLatency	XtorPlugLatency	integer		0
XtorPlugDuration	XtorPlugDuration	integer		0
DeviceWaitTime	Wait time for device (seconds)	integer		900
RequestWaitTime	Request wait time (seconds)	integer		0
EnableWatchdog	EnableWatchdog	array	enum string	"false"
Usb32	usb32 with 0 being USB 3.1 and 1 being USB 3.2	array	enum string	"false"
Lanes	Number of lanes	array	integer	[1,2] Default: [1]
DrdSpeed	DrdSpeed	array	enum string	["zusb_speed_low", zusb_speed_full", "zusb_speed_high", "zusb_speed_super", "zusb_speed_super_plus"] Default: ["zusb_speed_full"]
BinFile	Default binfile.	string		Provided in ZEBU_IP_ROOT
ResetDevice	Definitions for resetting device	object		
IsEnabled	If reset is enabled	array	enum string	"false"
IsWarmReset	If reset is warm or not	array	enum string	"false"
XtorEnumerate	Whether transactor is enumerated or not	array	enum string	"false"

Property	Description	Type	Items for Array Type Only	Default
BulkStreaming	BulkStreaming Setup	object		
IsEnabled	Whether bulk streaming is enabled or not	array	enum string	"false"
Debug	Debug	object		
ProtocolTrace	USB Protocol Analyzer (PA) Settings	object		
IsEnabled	Enable USB Protocol Monitor	array	enum string	"false"
FileName	Name of file	string		
USBMonitorXtor	USB Monitor Setup	object		
IsEnabled	Whether the USB Monitor is enabled	array	enum string	"false"
FileName	Name of file	string		
HdlInst	Monitor transactor Driver Inst	string		
CreateFSDB	Monitor transactor Create FSDB	array	enum string	"false"
ProgName	Monitor transactor Prog Name	string		
AutoInc	Monitor transactor Auto Inc	array	enum string	"false"
Debug	Monitor transactor Debug	array	enum string	"false"
Descramble	Monitor transactor Descramble	array	enum string	"false"
Lane1	Monitor transactor Lane1	array	enum string	"false"

Property	Description	Type	Items for Array Type Only	Default
AutoConnect	Whether to auto connect or not	array	enum string	"false"
UseXtorScheduler	Whether to use the transactor scheduler or not	array	enum string	"false"

Enhanced APIs for U1/U2 Timer Feature in VHS

With the current release, functionality of the `setU1U2timeout ()` API is enhanced to help configure U1 or U2 timeout values. This method is applicable only for Host. The syntax for the API is as follows: `int setU1U2timeout(uint32_t U1U2 = 0, uint32_t timeout = 0)`

The parameters for the `setU1U2timeout ()` API are as follows:

- **U1U2:** Choose from the following options: 1 for U1 timer and 2 for U2 timer.
- **timeout:** Choose from the following options for the `PORT_U1_TIMEOUT` value:

00H Zero (Default)

01H 1 μ s

02H 2 μ s

03H 3 μ s

...

7FH 127 μ s (Maximum allowed)

Choose from the following options for the `PORT_U2_TIMEOUT` value:

00H Zero (Default)

01H 256 μ s

02H 512 μ s

03H 768 μ s

...

FEH 65.024 ms (Maximum allowed.)

The return values for the `setU1U2timeout ()` API are as follows:

0 indicates Success; otherwise, the API returns an error.

Enhanced Suspend and Resume APIs for USB Transactor in VHS

In VHS runs, a delay is observed after enumeration is complete that causes Link to auto suspend due. With the current release, functionality of two APIs, `zusb_status_suspendL1Device ()` and `zusb_status_resumeL1Device ()`, has been enhanced to address this issue for the USB Host transactors.

The `zusb_status_suspendL1Device ()` API configures the USB Host transactor to initiate L1 Suspend. The possible return values for this API are as follows:

- `ZUSB_STATUS_BUSY`: Indicates that the device has responded with an `NYET` status. Check if the device has scheduled transfers; these should not occur.
- `ZUSB_STATUS_NOT_SUPPORTED`: Indicates that the device does not support L1. Check Device's USB 2.0 capability is set as L1 in `bmAttributes`.
- `ZUSB_STATUS_TIMEOUT`: Indicates that the device does not respond to L1 control transfer.
- `ZUSB_STATUS_NO_DEVICE`: Indicates that the speed of the device is set as FS or HS.
- `ZUSB_STATUS_ERROR`: Indicates that the device is throwing a non-recognizable error.
- `ZUSB_STATUS_SUCCESS`: Indicates that the L1 suspend is successful.

The `zusb_status_resumeL1Device ()` API configures the USB Host transactor to initiate L1 Resume from the device side. The possible return values for this API are as follows:

- `ZUSB_STATUS_NO_DEVICE`: Indicates that the speed of the device is set as FS or HS.
- `ZUSB_STATUS_ERROR`: Indicates that the device is throwing a non-recognizable error.
- `ZUSB_STATUS_SUCCESS`: Indicates that the resume command is successful on the device.

Note:

The loop call continues until the `ZUSB_STATUS_RESUME_COMPLETE` message is displayed after calling `resumeL1Device`.

Limitation

It is recommended to add a delay after calling the `resumeL1Device ()` API if, for the USB transactor, the `loop ()` API changes the link state before it returns the `ZUSB_STATUS_RESUME_COMPLETE` status. This ensures that the loop also returns along with link state update.

USB Device Hot Plug Support

The USB Device Hot Plug Support is available only with Qemu and is not supported with VirtualBox.

Proxy Side:

Add the following in the config.xml:

```
<vHost:EnableDeviceHotPlug>true</vHost:EnableDeviceHotPlug>
```

Qemu Side:

1. Add the following command line option to the Qemu:

```
-monitor telnet:126.0.0.1.1243,server,nowait
```

Do not add the following command line option:

```
-device ZebuProxyUSB, debug=1, id=pv0.usb0 -device snps-usb-dev
```

2. Open the Qemu monitor using the following:

```
telnet 127.0.0.1.1243; //Here you can see that host and port are same  
that you added in the step 1.
```

3. Now add the USB device through the Qemu monitor:

```
(qemu) device_add snps-usb.dev,InstName=pv0.usb0, id=usb0
```

The USB device is displayed in the VM.

4. To remove the device from the VM, use the following command:

```
(qemu) device_del usb0; //Here usb0 is derived from id=usb0 at step 4.
```

Initiating USB Speed Change

To initiate USB speed change, do the following:

1. Configure the host and device.

- a. Set initial speed of the device in config.xml through DrdSpeed.

```
<vHost:DrdSpeed>zusb_speed_super_plus</vHost:DrdSpeed>
```

- b. Set AutoReconnectWaitMins in config.xml to non-zero value to run VHS even if QEMU is closed.

```
<vHost:AutoReconnectWaitMins>60</vHost:AutoReconnectWaitMins>
```

- c. Set EnableDeviceHotPlug to true if you are using the hot plug mechanism to restart the device.

```
<vHost:EnableDeviceHotPlug>true</vHost:EnableDeviceHotPlug>
```

2. Run the test on ZeBu.
3. Launch VM using the following steps, after the VM.socket file is ready:
 - a. Ping QEMU VM using the following command:

```
ProxyPing -D 0 -d 0 -Q -x <name_of_config_xml>
```

- b. QEMU Command line:

```
qemu-system-x86_64 -M q35,accel=kvm -m 8192 -machine smm=off -drive  
file=<qemu_image_path> -chardev stdio,  
id=seabios -device isa-debugcon,iobase=0x402,chardev=seabios -net  
user,hostfwd=tcp::3022-:22 -monitor telnet:127.0.0.1:1234,  
server,nowait -fsdev  
local,id=SharedFolder,path=<Shared_Folder_path>,security_model=none  
-device virtio-9p-pci,fsdev=SharedFolder,mount_tag=SharedFolder
```

4. Check the device connect status.
5. Check the device descriptors and information using the lsusb utility on the VM.

Run the lsusb utility to list connected USB devices.

Disconnect the Device

To disconnect the device, do the following:

1. Either disconnect the Qemu VM or unplug the USB device from running Qemu VM (if device hot plug feature is in use).
2. Unplug the USB device by calling `USBUnplug()`.

Device Speed Change

To change the speed of the device, use:

```
xlor->addConfigParam("DRD_Speed", (uint32_t) speed)
```

Connect the Device

To connect the device, do the following:

1. Plug back the device again using `USBPlug()` and wait for the link-up and enumeration to complete.
2. Check USB device speed using `getDeviceSpeed()` API.
3. Use Hot Plug mechanism to update the new configuration in the VM or restart the VM to load the new device configuration.

Note the following:

- USB device speed can be configured by setting following transactor parameter:

```
setConfigParam("DRD_Speed", (uint32_t) speed
```

where,

speed can be `ZUSB_SPEED_FULL`, `ZUSB_SPEED_HIGH`, `ZUSB_SPEED_SUPER`, `ZUSB_SPEED_SUPER_PLUS`.

To check successful transfers, after each disconnect or connect cycle, use the common checks for enumeration and bulk transfers. For speed change, check the configuration printed in the log file followed by the bus ID and device ID that is printed after enumeration.

Ethernet

For Ethernet-related features, see the following topics:

- [Ethernet PHY Interfaces](#)
- [Connecting Virtual Ethernet to ZeBu](#)
- [Running Virtual Ethernet Connector](#)
- [Virtual Ethernet TAP](#)
- [Virtual ZEMI-3 Ethernet Transactor Support](#)
- [Interpreting Ethernet Packet From Log](#)

Ethernet PHY Interfaces

The Ethernet solution connects to the DUT through a MAC or PCS Ethernet PHY interface. Depending on the speed, different interfaces are available.

MAC Interface

MII or MAC interface supports 10Mbps to 800Gbps speeds with various configurations. These are listed in following table:

Speed	MAC Gear Box Width	VHS XML Configuration Value
10M MII	4 (MII_MODE_SEL 0)	MII_10Mbps
100M MII	4 (MII_MODE_SEL-0)	MII_100Mbps
1G GMI	8(default)	GMII_1Gbps
10M RMII	2 (MII_MODE_SEL-2)	RMII_10Mbps
100M RMII	4 (MII_MODE_SEL-1)	RMII_100Mbps
1G RGMII	4	RGMII_1Gbps
10G	32	MII_10Gbps_32b
10G	64	MII_10Gbps_64b
25G	32	MII_25Gbps_32b
25G	64	MII_25Gbps_64b
25G	128 (default)	MII_25Gbps_128b
25G	256	MII_25Gbps_256b
40G	64	MII_40Gbps_64b
40G	128	MII_40Gbps_128b
40G	256	MII_40Gbps_256b
50G	64	MII_50Gbps_64b
50G	128	MII_50Gbps_128b
50G	256	MII_50Gbps_256b
100G	64	MII_100Gbps_64b
100G	128	MII_100Gbps_128b
100G	256	MII_100Gbps_256b
100G	512	MII_100Gbps_512b

Speed	MAC Gear Box Width	VHS XML Configuration Value
200G	128	MII_200Gbps_128b
200G	256	MII_200Gbps_256b
200G	512	MII_200Gbps_512b
400G	256	MII_400Gbps_256b
400G	512	MII_400Gbps_512b
800G	256 (800G MAC Driver only)	MII_800Gbps_256b
800G	512 (80G MAC Driver only)	MII_800Gbps_512b

PCS Interface

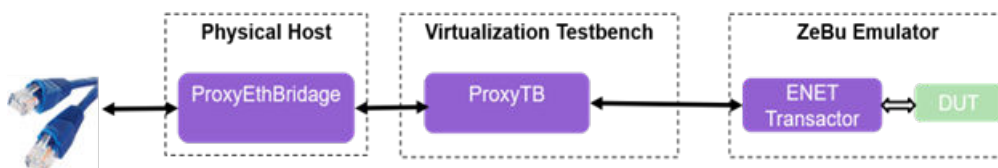
PCS interface supports from 1Gbps up to 800Gbps speeds with various configurations. These are listed in following table:

Speed	CONFIG and Standards Clause	VHS XML Configuration Value
1G	10 bit	PCS_1Gbps_1x10b
10G	20 bit	PCS_10Gbps_1x20b
10G	40 bit	PCS_10Gbps_1x40b
10G	20 bit CL74 FEC	PCS_10Gbps_1x20b_CL74
25G	40 bit CL74	PCS_10Gbps_1x40b_CL74
25G	40 bit	PCS_25Gbps_1x40b
25G	40 bit RSFEC 528	PCS_25Gbps_1x40b_RS528
40G	4 Lane 20 bit	PCS_40Gbps_4x20b
40G	4 Lane 40 bit	PCS_40Gbps_4x40b
40G	4 lane (Regular/CL74 FEC) 20bit	PCS_40Gbps_4x20b_CL74
40G	4 lane CL74 FEC- 40bit	PCS_40Gbps_4x40b_CL74
40G	4 lane 40 bit	PCS_40Gbps_2x40b
40G	2 Lane CL74 FEC- 20bit	PCS_40Gbps_2x40b_CL74

Speed CONFIG and Standards Clause VHS XML Configuration Value		
50G	2 Lane 40 bit	PCS_50Gbps_2x40b
50G	2 Lane 40 bit RS528	PCS_50Gbps_2x40b_RS528
50G	2 Lane 40 bit RS544	PCS_50Gbps_2x40b_RS544
50G	1 Lane 80 bits RS544	PCS_50Gbps_1x80b_RS544
100G	4 Lane 40 bit	PCS_100Gbps_4x40b
100G	4 Lane 40 Bit RS528	PCS_100Gbps_4x40b_RS528
100G	4 Lane 40 bit RS544_CK	PCS_100Gbps_4x40b_RS544_CK
100G	2 Lane 80 bit RS544	PCS_100Gbps_2x80b_RS544
100G	1 Lane 160 bit RS544	PCS_100Gbps_1x160b_RS544
100G	1 Lane 160 bit RS544_CK	PCS_100Gbps_1x160b_RS544_CK
200G	4 Lane 80 bit	PCS_200Gbps_4x80b
200G	2 Lane 160 bit	PCS_200Gbps_2x160b
400G	8 Lane 80 bit	PCS_400Gbps_8x80b
400G	4 Lane 160 bit	PCS_400Gbps_4x160b
800G	16 Lane 80 bit	PCS_800Gbps_16x80b
800G	8 Lane 160 bit	PCS_800Gbps_8x160b

Connecting Virtual Ethernet to ZeBu

For Internet interfaces, it is also possible to virtualize a physical Ethernet interface on a Linux machine to one or more Ethernet interface in a design emulated in ZeBu. The emulated design appears as if it were physically connected to the Ethernet interface instead of the Linux machine. The following figure illustrates virtual ENET Connector:



Running Virtual Ethernet Connector

The Virtual ENET Connector is a software component that bridges a physical or a virtual Ethernet interface to one or more ENET ProxyXtor in the Virtualization Testbench.

The Virtualization Testbench is running on the ZeBu host, whereas the Virtual ENET Connector is running on the machine with the physical interface. It can be the same machine, or different machines.

The Virtual ENET Connector connects to the Virtualization Testbench through its own `ProxyVisor` instance. The `ProxyEthXtor` instances that connect with the physical interface must be associated with that `ProxyVisor`. A VM can be connected to the same Virtualization Testbench in parallel with the Virtual ENET Connector using a separate `ProxyVisor` instance. The Virtual ENET Connector uses the following command-line options:

```
% ProxyEthBridge [options] -i <ethIf> [<hostname>:]portnum
% ProxyEthBridge [options] -i <ethIf> [client:]socketFile
```

Here, `<ethIf>` is the physical Ethernet interface to bridge. If it is specified as "tap", the connector creates a new virtual Ethernet interface, "tapN".

Configuring ENET Connector as Server

If you want to configure the Virtual ENET Connector as a server (implying the Virtualization Testbench is configured as a client), specify ONLY a port number using `portnum`. This starts the Virtual ENET Connector as a server listening on the specified port number.

Alternatively, if you specify a filename instead of a port number, the port number is dynamically picked and written (along with the current `hostname`) in the specified file. This file can then be read by the Virtualization Testbench to connect to the Virtual ENET Connector server.

Configuring ENET Connector as Client

If you want to configure the Virtual ENET Connector as a client (implying the Virtualization Testbench is configured as a server), specify `hostname:portnum`.

where,

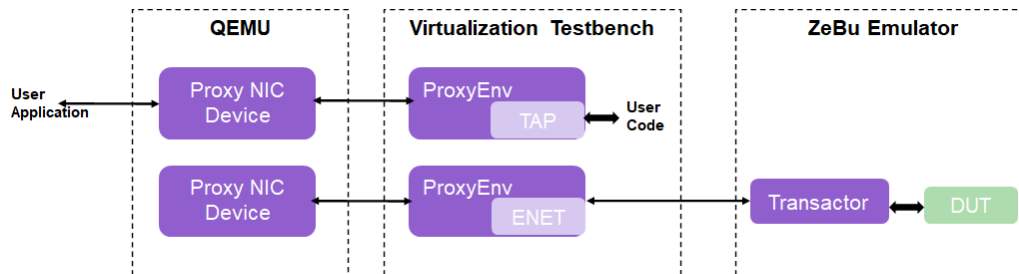
- `hostname` is the name of the host. The Virtualization Testbench is running on this host.
- `portnum` is the port number. The Virtualization Testbench server is listening through this port number.

Alternatively, you can specify "`client:filename`". The specified `filename` is assumed to be a file created by the Virtualization Testbench server containing the `hostname` it is running on and the port number it is listening on.

Virtual Ethernet TAP

The TAP interface on the `ProxyEthXtor` virtualization component creates a generic communication channel between an application or script running on the Guest O/S and the Virtualization Testbench. It differs from the normal use model of the other virtualization components in that it does not interface with the DUT in ZeBu, but rather with the testbench running on the ZeBu host. The following figure shows the block diagram of virtual Ethernet Tap:

Figure 8 Virtual Ethernet Tap



The Virtual Ethernet Tap allows arbitrary messages to be exchanged between an application or script running on the Guest O/S and the Virtualization Testbench. For example, it can be used to trigger commands on the VM from the Virtualization Testbench after the DUT has reached a specific state, or ask for emulation state information outside of the normal protocol interfaces, such as internal DUT state information. It can coexist with other virtualization components.

To add a Virtual Ethernet TAP communication channel, you only need to modify your Virtualization Testbench to add an instance of the `ProxyEthXtor` configured as a TAP component and update your VM configuration. When the VM starts next, a Virtual NIC interface with MAC address `cc.cc.cc.cc.cc.55` exists and can be used to send and receive messages to the Virtualization Testbench by binding it to a raw-mode socket that passes all Ethernet frames.

Configuring the Virtualization Testbench Component

The Virtual Testbench side of the Virtual Ethernet TAP is implemented by an instance of a `ProxyEthXtor` component configured as a TAP component. A pointer to a suitable "receive" function can optionally be specified that is called when a message is received from the VM side. If no "receive" function is specified, received messages are ignored.

```

bool rxFct(uint16_t len, const uint8_t *data)
{
    ...
}
...
ProxyEthXtor::Config_t cfg;
cfg.u.TAP.receive = rxFct;
  
```

```
ProxyEthXtor *tap0 = ProxyEthXtor::get("tap0", ProxyEthXtor::TAP, cfg);
...
```

Opening the Virtual NIC Interface in the VM

The Virtual NIC interface must be bound to a socket opened in raw mode, and configured to accept frames from all protocols (*ETH_P_ALL*). In Python, this can be accomplished using the following statements:

```
import socket
import sys

ethif = "enp0s8";
try:
    skt= socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
        socket.htons(3))
    skt.bind((ethif, 0))
except:
    print "Unable to open Ethernet Interface '{}'.format(ethif)
    sys.exit(-1)
```

Sending Messages to the VM

To send a message to the VM, call the *send()* function in the configuration object of the Virtual Ethernet TAP component. The message must have a minimum length of 14 bytes. It is usually a good idea to make sure bytes at offset 12 and 13 contain an invalid *EtherType*.

```
while (running) {
    // Send somer internal DUT state to VM every second
    uint64_t msg[2];
    msg[0] = myXtor.getDutState();
    msg[1] = 0; // EtherType will be 0x0000
    cfg.u.TAP.send(sizeof(msg), (const uint8_t*) &msg);
    sleep(1);
}
```

The message is received by the corresponding Virtual NIC in the VM and can be retrieved from the socket bound to it. The message might be padded with additional bytes by the transport infrastructure, but the content should not be otherwise modified.

```
while (running):
    msg = skt.recv(4096)
    # Ignore IPv4 and IPv6 frames
    if (len(msg) >= 14 &&
        ((msg[12] == 0x08 and msg[13] == 0x00)
         or (msg[12] == 0x86 and msg[13] == 0xdd))):
        continue
    print "Received {}-bytes message from Virtualization
    Testbench.".format(len(msg))
```

Sending Messages to the Virtualization Testbench

To send a message to the Virtualization Testbench, send it via the socket bound to the Virtual NIC interface. The message must have a minimum length of 14 bytes. It is usually a good idea to make sure bytes at offset 12 and 13 contain an invalid *EtherType*.

```
msg = "..."  
skt.send(len(msg), msg)
```

The message is received by the corresponding Virtual Ethernet TAP component in the Virtualization Testbench. The message may be padded with additional bytes by the transport infrastructure, but the content should not be otherwise modified. It is passed to the Virtualization Testbench via a callback to the function specified in the "receive" configuration parameter. If no such receive function was configured, the received message is ignored.

```
bool rxFct(uint16_t len, const uint8_t *data)  
{  
    // Ignore IPv4 and IPv6 frames  
    if ((data[12] == 0x08 && data[13] == 0x00)  
        or (data[12] == 0x86 && data[13] == 0xdd)) return true;  
  
    printf("Received %d-bytes message from VM.\n", len);  
  
    return true;  
}
```

It is important to note that the receive callback function is called by a POSIX thread internally spawned by the Virtual Ethernet TAP component, and is not thread-safe. If the received message must be consumed by the main Virtualization Testbench thread (or any other thread), a suitable thread-safe message passing mechanism must be used. For an example of thread-safe message passing implementation, see the file **Examples/TAP/tb/proxyTB.cc** in the distribution.

Virtual ZEMI-3 Ethernet Transactor Support

To enable the virtual ZEMI-3 Ethernet transactor support, update the configuration options available at *shared/Config.mk*. ZEMI-3 Ethernet transactor supports both C++ and **zRci** testbenches. Utilities and *Config.mk* are common for both C++ and **zRci** testbenches that are available at the shared directory.

In *Config.mk*, high level compile and runtime configurations can be modified. In addition, it helps to create VBox/QEMU and clone frequently used targets for Snapshot/Restore. Both C++/Makefile and **zRci**/runtime/Makefile includes *Config.mk*. Therefore, all the targets in *Config.mk* can be executed.

To compile the ZEMI-3 Ethernet transactor, perform the following steps:

1. `cd` to `zebu/MII` (or) `zebu/PCS`
2. `% make zebu`

To run the ZEMI-3 Ethernet transactor, use *ProxyEnv*-based (XML-based) VHS testbench. In a C++ testbench, *proxyTBEEnv.cc* is available within the C++ directory. *ProxyEnv* uses XML-based *ProxyXtor* and *ProxyVisor* configuration.

To run the ZEMI-3 Ethernet transactor in a C++ testbench, execute the following:

```
% make clean ProxyTB ProxyTBRUN (make all)
```

In a zRci testbench, *proxyTBEEnv.cc* is available within the *zRci/runtime/bench* directory.

To run the ZEMI-3 Ethernet transactor in an **zRci** testbench, execute the following:

```
% make run_zrci (make all)
```

ProxyEnv for back-to-back transactor setup is available at *shared/config/config_enet_dpi.xml*.

```
<vHost:Ethernet>
  <vHost:Type>ENETDPI</vHost:Type>
  <vHost:Interface>MII_400Gbps_256b</vHost:Interface>
  <vHost:TxQueueLowWaterMark>10</vHost:TxQueueLowWaterMark>

  <vHost:TxQueueHighWaterMark>20</vHost:TxQueueHighWaterMark>
  <vHost:TxQueueMaxSize>30</vHost:TxQueueMaxSize>
  <vHost:MACAddress>000000112233</vHost:MACAddress>
</vHost:Ethernet>
...
```

Valid tags, values, and types are specified in the *schema.yaml*, which can be found at *\$VSA_VHS_ROOT/Schemas/ENETDPI/schema.yaml*.

Interpreting Ethernet Packet From Log

This section provides information on the Ethernet frame structure.

```
DEBUG: VMLog [eth0 [ Proxy ] ] 000096.608651548s Sending 110-bytes frame:
 333300000016 000000112233 86dd ...
33330000 00160000 00112233 86dd6000
00000038 00010000 00000000 00000000
00000000 0000ff02 00000000 00000000
00000000 00163a00 05020000 01008f00
d4230000 00020300 0000ff02 00000000
00000000 00000001 00030400 0000ff02
00000000 00000000 0001ff96 98b4
```


Hexadecimal Breakdown

The frame you provide is represented in hexadecimal format, which requires some interpretation.

Split Component	Description
333300000016	Destination MAC Address(next 6 bytes)
000000112233	Source MAC Address (next 6 bytes)
86dd	Ethertype value indicating the protocol encapsulated in the payload

Ethernet Frame Structure

A standard Ethernet frame has the following structure:

Preamble (7 bytes)	Not visible as the frame starts immediately with the Destination MAC
Start Frame Delimiter (1 byte)	Not visible
Destination MAC Address (6 bytes)	33:33:00:00:00:16
Source MAC Address (6 bytes)	00:00:00:11:22:33
Ethertype (2 bytes)	86dd indicates that the payload is IPv6
Payload	The rest of the bytes form the payload or data being sent
Frame Check Sequence (4 bytes)	Visible at the end of the frame for error-checking (not shown in your excerpt)

Step-by-Step Breakdown

Destination MAC Address (333300000016)	Represents a multicast IPv6 address (the leading 33:33 indicates an IPv6 multicast).
Source MAC Address (000000112233)	Represents the MAC address of the sender (the machine initiating the communication).
Ethertype (86dd)	Corresponds to the EtherType value for IPv6 (0x86DD).

Payload	• Next section of bytes represents the actual data transmitted which starts after the Ethertype. • Breakdown example for the payload starting from 6000 onwards.
---------	---

Payload Analysis

Starting from 6000, the following fields in an IPv6 packet are visible:

Payload Length (0000 0038)	The next field represents the length of the payload in bytes, excluding the IPv6 header.
Next Header (00)	Indicates the protocol encapsulated in the IPv6 packet (for example, 06 for TCP, 11 for UDP). This depends on the protocols being used.
Hop Limit (00)	The Hop Limit value specifies the number of hops that the packet can go through before being discarded.
Source Address	Large groups of zeros often indicate a non-specific address or unspecified source address.
Destination Address	This is often either multicast or specific address.

UFS

For UFS-related features, see the following topics:

- [Inserting UFS Kernel](#)
- [Running the Linux UFS Driver](#)
- [Formatting and Mounting the UFS Disk](#)
- [UFS Utilities](#)

Inserting UFS Kernel

To use UFS from a Linux machine, a custom driver is required.

The QCOW2 VM Image with latest custom kernel is available at `/remote/sdg_sv_vsa/VMImages/ubuntu_16_ufs.qcow2`. This is Synopsys specific path.

Running the Linux UFS Driver

When the VM starts, directly open a terminal in the VM or connect through *ssh* port-forwarding setup from the host machine.

```
% ssh -Xp 3025 root@localhost (If using ssh port connect)
% cd kernelbuild/linux-stable/userScripts
% ./modinsert.sh
```

After it is successfully executed, you should see the UFS disks/devices under Disks (in ubuntu) or under `/dev/sd*` (which will be listed under `dmesg`)

Formatting and Mounting the UFS Disk

To format, mount, and access the UFS disk, use the following commands:

```
# Format Disk
% echo 'type=83' | sfdisk /dev/sdb (Say it is sdb disk. Can vary)
# Create the group table, inode table, superblocks and filesystem
information
% mkfs -F /dev/sdb1
# Create and file system"
% mkdir -p /mnt/mydisk
# Mount file system"
% mount /dev/sdb1 /mnt/mydisk
# Create a 1M file using dd utility
% dd if=/dev/zero of=file.txt count=1024 bs=1024
# Copy the file into the disk
% cp file.txt /mnt/mydisk
# Flush disk I/O cache buffer by executing sync"
% sync
```

UFS Utilities

For generating UFS traffic and for modifying the configurations two utilities are recommended. The utility for modifying the UFS configurations is Synopsys proprietary utility called `vsa-vhs-ufs-utils` that is available in-house at Synopsys and can be shipped to customer. In order to generate UFS traffic you can use `sg3_utils` utility provided by Linux.

vsa-vhs-ufs-utils

This utility is specific to VHS UFS solution, that is, it works only with UFSHC attached with PCIe Vendor ID 0x16c3 and Device ID 0xB102. Using this utility, UFSHC registers can be accessed using PCIe mmap and UFS configuration settings like Gear change(Gear-5, Gear-4), speed change (High Speed to Low Speed) can be done.

This utility can be picked from the following location in Synopsys and transferred inside VM using Shared Folder:

```
/remote/sdg_sv_vsa/zebu_vsa/vsa-vhs-ufs-utils/latest/vsa-vhs-ufs-utils
% ./vsa-vhs-ufs-utils -h
```

Usage

```
vsa-vhs-ufs-utils [-hv] [--txlanes=] [--rxlanes=] [--txgear=] [--rxgear=]
  [--hsseries=] [--txtermination=] [--rxtermination=] [--pwrmode=]
  [--getpmsettings] [--timeout=] [--bdf=]
```

where,

```
-h, --help      Display this help and exit
-v, --version   Display version of this application
--txlanes=<n>   Number of tx lanes
--rxlanes=<n>   Number of rx lanes
--txgear=<n>    Number of tx gear
--rxgear=<n>    Number of rx gear
--hsseries=<n>  Indicates Series-A or Series-B (1 => A, 2 => B)
--txtermination=<n> Tx Termination (1 => Enable, 0 => Disable)
--rxtermination=<n> Rx Termination (1 => Enable, 0 => Disable)
--pwrmode=<n>   Rx/Tx Power Mode (0x11 => Fast Mode, 0x22 => Slow Mode,
0x44 => FastAuto Mode, 0x55
--getpmsettings Get Power Mode settings of Local and Peer
--timeout=<n>   Time (in seconds) to wait for command completion interrupt
status. Default is 60 seconds
--bdf=<bus:dev.func> BDF number of Synopsys Virtual Host UFS Solution in
B:D.F format. Default is 1:0.0
```

Sg3 Utilities

Sg3_utils is a linux utility that can be used for sending SCSI commands. It is available online for download. Refer to the documentation from linux for more information on the usage. For using it with VHS UFS, download the utility from online resources and install it inside the VM. To initiate SCSI commands from VM, following steps need to be done:

1. Run the VHS UFS setup, connect to a VM and mount the disk as per the instructions in previous sections
2. Inside the UFS VM image file with UFS custom kernel

```
cd /home/synopsys
./diskUFS.sh
```

where, diskUFS creates user image file.

3. Install the sg3_utils utility inside the VM and build the utilities:

```
cd sg3_utils-1.48
./configure --prefix=/home/synopsys/sg_utils_commom/ --disable-static
ls ../sg_utils_custom/
```

```
make clean

make

make install
```

4. **export** PATH=\$/home/synopsys/sg_utils_commom/bin:\$PATH
5. **SCSI_READ** command can be executed as follows with VM root access → `./sg_read if=/dev/sdb1 bs=512 count=1MB time=2 dpo=1 dio=1`
6. **SCSI_WRITE** command can be executed as follows with VM root access → `./sg_write_buffer --in=/home/synopsys/diskUFS.sh /dev/sdb1`

After these commands are executed, it is visible in the `ufs_host.log` file. To execute the command multiple times, the disk needs to be unmounted and relaunched.

PCIe End Point vNIC Solution

The PCIe End Point NIC is a special use model in VHS where peer-to-peer communication between two hosts connected over a network is possible.

For more information, see the following topics:

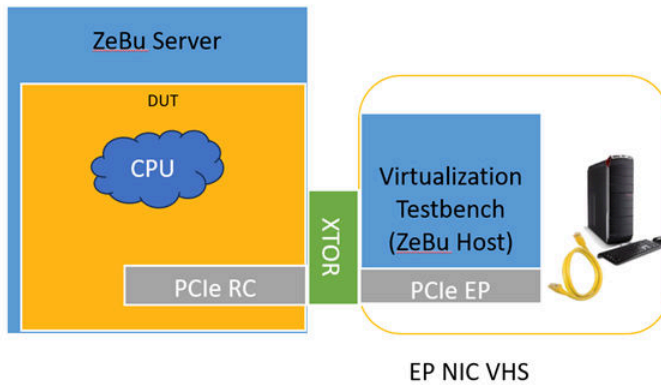
Solution Overview

This solution mimics the use case of an emulated DUT Host SoC with PCIe Root Port connected through an Endpoint PCIe transactor to a PCIe NIC card. The network interface of the card is directly connected to the network interface of a QEMU VM running a Linux kernel. You can ping the interface of the DUT running on ZeBu from within the QEMU VM and can perform network traffic as if it is pinging between two hosts connected through a network cable.

The Endpoint transactor appears to the DUT SoC as a (partial) Intel 82573 NIC device. It is the responsibility of the SoC to load the appropriate driver to manage the emulated NIC. Note that this solution is designed to allow communication between processes running in the embedded SoC and the QEMU VM. It is not intended to stress the networking or PCIe capabilities of the SoC.

The following diagram depicts the high-level usage of EP NIC VHS Solution with RC Host DUT:

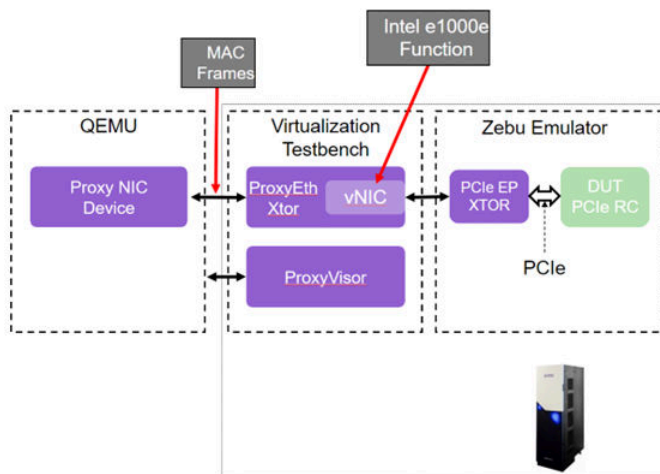
Figure 9 High-Level Usage of EP NIC VHS with RC Host DUT



Solution Architecture

The following diagram shows the low-level details of the PCIe EP architecture. The DUT interfaces with PCIe EP transactor over PCIe link. On top of the PCIe EP transactor, the solution partially models the e1000e (Intel 82573 NIC). The content of the Ethernet frame T_x buffers is read over PCIe and sent to QEMU. The Ethernet frames coming from QEMU are similarly stored in R_x buffers over PCIe.

Figure 10 Solution Architecture



Enable the Solution

To enable this solution, include the following XML code in the configuration file:

```
<vHost:ProxyVisor>
  <vHost:Name>pv0</vHost:Name>
  <vHost:IsServer>true</vHost:IsServer>
  <vHost:PortNumber>0</vHost:PortNumber>
  <vHost:ServerHostName></vHost:ServerHostName>

  <vHost:ServerSpecFile>${HOME}/Regress.Socket</vHost:ServerSpecFile>
  <vHost:MaxClientWaitMins>5</vHost:MaxClientWaitMins>
  <vHost:IsTunneling>false</vHost:IsTunneling>
  <vHost:DebugLevel>0</vHost:DebugLevel>
  <vHost:MaxServerStartSecs>600</vHost:MaxServerStartSecs>
  <vHost:AutoShutdownMins>15</vHost:AutoShutdownMins>
  <vHost:ProxyXtor>
    <vHost:Name>eth0</vHost:Name>
    <vHost:HdlPath>wrapper.pcie_driver_ep</vHost:HdlPath>
    <vHost:DebugLevel>21</vHost:DebugLevel>
    <vHost:ClockControlled>false</vHost:ClockControlled>
    <vHost:Ethernet>
      <vHost:Type>E1K_DPI</vHost:Type>
    </vHost:Ethernet>
  </vHost:ProxyXtor>
  <vHost:Equalization>
    <vHost:Disable>true</vHost:Disable>
    <vHost:DisableBypassHighRate>false</vHost:DisableBypassHighRate>
    <vHost:EnablePhase23>false</vHost:EnablePhase23>
  </vHost:Equalization>
  <vHost:MACAddress>cc0000807700</vHost:MACAddress>
  <vHost:ATS>

  <vHost:ForBufferDescriptor>false</vHost:ForBufferDescriptor>
  <vHost:ForBuffer>false</vHost:ForBuffer>
  </vHost:ATS>
</vHost:Ethernet>
</vHost:ProxyXtor>
</vHost:ProxyVisor>
```

The following figure shows the Ethernet frames captured with increased debug level in the XML file:

Figure 11 Example of a Log Showing Solution Implementation

```

3637
DEBUG: VMLog [eth0 [ Proxy ] ] 000450.985283069s Sending 98-bytes frame: cc0000807701 cc0000807700 0800 ...
cc000080 7701cc00 00807700 08004500
00546ab2 00004001 8c88c0a8 010ac0a8
01140000 f0910a22 0001522f bb650000
00002de3 0b000000 00001011 12131415
16171819 1a1b1c1d 1e1f2021 22232425
26272829 2a2b2c2d 2e2f3031 32333435
3637
DEBUG: VMLog [eth0 [ Proxy ] ] 000451.986516324s Received 98-bytes frame: cc0000807700 cc0000807701 0800 ...
cc000080 7700cc00 00807701 08004500
0054dc2e 40004001 db0bc0a8 0114c0a8
010a0800 668a0a22 0002532f bb650000
0000aee9 0b000000 00001011 12131415
16171819 1a1b1c1d 1e1f2021 22232425
26272829 2a2b2c2d 2e2f3031 32333435
3637
DEBUG: VMLog [eth0 [ Proxy ] ] 000451.986961263s Sending 98-bytes frame: cc0000807701 cc0000807700 0800 ...
cc000080 7701cc00 00807700 08004500
00546b35 00004001 8c05c0a8 010ac0a8
01140000 6e8a0a22 0002532f bb650000
0000aee9 0b000000 00001011 12131415
16171819 1a1b1c1d 1e1f2021 22232425
26272829 2a2b2c2d 2e2f3031 32333435
3637
DEBUG: VMLog [eth0 [ Proxy ] ] 000452.988301839s Received 98-bytes frame: cc0000807700 cc0000807701 0800 ...
cc000080 7700cc00 00807701 08004500
0054dc62 40004001 dad7c0a8 0114c0a8
010a0800 38820a22 0003542f bb650000
0000dbf0 0b000000 00001011 12131415
16171819 1a1b1c1d 1e1f2021 22232425
26272829 2a2b2c2d 2e2f3031 32333435
3637

```


7

IDE Support

To use Integrity and Data Encryption (IDE), you need a special configuration of the PCIe transactor. Refer to *ZeBu® PCIe Transactor User Manual* for details on instantiating the secure engine with the transactor.

Note:

Only one link and one selective IDE streams are currently supported.

The traffic between a Virtual Host Root Complex (RC) and a PCIe function is not encrypted by default. To encrypt some or all of the traffic between the RC and a DUT, you need to integrate the PCIe transactor with the Synopsys Secure IP.

You also need to ensure that IDE is enabled in the XML configuration specification of the Root Complex.

```
<vHost:ProxyXtor>
  ...
  <vHost:PCIE>
    <vHost:Type>DPI</vHost:Type>
    <vHost:IDE>
      <vHost:IsEnabled>true</vHost:IsEnabled>
    </vHost:IDE>
  </vHost:PCIE>
</vHost:ProxyXtor>
```

For more information about IDE support with VHS, see the following topics:

- [Pre-Shared Keys \(PSK\)](#)
- [Using IDE_KM](#)

Pre-Shared Keys (PSK)

Using pre-shared keys allows IDE streams to be configured and secured without any specialized driver code executing on the host. The RC instance must be configured with at least one IDE stream as shown in the example XML specification given below.

```
<vHost:ProxyXtor>
  ...
  <vHost:PCIE>
    <vHost:Type>DPI</vHost:Type>
```

```

        <vHost:IDE>
            <vHost:IsEnabled>true</vHost:IsEnabled>
            <vHost:Stream>
                ...
            </vHost:Stream>
        </vHost:IDE>
    </vHost:PCIE>
</vHost:ProxyXtor>

```

For more information, see the following topics:

- [Link IDE Stream](#)
- [PSK Selective IDE Stream](#)
- [Specifying Pre-Shared Keys](#)
- [Swapping Pre-Shared Keys](#)
- [Unsecuring a Pre-Shared Key Stream](#)

Link IDE Stream

A link IDE stream is configured with a `<vHost:StreamType>` value of “Link” and by specifying a unique stream id and a traffic class. The stream id must be a value between 0 and 255 and the class code must be a value between 0 and 7. The overall configuration of the link IDE stream must be valid as per PCIe requirements and is outside the scope of this document.

```

<vHost:ProxyXtor>
    ...
    <vHost:PCIE>
        <vHost:Type>DPI</vHost:Type>
        <vHost:IDE>
            <vHost:IsEnabled>true</vHost:IsEnabled>
            <vHost:Stream>
                <vHost:StreamType>Link</vHost:StreamType>
                <vHost:StreamId>1</vHost:StreamId>
                <vHost:TrafficClass>0</vHost:TrafficClass>
                <vHost:KeySet>
                    ...
                </vHost:KeySet>
                ...
            </vHost:Stream>
        </vHost:IDE>
    </vHost:PCIE>
</vHost:ProxyXtor>

```

Note:

The current transactor implementation is limited to a single link IDE stream.

PSK Selective IDE Stream

A selective IDE stream with pre-shared keys is configured with a `<vHost:StreamType>` value of “Selective” and by specifying a unique stream id and a traffic class. The stream id must be a value between 0 and 255 and the class code must be a value between 0 and 7. Up to three address ranges may also be specified using up to three `<vHost:Address>` elements. The overall configuration of the selective IDE stream must be valid as per PCIe requirements and is outside the scope of this document.

```
<vHost:ProxyXtor>
...
  <vHost:PCIe>
    <vHost:Type>DPI</vHost:Type>
    <vHost:IDE>
      <vHost:IsEnabled>true</vHost:IsEnabled>
      <vHost:Stream>
        <vHost:StreamType>Selective</vHost:StreamType>
        <vHost:StreamId>3</vHost:StreamId>
        <vHost:TrafficClass>4</vHost:TrafficClass>
        <vHost:Address>
          <vHost:Base> 0x000000000</vHost:Base>
          <vHost:Limit>0xFFFFFFFF</vHost:Limit>
        </vHost:Address>
        <vHost:KeySet>
          ...
        </vHost:KeySet>
        ...
      </vHost:Stream>
    </vHost:IDE>
  </vHost:PCIe>
</vHost:ProxyXtor>
```

Note:

The current transactor implementation is limited to a single selective IDE stream with three address ranges. Selective configuration requests are not supported. Default selective stream for a traffic class is not supported. If no address ranges are specified, the Selective stream behaves like a Link stream where all memory requests on the specified traffic class are secured.

Specifying Pre-Shared Keys

Each IDE stream must be configured with sets of Tx and Rx key that match the Rx and TX keys configured in the peer endpoint function. Keys are specified as sets of 256-bit AES keys and 64-bit Initial Values (IV), one for each direction in each substream.

```
<vHost:ProxyXtor>
...
```

```

    <vHost:PCIE>
      <vHost:Type>DPI</vHost:Type>
      <vHost:IDE>
        <vHost:IsEnabled>true</vHost:IsEnabled>
        <vHost:Stream>
          <vHost:StreamId>1</vHost:StreamId>
          <vHost:TrafficClass>0</vHost:TrafficClass>
          <vHost:KeySet>
            <vHost:PR>
              <vHost:TxKey>...</vHost:TxKey>
              <vHost:TxIV>...</vHost:TxIV>
              <vHost:RxKey>...</vHost:RxKey>
              <vHost:RxIV>...</vHost:RxIV>
            </vHost:PR>
            <vHost:NPR>
              <vHost:TxKey>...</vHost:TxKey>
              <vHost:TxIV>...</vHost:TxIV>
              <vHost:RxKey>...</vHost:RxKey>
              <vHost:RxIV>...</vHost:RxIV>
            </vHost:NPR>
            <vHost:CPL>
              <vHost:TxKey>...</vHost:TxKey>
              <vHost:TxIV>...</vHost:TxIV>
              <vHost:RxKey>...</vHost:RxKey>
              <vHost:RxIV>...</vHost:RxIV>
            </vHost:CPL>
          </vHost:KeySet>
          <vHost:KeySet>
            ...
          </vHost:KeySet>
          ...
        </vHost:Stream>
      </vHost:IDE>
    </vHost:PCIE>
  </vHost:ProxyXtor>

```

Note:

Determining key and IV values is application-specific and outside the scope of this document. The 'genKeySet' utility script may be used to generate random keys.

Pre-shared key sets are used in the order they are specified. After all key sets are used(through a number of key swaps), the first key set is reused. How the same keys are programmed in the peer device is outside the scope of this document.

Swapping Pre-Shared Keys

Swapping keys and programming new ones is a normal part of a secure stream operation.

On the *Rx* path, keys are automatically swapped when a TLP is received with a different K-bit in the IDE prefix. New *Rx* keys are automatically programmed in the now-invalid key slot using the next key set specified in the XML configuration.

On the *Tx* path, keys swapping is triggered by writing the value `0x0000ss04` to the IDE Capabilities register in the RC function. *Tx* keys are automatically programmed in the now-invalid key slot using the next key set specified in the XML configuration.

If the peer function supports SPDМ or the Synopsys IDE_KM-over-DOE protocol, the new *Tx* and *Rx* key sets are automatically exchanged prior to the swap.

Note:

The keys are swapped and new *Tx* and *Rx* keys are exchanged with the peer function is DUT-specific and outside the scope of this document.

Unsecuring a Pre-Shared Key Stream

To unsecure a stream and invalidate all keys, write the value `0x0000ssFF` to the IDE Capabilities Register in the RC function, where “ss” is the stream ID. How the stream is invalidated in the peer function is DUT-specific and outside the scope of this document. As per the IDE protocol, the RC also sends an `IDE Fail` message to the peer function.

A stream may also be unsecured by the reception of an `IDE Fail` message from the peer function. How that message is triggered is DUT-specific and outside the scope of this document.

Using IDE_KM

Integration and Data Encryption (IDE) streams and keys are normally managed using the integrated `IDE_KM` protocol. This requires a suitable key management device driver running on the host. Alternatively, if one is not readily available, the `vsa-vhs-ide-cli` command-line utility can be used. This utility can create, list, and destroy IDE links, as well as triggering key swaps.

This section describes the following topics:

- [Configuring the Host Side](#)
- [IDE_KM Over SPDМ](#)
- [IDE_KM Over DOE](#)

Configuring the Host Side

The IDE_KM protocol specifies how keys can be exchanged with and programmed in a device. How the same keys are programmed on the host side is outside of that specification. This solution implements the same programming model as the Intel Root Complex IDE Key Configuration Unit.

When ProxyPing detects that the DUT supports IDE, it automatically configures QEMU to instantiate a Key Configuration Unit Register Block at physical address `0xA0000000`. The base address used by ProxyPing for the KCURB is only a default and can be changed using the `-PA0xxxxxxxx` command-line argument.

IDE_KM Over SPDM

Security Protocol Data Management (SPDM) is a protocol for securely exchanging secret key information over a clear channel, similar to establishing a `https://` session over the web. After the secure channel is established, it can be used to exchange IDE_KM messages.

Secure IDE_KM-over-SPDM messages are DOE-encapsulated messages using Vendor ID `0x0001` (PCI-SIG) and type `0x02` or `0x04`. For example, an IDE_KM QUERY message would be encapsulated as follows, with the highlighted fields encrypted.

31				0
Reserved		02h / 04h	0001h	
Reserved / Connection ID		0000Ch (Length)		
Session ID				
0Fh (App Data Length)		12h (Length)		
Reserved	Reserved	FEh	SPDM Version	
01h (PCI-SIG)	02h	0003h (PCI-SIG)		
00h (IDE)	04h (Length)		00h (PCI-SIG)	
random	Port Index	Reserved	00h (QUERY)	
MAC				
MAC				
MAC				
MAC				

The establishment of a secure SPDM session is a lengthy multi-step process that ultimately results in the creation of a secure channel for exchanging IDE_KM messages. If the verification objective is not to verify SPDM but IDE, that is a significant overhead that does not contribute to the end goal. The IDE Command-Line utility provides several alternative transport layers for IDE_KM messages. This assumes a similar capability exists in the device under a special test mode or firmware load.

Pre-Established SPDM Session

The IDE Command Line can create a secure SPDM session using pre-shared keys. If the DUT can similarly have secure SPDM sessions provisioned in advance, the entire SPDM stack can be short-circuited. And secure SPDM messages as shown above exchange immediately.

Unencrypted SPDM Sessions

The IDE Command Line can create a “pseudo-secure” SPDM session that does not use any encryption. The IDE_KM messages sent as a “secure” DOE request using types 0x02 or 0x04 but without any content actually encrypted with the following layout. This transport layer is used by the IDE command line utility if the SPDM session is defined as “UNSECURED”. How the peer Endpoint is configured with an unencrypted “secure” session is endpoint-specific.

31			0	
Reserved	02h / 04h	0001h		
Reserved / Connection ID		00008h (Length)		
Session ID				
0Fh (App Data Length)		11h (Length)		
Reserved	Reserved	FEh	SPDM Version	
01h (PCI-SIG)	02h	0003h (PCI-SIG)		
00h (IDE)	04h (Length)		00h (PCI-SIG)	
	Port Index	Reserved	00h (QUERY)	

Unsecure SPDM Messages

The IDE Command Line can send IDE_KM messages using standard unsecure message types 0x01 or 0x03 with the following layout. This layout is used by the IDE command line utility if no SPDM session is defined. How the peer Endpoint is configured to handle IDE_KM via an unsecure SPDM channel is endpoint-specific.

31				0
Reserved		02h / 03h	0001h	
Reserved / Connection ID		00006h (Length)		
Reserved	Reserved		FEh	SPDM Version
01h (PCI-SIG)	02h		0003h (PCI-SIG)	
00h (IDE)	04h (Length)			00h (PCI-SIG)
	Port Index		Reserved	00h (QUERY)

Raw IDE_KM Over DOE

The IDE_KM messages are simply encapsulated in a DOE message with vendor ID 0x16C3 and data object type 0x01. If a function publishes a DOE capability that when probed via the Discovery process, indicates support for this transport layer, the

Synopsys IDE Command Line will send raw IDE_KM messages with the following

31			0
Reserved	01h	16c3h	
Reserved		00003h (Length)	
Port Index	Port Reserved	00h (QUERY)	00h (IDE)

layout:

IDE_KM Over DOE

The establishment of a secure SPDH session is a lengthy multi-step process that ultimately results in the creation of a secure channel for exchanging IDE_KM messages. If the verification objective is not to verify SPDH but IDE, that is a significant overhead that does not contribute to the end goal. Synopsys provides an unsecure channel for exchanging IDE_KM messages between the RC and a function, short-circuiting the entire SPDH stack. Note that you need to limit the use of this channel only to verification; do not use it in the end product.

The IDE_KM messages are simply encapsulated in a DOE message with vendor ID `0x16C3` and data object type `0x01`. If a function publishes a DOE capability that when probed via the Discovery process, advertises support for this protocol, it is assumed it supports IDE_KM-over-DOE.

Consider the following IDE_KM QUERY encapsulated in a Synopsys IDE_KM-over-DOE object as an example.

31			0
Reserved	0x01	0x16C3	
Reserved		00003h (Length)	
Port Index	Reserved	00h (QUERY)	00h (IDE)

8

Performance Management

This chapter provides details on how to get performance measurement metrics while running PCIe traffic in VHS and also details issues in real time performance measurement issues when translating the wall-clock time in VM to ZeBu emulation clock time.

It contains the following sections:

- [PCIe Performance Metrics Print](#)
- [Enabling and Controlling the Prints](#)
- [Real Time Issues in Performance Management](#)

PCIe Performance Metrics Print

VHS provides an option to print PCIe bandwidth information for a given interval of time. The prints are as follows:

```
=====
                                Performance Summary  -- Data payload bytes only
                                DOWN                                UP                                Latency
                                #Bytes  Avg BW                    #Bytes  Avg BW                    Min      Avg      Max
                                Window
PCIe      18196 us            18796      1032 kB/s            373476      20 MB/s            0 ns     0 ns     0 ns
=====
```

Here,

Window: Defines the time taken from the first PCIe transfer to the last PCIe transfer.

DOWN: Provides information for Downstream traffic.

UP: Provides information for Upstream traffic.

#Bytes: Specifies number of bytes transferred.

Avg BW: Specifies average bandwidth calculated as #Bytes/Window.

Note:

Latency information capture is currently not implemented in the performance statistics.

After the run ends, a “Cumulative Summary” is printed as a sum of all the information captured in previous performance windows.

```
=====
```

Cumulative Performance Summary -- Data payload bytes only								
		DOWN		UP		Latency		
	Window	#Bytes	Avg BW	#Bytes	Avg BW	Min	Avg	Max
PCIe	202024 us	459084	2272 kB/s	3392050	16 MB/s	0 ns	0 ns	0 ns

```
=====
```

Enabling and Controlling the Prints

Enabling, displaying, disabling, and resetting of performance metrics is done from inside the Virtual Machine. You need to be a “sudo user” or “administrator” to run these commands.

To enable this feature, write `16c302AA` to class code of PCIe Root Complex(RC), that is, execute the following command:

```
setpci -s 0:02.0 08.1=16c302AA
```

This enables PCIe performance data capture, and you see a print in the testbench log with the message, “Performance Metrics: enabled”.

To print the performance statistics, write `16c302BB` to class code of RC, that is, execute the following command:

```
setpci -s 0:02.0 08.1=16c302BB
```

To disable the performance statistics, write `16c302EE` to class code of RC, that is execute the following command:

```
setpci -s 0:02.0 08.1=16c302EE
```

This disables PCIe performance data capture, and you see a print in the testbench log with the message, “Performance Metrics: disabled”.

To stop/reset performance statistics, you can write `16c302FF` to class code of RC, that is, execute the following command:

```
setpci -s 0:02.0 08.1=16c302FF
```

This disables the statistics and resets the count. So, when you enable the statistics again, the window of calculation starts from 0.

Real Time Issues in Performance Management

Real time is one of the challenges when using the VHS. Virtual Machines are designed to track wall-clock time. This is because they appear to be real machines that behave just like physical ones. All applications are written assuming they are running on a real-time host.

When virtualizing a device, its response time is in the milliseconds range rather than in the microseconds because of the following reasons:

- Virtualized memory regions are no longer accessing physical memory but must be trapped and instead execute a READ or WRITE function.
- Virtualized memory accesses are translated to commands sent to the Virtualization testbench over a socket.
- The Virtualization Testbench must decode the command, translate it into protocol transaction(s), and send the transaction(s) to the transactor on the ZeBu emulator.
- The design in the ZeBu emulator executes the transaction at a clock speed that is 1000x slower than the real silicon, eventually producing a response observed by a transactor.
- The Virtualization Testbench reads the response from the ZeBu emulator and translates it into a response command sent back to the Virtual Machine over the same socket.
- The Virtualized Memory access eventually receives the response command, translates it into the appropriate memory response (or interrupt service request), and returns control to the Virtual Machine.

Meanwhile, the notion of time of the VM has advanced according to the real time elapsed, making it appear that a simple memory-mapped access took far much longer than it normally would have. This apparent slow response time might cause timers to expire prematurely, and completely impacts performance calculations.

Measuring Performance

Customers often want to run performance benchmarks on the Virtual Host solution, for example, IO Meter to benchmark an NVMe device. However, that is not possible if the objective is to get reliable performance metrics. It is quite possible to run a performance benchmark application to generate stimulus for the emulated design and verify its response under extreme conditions. However, any reported performance metric is off by

several orders of magnitude because it is still measuring the performed tasks against the real time clock, without correcting for the additional time consumed by the virtualization mechanism and the slower speed of the emulation.

In this case, time warping does not help. Though you can slow down the real time up to 50x to obtain the required performance metrics, but the results are meaningless. This is so because you tweak the elapsed time (denominator) until you get the expected response. There is no reliable way of determining a suitable time warp percentage as the virtualization slow down is a function of how much time is spend accessing virtualized functions versus other processing in the Virtual machine, which varies from application to application.

To resolve this problem in measuring performance, you can suspend the time advance in the Virtual Machine while a virtualized function is executed and advance the time based on the actual number of clock cycles that were necessary to execute the function. This would work with pure slave functions. But any function with DMA capabilities or independent execution threads is not suitable for this approach.

Example: NVMe

NVMe transfer is performed by writing a command description in memory, then writing a single address in the NVMe memory-mapped space to indicate there is a new command to be executed. This is known as ringing the request doorbell. A thread in the NVMe device responds to that doorbell, reads the next command description directly from the Virtual Machine's memory using an upstream memory read and interprets the command. It then performs the transfer directly to or from the Virtual Machine's memory using a series of upstream memory writes or reads. After the command is complete, the NVMe device sends an interrupt, known as ringing the completion doorbell. In this entire process, only the request doorbell involves a virtualized memory operation. All other transfers are executed concurrently to the Virtual Machine (potentially).

In this case, it is difficult to ascertain the following:

- Number of clock cycles required to execute the transfer.
- Number of actual clock cycles passed.
- State of the Virtual Machine and whether it can resume its normal clock advance.

Measuring performance requires a performance analyzer that accurately measures the data transferred and required clock cycles with a design running in clock-controlled mode.

The VHS includes NVMe Performance Analyzer, which is used to debug and analyze the performance of a NVMe device.

9

Troubleshooting

This chapter describes common issues and frequently asked questions regarding VSA bring up and debug.

For more information, see the following topics:

- [Setup Related Issues](#)
- [ProxyEnv-Related Issues](#)
- [Virtual Machine Issues](#)
- [Interpreting ProxyTB Log](#)
- [Interpreting QEMU Log](#)
- [Interpreting NVMe Log](#)

Setup Related Issues

Setup related issues can occur due to a variety of issues, such as version mismatch between VS solution and VSA and missing mandatory libraries in the host machines. For debugging system-level issues or latency issues in the data path, there is a special utility called the **VHS Profiler** that provides details on the time taken at each stage in the data path. This feature is not available in the regular VSA package. To use this tool contact the VSA Support team.

Compatibility Check Between VSA Package and Transactors

A VSA package contains transactor libraries corresponding to multiple supported transactor versions. After you source the setup.csh file inside the package, the `ZEBU_IP_ROOT` set in your environment is identified and `VSA_VHS_ROOT` is set to the corresponding transactor libraries inside the package.

If the transactor version used is not supported, there can be unexpected system behavior during runtime.

To ensure compatibility, check the `VERSIONS_BUILD` file inside `$VSA_VHS_ROOT`. This file contains all the supported `ZEBU_IP_ROOT` versions. Ensure that the `ZEBU_IP_ROOT` you

are using is listed in the `VERSIONS_BUILD` file. If it is not listed, reach out to the support team for a VSA package that is compatible with your `ZEBU_IP_ROOT`. Also, ensure that you are using the compatible *gcc version* listed in the `VERSIONS_BUILD` file.

ProxyEnv-Related Issues

ProxyEnv-related issues can be caused either during ProxyEnv testbench compile or runtime. This section describes various symptoms and how to debug issues. For debugging such issues, it is recommended to increase the debug level to 2 or 3 where the upper digit corresponds to the transactor debug level and the lower digit corresponds to the ProxyXtor debug level.

ProxyEnv Testbench Compile Issues

If you encounter errors when compiling the ProxyEnv testbench, it could be because of any of the following reasons:

1. Check the gcc version used. The gcc version required for a VSA package is listed in `$VSA_VHS_ROOT/VERSIONS_BUILD` file.
2. Check if you are missing any libraries during testbench compile. Refer to the `-L<libname>` in the Makefile, which is available in the `Example/<Protocol>` folder for the required libraries.
3. Check if you are missing any references during testbench compile. Refer to the `-I<includeFile>` in the Makefile, which is available in the `Example/<Protocol>` folder for all the files that need to be included.
4. Check if you have used the correct ABI version when compiling the testbench. Update `-D_GLIBCXX_USE_CXX11_ABI=<ABI Used>` with the correct ABI version. Refer to the [ABI Support](#) section for details about the correct ABI version to use.
5. Check if you have all the namespaces corresponding to the tools used in the testbench. Refer to the testbench in the `Example/<Protocol>` folder for the required namespace.

ProxyEnv Testbench Runtime Issues

A testbench can stop responding at runtime due to a number of reasons. Some of the commonly seen situations when a testbench stops responding and the steps to workaround them are as follows:

1. During a License Checkout

If you see a freeze during license checkout, check if you are using the correct license versions. Setting `FLEXLM_DIAGNOSTICS` environment variable to 4 on the runtime terminal displays a list of licenses that are checked out.

Note:

Licenses are updated for release versions starting from S-2023.03.

2. During PCIe Link Training

If the log displays “Starting Link Training” and freezes, it is mostly likely a link training issue that needs to be debugged from a transactor or design perspective.

3. During PCIe Pre-Enumeration

Pre-enumeration is the stage in PCIe VHS flow where the VHS software is going to do the first access to endpoint configuration space. If you see a freeze during this stage, ensure that the non-posted transaction to endpoint is complete and a completion has been received from the endpoint. Any pending response can cause the testbench to freeze and requires waveform debug.

4. Symbol Not Found Error

These errors generally occur due to version incompatibility between the `ZEBU_IP_ROOT` used and the transactor libraries loaded dynamically through VHS `libProxy<XTOR>.so` files. The first step is to make sure `$ZEBU_IP_ROOT` and `$VSA_VHS_ROOT/ZEBU_IP_ROOT/<XTOR>/` versions are the same and `$LD_LIBRARY_PATH` is also pointing to the correct version. Also, ensure the correct ABI libraries are used.

5. ProxyPing Failures

For debugging these issues use the “`-D <debugLevel>`” option with the *ProxyPing* command line to increase the number of debug messages. The most common issue in this type of failure is inconsistency between XML configuration file used during testbench run and the one passed during “`make ProxyPingQemu`” command. So, if you are using XML configuration file, ensure that the XML configuration file used at both times is the same.

Virtual Machine Issues

A VM can fail at startup due to a number of reasons. To isolate the issue, first step is to startup the VM in standalone mode without connecting to ZeBu.

To startup the VM in standalone mode, at the QEMU run prompt, remove `QemuRun.args` and then execute the QEMU run command. Alternatively, in the case of VirtualBox, do a `ProxyPing -C <VM_Name>` and then launch the VM. If you see the same issue even after startup in standalone mode, check the following:

1. Hardware virtualization must be enabled in the host machine.
2. KVM Hypervisor must be installed in the host machine.
3. For “Gtk initialization failed” error, set the display on the host machine correctly.
4. For a freeze due to missing libraries, do an `ldd `which qemu-system-x86_64`` and resolve any issues for libraries that are “not found”. Since QEMU is a third-party software, it is not guaranteed that all QSC-V compatible machines can run on the Virtual platform successfully.
5. The qcow2/vdi image and shared folder path provided at the QEMU command line should be correct and must exist.
6. For a VM freeze with `init ramfs` prompt and blank screen, the issue is likely a corrupted image file.
7. Try adding more resources for the VM to startup by using the `-m <RAMs>` or `-smp<Cores>` command options.

If you do not see the issue in standalone mode and only see it when ZeBu is connected, check the following :

- A “Timeout while waiting for PCI Completion” error message in the log file means the default 10 seconds timeout value is reached and the VM has not yet received a completion for the non-posted PCIe transaction. In this situation, you can disable the timeout setting by adding `cplMaxSecs=0` to `-device pcie-zebu-rp` line in `QemuRun.args` file as follows:

```
-device pcie-zebu-rp,id=pv0.pci0.bus1,...,cplMaxSecs=0
```

- A VM can stop responding because of a freeze on the testbench side. In this situation, increase the debug level on both the testbench and the VM to investigate the issue further. On testbench side, set the debug level to 2 or 3 in the configuration XML under the `ProxyXtor` field. On the VM side, during *ProxyPing* command, add `-D 3` to set the debug level to 3. You should also enable `gdb` and get a stack trace in case the VM stops responding abruptly.

Interpreting ProxyTB Log

The testbench log prints all information required to understand what is happening in the virtualized interface. It is the first place to check for debugging any issue. The log messages provide protocol-level information and transaction layer details between ZeBu and transactors or the ProxyXtor interface. However, the amount of information printed in the log varies depending on the verbosity level. Transactor and ProxyXtor debug levels can be independently controlled by modifying the following XML parameter:

```
<vHost:ProxyXtor>
.
.
<vHost:DebugLevel>13</vHost:DebugLevel>
.
.
</vHost:ProxyXtor>
```

Here, the upper digit (1) corresponds to transactor debug level and lower digit (3) corresponds to ProxyXtor debug level. At least a debug level of 13 is recommended for debugging VHS issues.

The following sections describe interpreting these log messages.

Configuration Reads/Writes

```
DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0034.881850152
@00052515914 ( 117) DN CfgRd 00:00.0/000 -> 81:00.0 A:0x0028 L: 4
D:0x00000000 CardbusPtr
```

DEBUG1:	- Debug Level
pci0 [ProxyPCIeXtor]	- ProxyXtor instance name and Type of ProxyXtor
0034.881850152	- Wall clock time
@00052515914	- Transactor clock cycle
(117)	- Number of clock cycles it took to sent the CfgRd Request and get a response
DN CfgRd	- Transaction is a Downstream Configuration Read
00:00.0/000 -> 81:00.0	- Bus:Device.Function details of RC and EP indicating direction of traffic. This is a transaction from RC(00:00.0) to EP(81:00.0)
A:0x0028	- Address of device from where this data to be read
L: 4	- Length of data being Read
D:0x00000000	- Returned Read Data
CardbusPtr	- Description of the configuration register being read

```
DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0067.658746821
@00130031070
DN CfgWr  00:00.0/3f0 -> 00:00.0  A:0x0052  L:   2  D:0x00000081
MSI.Ctrl
```

DEBUG1:	- Debug Level
pci0 [ProxyPCIeXtor]	- ProxyXtor instance name and type of ProxyXtor
0067.658746821	- Wall clock time
@00130031070	- Transactor clock cycle
DN CfgWr	- Transaction is a Downstream Configuration Write
00:00.0/3f0 -> 00:00.0	- This is a transaction from RC(00:00.0) to another register in RC(00:00.0) where 00:00.0 represents Bus:Device.Function
A:0x0052	- Address to where this data is being written
L: 2	- Length of data being written
D:0x00000081	- Data being written
MSI.Ctrl	- Description of the configuration register being written to

Memory Reads/Writes

```
DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0559.258947463
@00135619678 ( 1704) UP MemRd  01:00.0/000 -> A:0x00007faf88000  L:  64
D:0x00180006
```

DEBUG1:	- Debug Level
pci0[ProxyPCIeXtor]	- ProxyXtor instance name and type of ProxyXtor
0559.258947463	- Wall clock time
@00135619678	- Transactor clock cycle
(1704)	- Number of clock cycles it took to send the Upstream Memory Read requests and get a response
UP MemRd	- Transaction is an Upstream Memory Read
01:00.0/000 -> A:0x00007faf88000	- This is an Upstream Read request from EP device (01:00.0) to address 0x00007faf88000
L: 64	- Length of data being Read
D:0x00180006	- Returned Read Data

```
DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0051.457988305
@00006975898      UP MemWr  01:00.0/000 -> A:0x000026e728000  L:4096
D:0x53504e5316c316c3
```

DEBUG1:	- Debug Level
pci0 [ProxyPCIeXtor]	- ProxyXtor instance name and Type of ProxyXtor
0051.457988305	- Wall clock time
@00006975898	- Transactor clock cycle

```

UP MemWr                               - Transaction is an Upstream Memory
Write
01:00.0/000 -> A:0x000026e728000 - This is a transaction from EP device
(01:00.0)                               to address 0x000026e728000
L:4096                                - Length of data being written
D:0x53504e5316c316c3                 - Data being written

DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0557.245105282
@00135062630 ( 1064) DN MemRd 00:10.0/100 -> A:0x00000e290001c L: 4
D:0x00000010 01:00.0 BAR[0][0x0000001c] NVMe.CSTS

DEBUG1:
pci0      [ ProxyPCIeXtor]           - Debug Level
0557.245105282                       - ProxyXtor instance name and
@00135062630                         Type of ProxyXtor
( 1064)                             - Wall clock time
DN MemRd                             - Transactor clock cycle
Read                                 - Number of clock cycles it
00:10.0/100 -> A:0x00000e290001c - took to sent the Downstream
                                      Memory Read requests and
                                      get a response
                                      - Transaction is a Downstream Memory
                                      Read
                                      - This is a Downstream Memory Read
                                      from RC (00:10.0) to
                                      address 0x00000e290001c
L: 4                                  - Length of data being Read
D:0x00000010                         - Returned Read Data
01:00.0 BAR[0][0x0000001c]          - Data is being read from EP device
(01:00.0)
                                      and the address from which it is
                                      being read corresponds to PCIe BAR 0
                                      register with offset 0x0000001c
NVMe.CSTS                             - Description of the register being
read

DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0557.284900314
@00135073822      DN MemWr 00:10.0/000 -> A:0x0000840000000 L: 4
D:0xffe00000 01:00.0 BAR[4][0x00000000] MSI-X[0].Addr.L

DEBUG1:
pci0      [ ProxyPCIeXtor    ]       - Debug Level
0557.284900314                       - ProxyXtor instance name and
@00135073822                         Type of ProxyXtor
DN MemWr                             - Wall clock time
Write                               - Transactor clock cycle
00:10.0/000 -> A:0x0000840000000 - Transaction is an Downstream Memory
(00:10.0)                               to address 0x0000840000000
L:4                                  - Length of data being written
D:0xffe00000                         - Data being written

```

```
01:00.0 BAR[4][0x00000000]      - Data is being written to EP device
(01:00.0)                      and the address to which data is
                                being written
                                to corresponds to PCIe BAR 4 register
                                with
                                offset 0x00000000
MSI-X[0].Addr.L                - Description of the register being
written to
```

Note:

For higher debug levels, additional details are printed in the logs.

Interpreting QEMU Log

Similar to ProxyTB log, QEMU log also prints protocol-level information to help understand details at the transactor, ProxyXtor, and VM interface. The following sections describe how to interpret these log messages.

With Debug Level 1

Configuration Reads/Writes

```
DEBUG: VMLog [pci0      [ Proxy      ] ] 002180.358174270s  DN
CfgRd      A:0.00.0:0x0044  L:  2  D:0x0000000b  T:0ns
```

```
002180.358174270s      - Wall clock time
DN CfgRd               - This is a Downstream Configuration
                        Read transaction
A:0.00.0:0x0044        - This is a Downstream Cfg Rd to
                        address 0x0044 of device 0.00.0 (RC)
L:2                    - Length of data requested is 2bytes
D:0x0000000b          - Returned read data
```

```
DEBUG: VMLog [pci0      [ Proxy      ] ] 000005.445292621s  DN
CfgWr      A:1.00.0:0x0114  L:  4  D:0x00000001  T:0ns
```

```
000005.445292621s      - Wall clock time
DN CfgWr               - This is a Downstream Configuration
                        Write transaction
A:1.00.0:0x0114        - This is a Downstream Cfg write to
                        address 0x0114 of 1:00.0 (EP mapped at B.D.F
                        1.00.0)
L:4                    - Length of data to be written is 4bytes
D:0x00000001          - Value that is being written to this EP register
```

Memory Reads/Writes

```
DEBUG:VMLog [pci0      [ Proxy          ] ] 000003.545868024s  UP
MemWr  Req:1:00.0  ID:0009  A:0x0007ffa6000  L:4096  D:0x00010000
```

```
000003.545868024s      - Wall clock time
UP MemWr              - This is an upstream Memory Write transaction
Req:1:00.0            - This upstream memory write request
                      - is being requested by device 1:00.0 (EP)
ID:0009              - Synopsys specific transaction identifier
A:0x0007ffa6000      - Address to which this Mem Write is
                      - going to be written
L:4096               - 4KB is the payload length
D:0x00010000         - 4Bytes of the payload (part of the data)
```

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.542900124s  UP
MemRd  Req:1:00.0  ID:0004  A:0x0007ffdd040  L: 64  D:0x00010005
```

```
000003.543751183s      - Wall clock time
UP MemRd              - This is an upstream Memory Read transaction
Req:1:00.0            - This upstream memory read request
                      - is being requested by device 1:00.0 (EP)
ID:0004              - Synopsys specific transaction identifier
A:0x0007ffdd040      - Address from which this Mem read is going to be
                      - read
L: 64                - Length of data being read
D:0x00010005         - This is the data that was read from the RC
```

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.529590295s  DN
MemRd                A:0x000fe800008  L: 4  D:0x00010000  T:0ns
```

```
000003.529590295s      - Wall clock time
DN MemRd              - This is a downstream Memory Read transaction to EP
A:0x000fe800008      - Address from which this Mem Read is going to be
                      - read
L:4                  - Length of data being read
D:0x00010000         - This is the value being read from EP
```

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.544947673s  DN
MemWr                A:0x000fe801000  L: 4  D:0x00000004  sent
```

```
000003.544947673s      - Wall clock time
DN MemWr              - This is a downstream Memory Write
A:0x000fe801000      - Address to which this Memory Write
                      - is going to be written
L: 4                  - Length of the data being written
D:0x00000004         - This is the value being written to EP
```

With Debug Levels 2 and 3

With higher debug levels 2 and 3, the nomenclature is maintained but the transactions are split into the following three lines:

1. Requester details
2. Completion details
3. Requester and Completion details in the same line

For instance, one Downstream Configuration Write transaction is represented as:

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.244467415s  DN
CfgWr      A:1.00.0:0x0010  L:  4  D:0xfe800000  request
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.244857208s  DN
CPL      Cpl:1:00.0  ID:003f  CRS:0                      T=0ns
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.244873300s  DN
CfgWr      A:1.00.0:0x0010  L:  4  D:0xfe800000  T:0ns
```

Requester Details

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.244467415s  DN
CfgWr      A:1.00.0:0x0010  L:  4  D:0xfe800000  request
```

000003.244467415s	- Wall clock time of request
DN CfgWr	- This is a Downstream Configuration Write transaction
A:1.00.0:0x0010	- This is a Downstream Cfg write to address 0x0010 of 1:00.0 (EP)
L:4	- Length of data being written
D:0xfe800000	- This is the value being written to EP

Completion Details

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.244857208s  DN
CPL      Cpl:1:00.0  ID:003f  CRS:0                      T=0ns
```

000003.244857208s	- Wall clock time of completion
DN CPL	- This is a downstream completion
Cpl:1:00	- Device 1:00.0 (EP) is the completer sending the response
ID:003f	- Synopsys specific transaction identifier
CRS:0	- Completion Retry Status (0-good completion, 1-bad completion)

Combination of 2 and 3

```
DEBUG: VMLog [pci0      [ Proxy          ] ] 000003.244873300s  DN
CfgWr      A:1.00.0:0x0010  L:  4  D:0xfe800000  T:0ns
```

Interpreting NVMe Log

The following section explains how to generate NVMe analyzer logs, the various stages of NVMe command execution, and how to view and interpret each of the stages in the NVMe Analyzer log.

For more information, see the following topics:

- [Enable NVMe Analyzer Log Generation](#)
- [Main Sections of the NVMe Analyzer Log](#)

Enable NVMe Analyzer Log Generation

In order to generate the NVMe Analyzer logs, you need to first set the following XML configuration option:

```
<vHost:PCIE>
  <vHost:ProtocolAnalyzer>
    <vHost:NVMe>
      <vHost:IsEnabled>true</vHost:IsEnabled>
      <vHost:DebugLevel>3</vHost:DebugLevel>
      <vHost:ReportEvery>0</vHost:ReportEvery>
      <vHost:InactiveIfIdleForSecs>5</vHost:InactiveIfIdleForSecs>
      <vHost:CQDBTimeoutSecs>3</vHost:CQDBTimeoutSecs>
    </vHost:NVMe>
  </vHost:ProtocolAnalyzer>
</vHost:PCIE>
```

This code enables the NVMe Analyzer in the run log. Thereafter, additional logs are printed with [NVMe Analyzer] tag and the PCIe commands are translated into NVMe-level commands.

Main Sections of the NVMe Analyzer Log

There are three main sections in the NVMe log: device detection, device initialization, and command execution. These are explained as follows:

NVMe Device Detection

As part of PCIe Enumeration. Class Code is read by RC to understand the type of device.

```
DEBUG1: Log      [pci0      [ ProxyPCIeXtor    ] ] 0061.497677966
@00012686251 ( 163) DN CfgRd 00:00.0/000 -> 01:00.0 A:0x0008 L: 4
D:0x01080200 Revision+ClassCode
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer    ] ] 0061.497772560
@00012686251 New NVMe device discovered
```

NVMe Device Initialization

As part of the Device Initialization, the key parameters, such as Command Size, Page Size, and Admin Queue, are initialized.

```
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0097.464088930
@00023223741 ( 155) DN MemRd 00:02.0/100 -> A:0x00000fe800014 L: 4
D:0x00000000 01:00.0 BAR[0][0x00000014] NVMe.CC
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0097.465172812
@00023223933 ( 158) DN MemRd 00:02.0/100 -> A:0x00000fe80000c L: 4
D:0x00000000 01:00.0 BAR[0][0x0000000c] NVMe.INTMS
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0097.466170532
@00023224117 ( 155) DN MemRd 00:02.0/100 -> A:0x00000fe800010 L: 4
D:0x00000000 01:00.0 BAR[0][0x00000010] NVMe.INTMC
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.300396389
@00027446651 ( 163) DN MemRd 00:02.0/100 -> A:0x00000fe80001c L: 4
D:0x00000000 01:00.0 BAR[0][0x0000001c] NVMe.CSTS
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.496452932
@00027492945 ( 156) DN MemRd 00:02.0/100 -> A:0x00000fe800000 L: 4
D:0xff010003 01:00.0 BAR[0][0x00000000] NVMe.CAP
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.497437844
@00027493118 ( 157) DN MemRd 00:02.0/100 -> A:0x00000fe800004 L: 4
D:0x00f00020 01:00.0 BAR[0][0x00000004] NVMe.CAP
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.498426504
@00027493290 ( 158) DN MemRd 00:02.0/100 -> A:0x00000fe80003c L: 4
D:0x00000000 01:00.0 BAR[0][0x0000003c] NVMe.CMBSZ
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.613575789
@00027516297 ( 158) DN MemRd 00:02.0/100 -> A:0x00000fe800008 L: 4
D:0x00010100 01:00.0 BAR[0][0x00000008] NVMe.VS
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.613852492
@00027516314 DN MemWr 00:02.0/000 -> A:0x00000fe800014 L: 4
D:0x00000000 01:00.0 BAR[0][0x00000014] NVMe.CC
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.614784950
@00027516473 ( 157) DN MemRd 00:02.0/100 -> A:0x00000fe80001c L: 4
D:0x00000000 01:00.0 BAR[0][0x0000001c] NVMe.CSTS
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.761165650
@00027554184 DN MemWr 00:02.0/000 -> A:0x00000fe800024 L: 4
D:0x001f001f 01:00.0 BAR[0][0x00000024] NVMe.AQA
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.773833367
@00027557401 DN MemWr 00:02.0/000 -> A:0x00000fe800028 L: 4
D:0x108c7000 01:00.0 BAR[0][0x00000028] NVMe.ASQ
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.773883198
@00027557402 DN MemWr 00:02.0/000 -> A:0x00000fe80002c L: 4
D:0x00000001 01:00.0 BAR[0][0x0000002c] NVMe.ASQ
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.773971180
@00027557404 DN MemWr 00:02.0/000 -> A:0x00000fe800030 L: 4
D:0x10940000 01:00.0 BAR[0][0x00000030] NVMe.ASC
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.774032037
@00027557405 DN MemWr 00:02.0/000 -> A:0x00000fe800034 L: 4
D:0x00000001 01:00.0 BAR[0][0x00000034] NVMe.ASC
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0114.775104314
@00027557564 ( 158) DN MemRd 00:02.0/100 -> A:0x00000fe800000 L: 4
D:0xff010003 01:00.0 BAR[0][0x00000000] NVMe.CAP
```

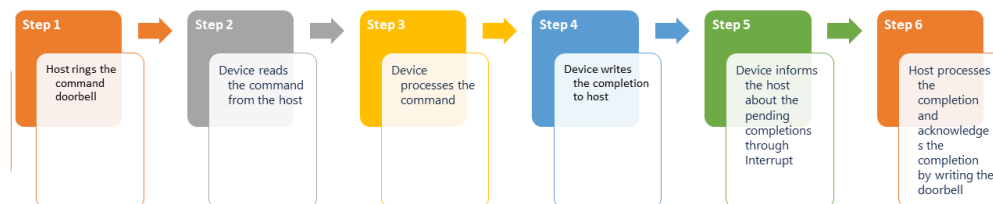


```
DEBUG3: Log [pci0::01:00.0 [ NVMe Analyzer ] ] 0114.776158334
@00027557741 ( 157) DN MemRd 00:02.0/100 -> A:0x00000fe800004 L: 4
D:0x00f00020 01:00.0 BAR[0][0x00000004] NVMe.CAP
DEBUG3: Log [pci0::01:00.0 [ NVMe Analyzer ] ] 0114.776415343
@00027557757 DN MemWr 00:02.0/000 -> A:0x00000fe800014 L: 4
D:0x00460000 01:00.0 BAR[0][0x00000014] NVMe.CC
DEBUG3: Log [pci0::01:00.0 [ NVMe Analyzer ] ] 0114.777325605
@00027557917 ( 158) DN MemRd 00:02.0/100 -> A:0x00000fe800014 L: 4
D:0x00460000 01:00.0 BAR[0][0x00000014] NVMe.CC
DEBUG3: Log [pci0::01:00.0 [ NVMe Analyzer ] ] 0114.777580646
@00027557933 DN MemWr 00:02.0/000 -> A:0x00000fe800014 L: 4
D:0x00460001 01:00.0 BAR[0][0x00000014] NVMe.CC
DEBUG3: Log [pci0::01:00.0 [ NVMe Analyzer ] ] 0114.778618755
@00027558093 ( 158) DN MemRd 00:02.0/100 -> A:0x00000fe80001c L: 4
D:0x00000001 01:00.0 BAR[0][0x0000001c] NVMe.CSTS
```

NVMe Command Execution

There are two types of NVMe commands in the log: **Admin** and **IO** Commands. The purpose of these commands is different but the basic command flow is the same. The commands submitted to `Q[0]` are the Admin commands and the commands submitted to other queues are the IO commands.

Most of the NVMe commands have the following execution flow:



Each of these steps is explained (along with the corresponding log section) in the following topic.

NVMe Command Execution - Steps

There are two types of NVMe commands in the log: **Admin** and **IO** Commands. The purpose of these commands is different but the basic command flow is the same. The commands submitted to `Q[0]` are the Admin commands and the commands submitted to other queues are the IO commands.

At a high-level, NVMe commands are executed in six steps. Each of the following steps explains the flow while interpreting the corresponding log.

Step 1: The host rings the command doorbell.

In this step:

- NVMe driver prepares a command and places that information in the Command Ring Buffer.
- The driver instructs the NVMe device to process the command through a Doorbell register.
- For each queue, there is pair of Submission Queue (SQ) and a Completion Queue (CQ).

There can be multiple queues. Q[0] is for Admin commands and rest of the queues are for IO commands.

```
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.594988609
@00027886051      DN MemWr 00:02.0/000 -> A:0x00000fe801000 L: 4
D:0x00000001 01:00.0 BAR[0][0x00001000] NVMe.SQ[0] DB
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.594996947
@00027886051 NVMe 01:00.0 SQDB[00]
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.595008829
@00027886051 NVMe 01:00.0 SQ [00] DB Tail=0x00000001 (+1 entries)
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.595015174
@00027886051 SQ [00] DB Tail=0x00000001 (+1 entries)
```

Step 2: The device reads the command from the host.

In this step:

NVMe Device determines the address based on the Initialization (registers or admin commands) as follows:

- For Admin Commands, it is through register programming.
- For IO Commands, it is through admin commands.

The NVMe device reads the command (typically 64B) since Device is fetching you find this transfer as UP MemRd.

```
// NVMe Device initiates the "UP MemRd" to fetch 64Bytes commands as
below
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.594988609
@00027886051      DN MemWr 00:02.0/000 -> A:0x00000fe801000 L: 4
D:0x00000001 01:00.0 BAR[0][0x00001000] NVMe.SQ[0] DB

// NVMe Device is reading 64 bytes from 0x0000110880000 (Address is
calculated based
on Queue Number and it's base address)
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608466433
@00027887513 ( 33) UP MemRd 01:00.0/400 -> A:0x0000110880000 L: 64
D:0x00140006
```

```

DEBUG4: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608468279
@00027887513      D[0x000]: 0x00140006 0x00000000 0x00000000
0x00000000
DEBUG4: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608469880
@00027887513      D[0x010]: 0x00000000 0x00000000 0x105b1000
0x00000001
DEBUG4: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608471476
@00027887513      D[0x020]: 0x00000000 0x00000000 0x00000001
0x00000000
DEBUG4: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608472995
@00027887513      D[0x030]: 0x00000000 0x00000000 0x00000000
0x00000000

// NVMe Analyzer is helping you understand the 64Bytes with command field
// level information
// Definition of the each field's info - Refer to the NVMe Spec.
// Definition of the each field's info - Refer to the NVMe Spec.
// Following command is an Admin command (anything going to Queue 0 i.e.
// SQ[00])
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608474846
@00027887513 NVMe 01:00.0 SQ[00] Entry @0x0000000110880000 (64 bytes)
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608484015
@00027887513 Fetched SQ[0] Entry 0x0000
000027887513 pci0::01:00.0 SQE[0000.0000]: Addr=0x0000000110880000
Size=64

pci0::01:00.0      +-[SQE]-----+
pci0::01:00.0      |  NSID      CID  PSDT  FUSE  OPC  |
pci0::01:00.0      DW00: | 00000000 0014  0    0    06  |
pci0::01:00.0      DW02: |                      |
pci0::01:00.0      DW04: | MPTR:      0000000000000000  |
pci0::01:00.0      DW06: | PRP1:      00000001105b1000  |
pci0::01:00.0      DW08: | PRP2:      0000000000000000  |
pci0::01:00.0      |-----|
pci0::01:00.0      DW10: | 00000001 | CDW11: | 00000000  |
pci0::01:00.0      DW12: | 00000000 | CDW13: | 00000000  |
pci0::01:00.0      DW14: | 00000000 | CDW15: | 00000000  |
pci0::01:00.0      +-----+
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608507697
@00027887513 New command CID=0x0014, in SQ[0].
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608564644
@00027887513 NVMe 01:00.0 SQE[00] CID=0x0014 Identify Controller
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.608568089
@00027887513 SQE[00] CID=0x0014 Identify Controller
DEBUG1: Log      [nvme0 [ NVMe Device  ] ] 0121.608960448
Identify Controller

```

Step 3: The device processes the command.

In this step:

The NVMe device parses the 64B command and takes an action based on that.

For example, if the commands need to move data, the following types of data transfer can happen (and PCIe TLP Type info):

- ◦ Host to Device (UP MemRd)
- Device to Host (UP MemWr)

The following example is that of a log that has the `Identify Controller` command. In this example, the host is fetching the controller information so that the data is moving from the device to host.

```
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.609769740
@00027887708      UP MemWr 01:00.0/000 -> A:0x00001105b1000 L:1024
D:0x53504e5316c316c3
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.611460712
@00027887725      UP MemWr 01:00.0/000 -> A:0x00001105b1400 L:1024
D:0x000000000
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.612258251
@00027887744      UP MemWr 01:00.0/000 -> A:0x00001105b1800 L:1024
D:0x000000000
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613121066
@00027887761      UP MemWr 01:00.0/000 -> A:0x00001105b1c00 L:1024
D:0x000000000

// The same 4KiB data is annotated in to the Identify Controller
Structure field level information for easier understanding
000027887761 pci0::01:00.0
+-[ID:Controller]-----+
virtNVMe 0001 pci0::01:00.0 0000 | VID: 16c3 SSVID: 16c3 SN: SNPS
NVMe-Compl pci0::01:00.0 0018 | MN: Functional model is not fully
pci0::01:00.0 0040 | FR: Q2020.09
pci0::01:00.0 0048 | RAB: 2^01 OUI: 00-50-c2 CMIC:
00 MDTs: 2^00 pci0::01:00.0 004E | CNTLID: 01 VER:
000000000 pci0::01:00.0 0054 | RTD3R: 00000000ms RTD3E:
00000000ms pci0::01:00.0 005C | OAES: 000
CTRATT: 0 pci0::01:00.0 0070 | FGUID:
23e7d473-2d5f-ec98-0050-c20100000000 pci0::01:00.0 0100 | OACS: 00 ACL: 03 AERL:
03 FRMW: 03 pci0::01:00.0 0105 | LPA: 00 ELPE: 00 NPSS:
00 AVSCC: pci0::01:00.0 0109 | APSTA: 00 WCTEMP: 0157
CCTEMP: 02ae MTFA: 0000 |
```

Step 4: The device writes completion to the host.

After the command is processed, the device informs the host through the Completion command (16B). This completion command is written to Completion Queue.

```
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613353446
@00027887762      UP MemWr 01:00.0/000 -> A:0x0000110889000 L: 16
D:0x000000000
```

```

DEBUG4: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613355237
@00027887762      D[0x000]: 0x00000000 0x00000000 0x00000001
0x00010014
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613357034
@00027887762 NVMe 01:00.0 CQ[00] Entry @0x0000000110889000 (16 bytes)
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613362719
@00027887762 Pushed CQ[0] Entry 0x0000
000027887762 pci0::01:00.0 CQE[0000.0000]: Addr=0x0000000110889000
Size=16
                pci0::01:00.0 +-[CQE]-----+
pci0::01:00.0 DW00: | 00000000 |
pci0::01:00.0 DW01: | Reserved |
pci0::01:00.0 DW02: | SQ: 0000 | SQHp: 0001 |
pci0::01:00.0 DW03: | St: 0000 | P: 1 | CID: 0014 |
pci0::01:00.0 +-[CQE]-----+
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613370517
@00027887762 NVMe 01:00.0 Completed command Q0:20 via CQ[00].
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613373827
@00027887762 NVMe 01:00.0 CQE[00] Q[00].Head=0x00000001 (-1 entries)
CID=0x0014 SF=0x0000
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613375969
@00027887762 CQE[00] Q[00].Head=0x00000001 (-1 entries) CID=0x0014
SF=0x0000
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613386309
@00027886051 - 00027887762 (Lat: 1711) NVMe 01:00.0 Q[00]
CID=0x0014 Identify Controller VID=0x16c3 SN='SNPS virtNVMe 0001'
MN='Functional model is not fully NVMe-Compl' FR='Q2020.09' SQES=102
CQES=68 MAXCMD=1 NN=1
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.613395769
@00027887762 Q[00] CID=0x0014 Identify Controller VID=0x16c3
SN='SNPS virtNVMe 0001' MN='Functional model is not fully NVMe-Compl'
FR='Q2020.09' SQES=102 CQES=68 MAXCMD=1 NN=1

```

Step 5: The device informs the host about the pending completions through Interrupt

In this step:

The device informs the host about the completion through MSI/MSI-x Interrupt because just writing the completion to the Completion to Ring buffer is not enough (as described in step 4).

```

DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.614067707
@00027887763      UP MemWr 01:00.0/000 -> A:0x00000fee00298 L: 4
D:0x00000000 MSI-X[0]
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer  ] ] 0121.614069742
@00027887763 NVMe 01:00.0 SSD->Host: Interrupt.

```

Step 6: : Host processes the completion and acknowledges the completion by writing the doorbell.

In this step:

After the Interrupt host recognizes the completion commands to be processed, it processes the pending completions and informs the status of the completed commands to the NVMe device. It does this by writing to the CQ doorbell. The NVMe device keeps a track of the status of the completed commands so that it does not overwrite the Pending Completions.

```
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer   ] ] 0121.641617412
@00027894718      DN MemWr 00:02.0/000 -> A:0x00000fe801004 L:   4
D:0x00000001 01:00.0 BAR[0][0x00001004] NVMe.CQ[0] DB
DEBUG3: Log      [pci0::01:00.0 [ NVMe Analyzer   ] ] 0121.641619523
@00027894718 NVMe 01:00.0 CQDB[00].
DEBUG2: Log      [pci0::01:00.0 [ NVMe Analyzer   ] ] 0121.641622092
@00027894718 NVMe 01:00.0 CQ [00] DB Head=0x00000001 (-1 entries)
DEBUG1: Log      [pci0::01:00.0 [ NVMe Analyzer   ] ] 0121.641624054
@00027894718 CQ [00] DB Head=0x00000001 (-1 entries)
```

10

Examples

This chapter provides instructions for running PCIe, Virtual NVMe, USB, ENET, and UFS examples.

It consists of the following topics:

- [Settings for Running an Example](#)
- [PCIe Example](#)
- [USB Example](#)
- [Ethernet Example](#)
- [UFS Example](#)

Settings for Running an Example

Environment Settings

For running VSA examples, you need to set the following mandatory environment settings. Before you begin with the settings, set `VSA_ABI` as specified in the **ABI Support subsection** of the [ABI Support](#) section.

```
% setenv ZEBU_ROOT <PATH_TO_ZEBU_ROOT>
% source ${ZEBU_ROOT}/zebu_env.csh
% setenv ZEBU_IP_ROOT <PATH_TO_ZEBU_IP_ROOT>
% setenv LD_LIBRARY_PATH
  ${ZEBU_IP_ROOT}/libABI${VSA_ABI}:${LD_LIBRARY_PATH}
% setenv PATH ${ZEBU_IP_ROOT}/binABI${VSA_ABI}:${PATH}
% setenv LD_LIBRARY_PATH <PATH_TO_GCC>/lib64:${LD_LIBRARY_PATH}
% setenv PATH <PATH_TO_GCC>/bin:${PATH}
% setenv VSA_VHS_ROOT
% source ${VSA_VHS_ROOT}/setup.csh
```

Testbench Location

VHS supports two types of testbench formats: C++ based and zRci based. Depending on the type of testbench you are using, you can go to the respective Examples folder under `$VSA_VHS_ROOT/Examples/<Protocol>`. For example, to access the PCIe example, go to `Examples/PCIe`.

PCIe Example

If you are using VirtualBox hypervisor, then a VM must be available (refer to section [Creating a Virtualization Testbench](#)) and the name of the Virtual Machine is assumed as myVM.

If you are using QEMU, then you do not need to create a separate VM but ensure that you have a valid image path specified in QCOW2_IMG.

Compile

To compile the PCIe example, follow the instructions in transactor examples area. After you have a successful zebu compile, update the compiled database path in VHS XML file as following:

```
<vHost:ZebuWorkPath>/mypath/16x32b_b2b.zebu.work/zebu.work</vHost:ZebuWorkPath>
```

There are two types of transactor compile models available in PCIe transactor example:

1. Root Complex (RC) as transactor and End Point (EP) as Designware IIP core (EP DUT).

For compilation of this example, follow the runtime instructions in [VHS With End Point as IIP DUT](#) section.

2. Both root complex (RC) and End Point (EP) as PCIE transactors connected in a back to back mode.

For compilation of this example, follow the runtime instructions in [VHS Example With VDM NVMe Device](#).

Note:

Depending on the compile you choose to perform, the run directory and XML file to be used at runtime changes.

Runtime

This example uses VDM NVMe device. Ensure that you have VSA VDM package and license to run this. To run the PCIe example, perform the following steps:

1. Copy the example directory corresponding to your PCIe transactor compile model to your local directory.

For RC as transactor and EP as Designware IIP core (EP DUT):

```
% cp -r $VSA_VHS_ROOT/Examples/PCIe somewhere/local
% cd somewhere/local
```

For both root complex and end point as transactor:

```
% cp -r $VSA_VHS_ROOT/Examples/VHS_VDM somewhere/local
% cd somewhere/local
```

VHS With End Point as IIP DUT

1. Update the `config_1pcie_dpi.xml` file in `shared/config` folder with your ZeBu compile path (`vHost:ZebuWorkPath`), `designFeatures` file path (`vHost:DesignFeaturesPath`) and other PCIe configurations.
2. Ensure all the configuration options in the `config_1pcie_dpi.xml` file are updated as required. Compile the ProxyEnv testbench and run the test on emulator host.

```
% cd zRci/runtime
% make run_zrci CONFIG_XML=config_1pcie_dpi.xml
```

3. Wait until you see the *waiting for client to connect* message, use the ProxyPing command, and then start the VM.

- For VirtualBox, use the following commands:

```
% make ProxyPingVBox CONFIG_XML=config_1pcie_dpi.xml
% make VM_NAME=myVM VBoxRun
```

- For QEMU, use the following commands:

- % make QemuRun CONFIG_XML=config_1pcie_dpi.xml QCOW2_IMG=<Path to your QCOW2 Image file>

Note:

Executing `QemuRun` executes `ProxyPingQemu` to generate `QemuArgs`.

- This starts the Virtual Machine.

4. Ensure that the DUT is virtualized and memory resources are allocated by using `lspci -v` command inside the VM as following:

Figure 12 DUT Virtualization (PCIe)

```
synopsys@synopsys-VirtualBox:~$ sudo su
root@synopsys-VirtualBox:/home/synopsys# lspci -v -s 01:00.0
01:00.0 Non-Volatile memory controller: Synopsys, Inc. Device 5100 (prog-if 02 [
NVM Express])
    Flags: bus master, fast devsel, latency 0, IRQ 16
    Memory at e2900000 (64-bit, prefetchable) [size=32K]
    Capabilities: [40] Power Management version 3
    Capabilities: [50] MSI: Enable- Count=1/1 Maskable- 64bit+
    Capabilities: [70] Express Endpoint, MSI 00
    Capabilities: [b0] MSI-X: Enable+ Count=2 Masked-
    Capabilities: [100] Advanced Error Reporting
    Capabilities: [148] Device Serial Number 00-00-00-00-00-00-00-00
    Capabilities: [158] Power Budgeting <?>
    Capabilities: [168] #19
    Capabilities: [198] #26
    Capabilities: [1c8] #27
    Capabilities: [210] #2a
    Capabilities: [240] Transaction Processing Hints
    Capabilities: [2cc] #00
    Capabilities: [2dc] Latency Tolerance Reporting
    Capabilities: [2e4] L1 PM Substates
    Capabilities: [2f4] #1b
    Capabilities: [340] Vendor Specific Information: ID=0002 Rev=4 Len=100 <
?>
    Capabilities: [440] #25
    Kernel driver in use: nvme
    Kernel modules: nvme

root@synopsys-VirtualBox:/home/synopsys# ls /dev/nvme*
/dev/nvme0 /dev/nvme0n1
root@synopsys-VirtualBox:/home/synopsys#
```

5. A simple C++ test that can be used to send reads and writes to the EP device can be found in the following location inside the VHS package:

<VHS_Package>/Examples/PCIe/shared/memTest

This test can be used to check whether reads and writes to EP are happening successfully. In order to run the test, copy this memTest/ folder inside the VM using Qemu Shared Folder feature. For more information, see [Shared Folder Settings](#).

Do the following to run the test:

```
% cd memTest/
% make
% ./memTest -W <phy address of BAR> <data to be written> //This
  performs a memory write
% ./memTest -R <phy address of BAR> <data expected> //This will
  perform a memory read + compare the read data with data passed in
  command
```

Example: `./memTest -W 0x2000000000 0x12345678`

VHS Example With VDM NVMe Device

The VSA VHS package contains an example with NVMe Virtual Device Model that can be used to run NVMe traffic using traffic generators such as the Flexible I/O tester (fio) or IO Meter.

Note:

For running this example, you must also have VSA VDM license.

This example setup is available in the VHS package in the following directory: `Examples/VHS_VDM`.

To run the Virtual NVMe example, perform the following steps:

1. Enter the runtime directory and compile the ProxyEnv testbench and run the test on the emulator.

```
% cd zRci/runtime
% make run_zrci CONFIG_XML=config_vhs_vdm_pcie_nvme_gen*.xml USE_VDM=1
```

Note:

Ensure that you update the configuration settings in the `config_vhs_vdm_pcie_nvme_gen*.xml` file located under `somehere/local/VHS_VDM/shared/config/` with all the required fields including the PCIe transactor back to back example compile database path `<vHost:ZebuWorkPath>`.

2. Wait until the *waiting for client to connect* message appears.
3. Add the Proxy ZeBu transactor PCIe driver to the VM and start the VM using Qemu.

```
% make QemuRun CONFIG_XML=config_vhs_vdm_pcie_nvme_gen*.xml
QCOW2_IMG=<Path to your QCOW2 Image file>
```

After the VM boots, use fio (for Ubuntu/Centos) or I/O meter (for Windows) to run the NVMe traffic.

Note:

A VHS simulation example using VDM NVMe Device is available in `Examples/PCIe/simulation` folder. Refer to the README file for instructions on running the example. Executing `QemuRun` executes `ProxyPingQemu` to generate `QemuArgs`.

NVMe Analyzer GUI

The NVMe Analyzer GUI is a special use case of VHS NVMe solution. By integrating SLED and Verdi applications, you can visualize NVMe transactions, simplifying debugging

and analysis. The GUI presents performance metrics such as latency and bandwidth in a clear, easy-to-read format.

To use the NVMe Analyzer GUI, follow the example setup in the section [VHS Example With VDM NVMe Device](#). Additional environment settings and post-processing commands are required.

Environment Settings

Configure the following environment variables to generate the SLED database:

```
setenv SLED_PROJECT <path to new .sld file>
setenv SLED_BACKEND BINFILES
```

After setting these variables, complete the steps in [VHS Example With VDM NVMe Device](#). Upon completion, a SLED database is created with the name specified in SLED_PROJECT.

Post-Run: SLED to FSDB Conversion

Convert the SLED database to FSDB format using the sled2fsdb utility. Use the following commands:

```
setenv VERDI_HOME <Path to your Verdi installation>
setenv PATH $VERDI_HOME/bin:$PATH
setenv PA_CUSTOM_VIP_PATH <Path to PA_CUSTOM_SLED installation>
sled2fsdb -P $SLED_PROJECT -all
```

Note:

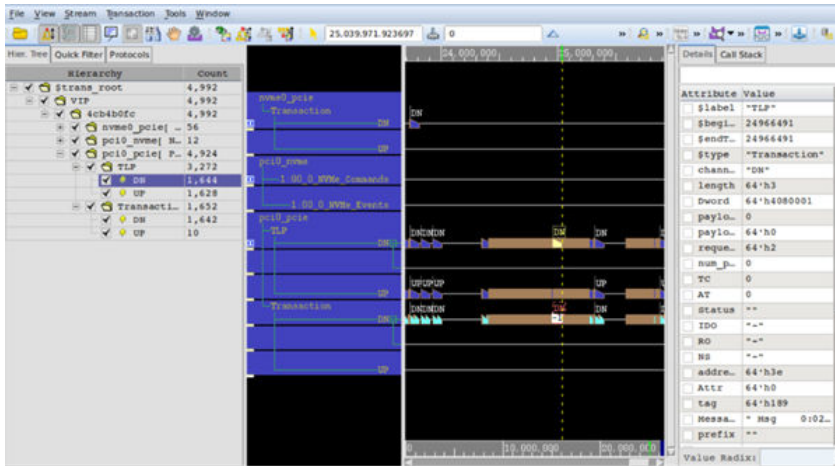
The sled2fsdb utility requires a specific version of Verdi, which differs from the version included with Zebu. Contact the product team for compatible Verdi and PA_CUSTOM_SLED packages.

Viewing the NVMe Analyzer GUI

Open the converted FSDB file in Verdi to access the NVMe Analyzer GUI:

```
verdi -workMode protocolDebug <GeneratedFSDB>
```

This opens the **Verdi NVMe Analyzer** window as shown in the following:



To generate performance analysis reports, select Verdi Performance Analyzer from the Tools menu under Transaction Debug. This opens the performance analyzer view.

For additional information about NVMe Analyzer GUI and Performance Analyzer, see *NVMe-SLED Solution Flow Application Note*.

DMTCP Save and Restore With VHS-VDM PCIe

To do a DMTCP save and restore after the VM is up and running, use the example test case available in the VHS package. To run the example, perform the following steps:

Set Up Environment Variables

Use the following commands to set up the terminal with the required environment variables:

```
%setenv DMTCP_HOME <DMTCP corresponding to your ZEBU_ROOT>
%source $DMTCP_HOME/dmtcp_settings.csh
%setenv DMTCP_PATH $DMTCP_HOME
%setenv PATH $DMTCP_HOME/bin:$PATH
%setenv LD_LIBRARY_PATH $DMTCP_HOME/lib/dmtcp:$LD_LIBRARY_PATH
%setenv LD_LIBRARY_PATH $DMTCP_HOME/../../DmtcpHandler:$LD_LIBRARY_PATH
%setenv DMTCP_DL_PLUGIN 0
%setenv ZEBU_INTERNAL_SIGNAL PWR
%setenv ZEBU_TXTOR_MODE 2
%setenv ZEBU_DONT_CANCEL_THREADS 1
```

Perform Checkpoint Run

To perform a checkpoint run, follow these steps:

1. Under the `$VSA_VHS_ROOT/Examples/VHS_VDM/zRci/scripts` folder, in the `Start_zebu.sh` file, set `DMTCP=1`.
2. Edit `$VSA_VHS_ROOT/Examples/VHS_VDM/shared/config/config_vhs_vdm_pcie_nvme_gen*.xml` file and update `zebu.work` path and other PCIe configurations.

3. On a runtime host, launch the run with `DMTCP=1` flag:

```
% make run_zrci DMTCP=1 CONFIG_XML=config_vhs_vdm_pcie_nvme_gen*.xml  
USE_VDM=1
```

After the run completes successfully, a `Ready to Connect` message is displayed.

4. Launch the VM:

```
%make ProxyPingQemu CONFIG_XML=config_vhs_vdm_pcie_nvme_gen*.xml  
%make QemuRun QCOW2_IMG=<Path to your Qcow2 VM Image file>
```

The VM starts.

5. Finally, to save the ZeBu hardware and VM states, from a terminal on the host machine, run the following command:

```
%make QemuTakeSnapshot
```

This saves the VM states and after its done, automatically does a hardware save.

Perform Restore Run

To perform a restore run, follow these steps:

1. Browse to access the run directory:

```
% cd inside zRci.run_*/runtime_data/
```

2. Launch the ZeBu restore run using following command:

```
% ./zRci.db/ZebuVMSnapshot*/dmtcp_restart_script_zebu.sh
```

After the run is successfully complete, a `Ready to connect` message is displayed and ZeBu hardware states are restored. You can now restore VM states.

3. To restore a VM state, ensure `RESTORE_DBDIR` and `ZEBU_SNAP_RESTORE` targets in the `Config.mk` file are populated correctly. For example: `RESTORE_DBDIR = zRci.run_0`
`ZEBU_SNAP_RESTORE = ZebuVMSnapshot.`

4. Launch the VM with following command:

```
%make QemuRun
QCOW2_IMG=<Path to your Qcow2 VM Image file>
```

The VM state is restored and connected to ZeBu. You can now continue working on the restored VM.

Note:

The Shared Folder feature is not supported in Qemu during checkpointing. Make sure to unmount the `SharedFolder` inside the VM before taking a VM snapshot. After snapshotting is done, you can mount the `SharedFolder` again.

USB Example

If you are using VirtualBox hypervisor, then a VM must be available. For details about the steps to create a virtualization testbench, see [Creating a Virtualization Testbench](#). Ensure that the name of the VM is updated in the `Makefile`.

To compile and run the USB example, perform the following steps:

1. Copy the example source file in a local directory and ensure that directory is the current working directory.

```
% cp -r $VSA_VHS_ROOT/Examples/USB3xDPI somewhere/local
% cd somewhere/local/USB3xDPI
```

2. Compile the Verilog code for the transactor by following the steps specified in the USB Transactor Example setup.
3. Compile the ZeBu proxy transactor testbench server.

```
% make ProxyTB
```

4. Use the `CONFIG_XML` file corresponding to your configuration and update `zebu.work` path, `designFeatures`, and other configurations.

Note:

There is a `CONFIG_XML` file available in the `shared/config/` folder corresponding to a type of device.

5. Run the test case on the emulator:

```
%make ProxyTBRUN #on Zebu host
```

6. Start the VM after you see the `waiting for client to connect` message.


```
% make QemuRun QARGS=<Path to qcow image file>
```

7. Ensure that the DUT is virtualized and associated with the correct USB Host controller by using the `lsusb` and `lsusb -t` commands, as follows:

Figure 13 DUT Virtualization (USB)

```
vusb_5_2_6_b - Oracle VM VirtualBox [STEP=0 CS=0 PAT=0 RR0=1 RR3=1 LOG=1 HWVirt=1]
ubuntu@ubuntu: ~
File Edit View Search Terminal Help
ubuntu@ubuntu:~$ lsusb
Bus 002 Device 002: ID 0499:3002 Yamaha Corp.
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
ubuntu@ubuntu:~$ lsusb -t
/: Bus 02.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/6p, 5000M
   |__ Port 1: Dev 2, If 0, Class=Vendor Specific Class, Driver=, 5000M
/: Bus 01.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/8p, 480M
ubuntu@ubuntu:~$
```

Ethernet Example

The VHS package contains an Ethernet (ENET) example in the `<VSA VHS Package>/Examples/ENETDPI` folder. The following steps describe how to run an ENET VHS test where two VMs are started, one, connected to each Ethernet transactor instance. An ENET traffic test is done by pinging from one VM to the other.

Compile

There are four different ENET drivers, namely, 400 G PCS, 400 G MII, 800 G PCS, and 800 G MII. For the purpose of this document, the example with 400 G speed available in the transactor `example_400G/enet_sanity_test/zebu/Makefile` area is used.

Note:

Few pointers about ENET compilation are provided. For detailed instructions, see the `readme` file in the ENET transactor `Example` setup.

- For PCS Compile, specify the following options with the compile command:

```
PCS_EN = 1 PORT_NB = 2 QIWC = 1 CLOCK_TYPE = RTL_CLK
```

- For MII Compile, specify the following options with the compile command:

```
PCS_EN = 0 PORT_NB = 2 QIWC = 1 CLOCK_TYPE = RTL_CLK
```

Note:

These are the recommended settings for this ENET example. However, refer to the latest transactor document most up-to-date options.

Runtime

Before you begin, ensure the following prerequisites are met:

- Latest VSA VHS package is downloaded from SolvNet and installed in your local area.
- Environment settings are sourced including ZEBU_ROOT, ZEBU_IP_ROOT, VSA_VHS_ROOT.
- Two VMs or two qcow2 images are used for running this example. In this case, ubuntu_16_enet.qcow2 is used for the first VM and ubuntu_16_enet2.qcow2 image is used for second.

Note:

Ethernet specific ubuntu images can be sourced from the following location:

/remote/sdg_sv_vsa/VMImages/ubuntu_16_enet.qcow2 and /remote/sdg_sv_vsa/VMImages/ubuntu_16_enet2.qcow2.

- Update the path of your first VM in Examples/ENETDPI/shared/Config.mk file under QCOW2_IMG target
- Ensure that you link your ENET transactor compile in the configuration XML file under ZebuWork field and also configure the right ENET mode.

For a list of ethernet drivers and their corresponding config.xml file, see [Table 3](#). The configuration xml files are found in Examples/ENETDPI/shared/config/ folder.

For a list of ENET configuration modes, see the \$VSA_VHS_ROOT/schemas/schema_eth_*.yaml file under Interfaces session.

Table 3 List of Ethernet Drivers With Corresponding config.xml Name

Driver Speed	Configuration XML
400G MII	config_enet_dpi_mii_400g.xml
400G PCS	config_enet_dpi_pcs_400g.xml
800G MII	config_enet_dpi_mii_800g.xml
800G PCS	config_enet_dpi_pcs_800g.xml

To run the ENET example, perform the following steps:

1. Get access to a runtime host.
2. Execute the run command for 400 G PCS as follows: % make run_zrci
CONFIG_XML=config_enet_dpi_pcs_400g.xml

3. After a ready to connect message is displayed, do ProxyPing for both VMs from a separate terminals on the runtime host:

For the first VM: % make ProxyPingQemu CONFIG_XML=<specify the one used>

For the second VM, % make ProxyPingQemu PV=pv1 QCOW2_IMG=\$PWD/
VirtualMachine/ubuntu_16_enet2.qcow2 QEMUARGS=QemuRun2.args
FORWARDING_PORTNUM=3023 TELNET_PORT=1235 LOG=qemu2.log
CONFIG_XML=<specify the one used>

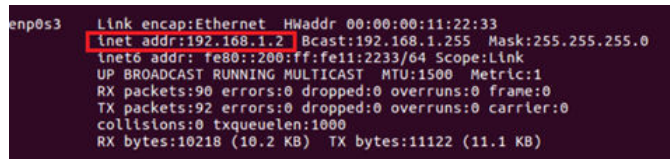
4. Launch the two VMs:

```
% make QemuRun (1st VM)
% make QemuRun PV=pv1
QCOW2_IMG=$PWD/VirtualMachine/ubuntu_16_enet2.qcow2
QEMUARGS=QemuRun2.args FORWARDING_PORTNUM=3023 TELNET_PORT=1235
LOG=qemu2.log
```

Running Ping Test Between VMs

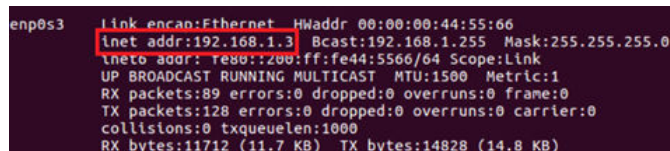
After both the VMs start, you can execute `ipconfig` command from VM terminal to see the respective IP addresses. For the purpose of this example, consider the following example IP addresses:

Figure 14 Example IP Address for VM1



```
enp0s3 Link encap:Ethernet HWaddr 00:00:00:11:22:33
inet addr:192.168.1.2 Bcast:192.168.1.255 Mask:255.255.255.0
inet6 addr: fe80::200:ff:fe11:2233/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:90 errors:0 dropped:0 overruns:0 frame:0
TX packets:92 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:10218 (10.2 KB) TX bytes:11122 (11.1 KB)
```

Figure 15 Example IP Address for VM2



```
enp0s3 Link encap:Ethernet HWaddr 00:00:00:44:55:66
inet addr:192.168.1.3 Bcast:192.168.1.255 Mask:255.255.255.0
inet6 addr: fe80::200:ff:fe44:5566/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:89 errors:0 dropped:0 overruns:0 frame:0
TX packets:128 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:11712 (11.7 KB) TX bytes:14828 (14.8 KB)
```

You can run the ping test from any VM to the other. For example, to run the test from VM1 use the command `ping 192.168.1.3`.

A successful Ethernet packet exchange is confirmed between the 2 VMs when the following conditions are met:

- The following messages are displayed on the terminal:

```
enp0s3 link encap:Ethernet HWaddr 00:00:00:44:55:66
inet addr:192.168.1.3 Bcast:192.168.1.255 Mask:255.255.255.0
lnet0 addr: fe80::200:ff:fe44:5566/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:89 errors:0 dropped:0 overruns:0 frame:0
TX packets:128 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:11712 (11.7 KB) TX bytes:14828 (14.8 KB)
```

- The run log shows no data loss between TX data sent via and RX data received at eth1/eth0 as shown in the following figure:

```
00001: Log [eth1] [ProxyEthTator] 0752.855124227 000440552438 Tx L: 98 00:00:00:11:22:33 00:00:00:44:55:66 8888 4500005479940000400170bfc
00002: Log [eth0] [ProxyEthTator] 0752.866482899 000440552438 Rx L: 182 00:00:00:11:22:33 00:00:00:44:55:66 8888 4500005479940000400170bfc
00003: Log [eth0] [ProxyEthTator] 0752.863142196 000440552438 Rx L: 182 00:00:00:11:22:33 00:00:00:44:55:66 8888 4500005479940000400170bfc
00004: Log [eth1] [ProxyEthTator] 0752.863142196 000440552438 Tx L: 98 00:00:00:11:22:33 00:00:00:44:55:66 8888 4500005479940000400170bfc
00005: Log [eth0] [ProxyEthTator] 0752.863142196 000440552438 Rx L: 182 00:00:00:11:22:33 00:00:00:44:55:66 8888 4500005479940000400170bfc
00006: Log [eth1] [ProxyEthTator] 0752.864266767 000440552438 Tx L: 98 00:00:00:11:22:33 00:00:00:44:55:66 8888 4500005479940000400170bfc
```

UFS Example

To compile and run the UFS example, perform the following steps:

- Copy the example source file in a local directory and ensure that directory is the current working directory.

```
% cp -r $VSA_VHS_ROOT/Examples/UFS somewhere/local
% cd somewhere/local/UFS/C++ (or) % cd somewhere/local/UFS/zRci/runtime
```

- Compile the Verilog code for the transactor.

```
% make compile
```

- Update the configuration options in the `shared/Config.mk` file and run the UFS example.

Note:

To use UFS from a Linux machine, a custom driver is required. The `QCOW2` image with the latest custom kernel is available at `/remote/sdg_sv_vsa/VMImages/ubuntu_16_ufs.qcow2`. The `CONFIG_XML` file is available in the `shared/config/` folder. Use the `CONFIG_XML` file corresponding to your configuration and update it in the `Config.mk` file. Ensure that the `CONFIG_XML` file is updated with `zebu.work`, `designFeatures`, and `hdl` path.

- Get the ZeBu runtime host.

- For C++ testbench, use the command: `% make all`
- For zRci testbench, use the command: `% make run_zrci`
`CONFIG_XML=config_ufs_dpi.xml`

A ready to connect message is displayed.

5. Execute the following command on the terminal: `%make ProxyPingQemu`
`CONFIG_XML=config_ufs_dpi.xml`

Note:

This setup requires all required arguments for Virtual Host.

6. Launch the Virtual Host.

```
% make QemuRun CONFIG_XML=config_ufs_dpi.xml
```

Note:

When the VM starts, directly open a terminal in the VM or connect through `ssh port-forwarding` setup from the host machine.

If using `ssh port connect`:

```
% ssh -Xp 3025 root@localhost
% cd kernelbuild/linux-stable/userScripts
% ./modinsert.sh
```

After the commands are successfully executed, you should see the UFS disks/ devices under Disks (in ubuntu) or under `/dev/sd*`, which is listed under `dmesg`.

To format and mount the UFS disk follow the instructions in the section on *Formatting and Mounting UFS Disk*.

11

Hardware and Software Interface

The hardware interfaces that physically connect to the emulated design are provided by protocol-specific transactors. For more information about each hardware interface, see one of the corresponding user manual.

- *PCIe Transactor User Manual*
- *CXL Transactor User Manual*
- *USB 3.X Transactor User Manual*
- *USB 2.0 Transactor User Manual*
- *ENET Transactor User Manual*
- *SATA Host Transactor User Manual*
- *UFS Host Transactor User Manual*

Note:

It is possible to obtain a reference to the underlying individual interface-specific proxy components to perform low-level operations as needed by a design or a test. This is entirely optional and only as needed.

This chapter consists of the following subtopics:

- [ProxyEnv](#)
- [ProxyPCleXtor Component](#)
- [ProxyUSBXtor Component](#)
- [ProxyEthXtor Component](#)

ProxyEnv

This is a C++ class that encapsulates all of ZeBu-side of VHS. Only a single instance of this class is needed for a ZeBu testbench, regardless of how many physical interfaces or Virtual Machine connections are used. An XML file describes the structure and optional configuration options of the ZeBu testbench.

Other ZeBu transactors might be instantiated along side of the Proxy Environment component. This use model requires a little more care in coordinating the instantiation, initialization, starting and running of the other components.

Constructor

This constructor creates an instance of a standalone proxy as follows:

```
SNPS::VSA::VHS::ProxyVisorProxyEnv::ProxyEnvVisor ( const char * name,
                                                    const char      *xml,
                                                    const Callbacks_t *cbs      = NULL,
                                                    unsigned int    debugLevel = 0,
                                                    long              XMLoffset  = 0);
```

This constructor creates an instance of a proxy environment that can be used with other transactors, or with simulation as follows:

```
SNPS::VSA::VHS::ProxyVisorProxyEnv::ProxyEnvVisor ( const char * name,
                                                    const char      *xml,
                                                    ZEMI3Manager    *zManager,
                                                    const Callbacks_t *cbs      = NULL,
                                                    unsigned int    debugLevel = 0,
                                                    long              XMLoffset = 0,
                                                    svt_c_runtime_cfg *runtime  = NULL);
```

where,

- `name`: Name of the instance
- `xml`: Path to the XML file containing the specification and configuration of the design interfaces to be virtualized
- `cbs`: An optional set of callbacks that might be called by the Proxy Environment to report exceptions or significant events
- `debugLevel`: Debug level for the Proxy Environment (0-4)
- `XMLoffset`: Offset where to start parsing the XML file
- `zManager`: Pointer to the ZEMI3Manager managing all the other transactors in the ZeBu testbench
- `runtime`: Configuration of the runtime environment

connect () Method

This method performs the following:

1. Initiates the connection process to all of the Virtual Machines
2. Waits for all connections to be established (which requires that all transactors have initialized their connection to the design)
3. Waits for all connections to be disconnected before returning

For information on how to configure the connections between the Proxy Environment and a Virtual Machine, see [ProxyEnv Schema Structure](#).

This method returns `FALSE` if connections were not successful.

```
bool connect
```

This method should be called after the `StartAllProxyXtors()` method has been called. After it returns, the `shutdown()` method should be called to cleanly terminate all internal threads and processes.

disconnect () Method

This method disconnects the Proxy Environment from all connected Virtual Machines.

```
bool disconnect ( )
```

This method returns `true`, if successful. This method might be called from callback functions.

shutdown () Method

This method disconnects (if still connected) the Proxy Environment from all connected Virtual Machines and cleanly shutdowns all internal threads and processes. The instance can be safely destroyed after this method returns.

```
bool shutdown ( )
```

This method returns `true`, if successful. This method might NOT be called from callback functions.

setDebugLevel () Method

This method sets the debug level for the Proxy Environment, but not the individual underlying interfaces.

```
void setDebugLevel ( unsigned int level )
```


where,

- `level`: Debug level, (0-4)

startAllProxyXtors () Method

This method starts all proxy transactors contained in this Proxy Environment instance, initiating their connections (example, link training) with the design.

```
bool startAllProxyXtors ( )
```

This method returns `true`, if successful. This method should be called after the design has been fully reset and ready to initialize its interfaces, such as link training. Subsequently, the `connect()` method should be called preferably after all interfaces have been initialized.

getProxy*Xtors () Methods

These methods return a pointer to interface specific virtualization components encapsulated in the Proxy Environment, or NULL if the specified name was not found.

The name of each component is the same specified in the XML description.

For details on the methods that can be called for further runtime configuration and interaction, the chapter specific to each virtualization component.

ProxyEnv XML Structure

An example of config is as follows:

```
<vHost:Config xmlns:vHost='urn:synopsys.com:vHost'>
  <vHost:Version>1.0</vHost:Version>
  <vHost:CreationTime>2021-06-10T16:38:17Z</vHost:CreationTime>
  <vHost:ZebuWorkPath>${PWD}/../zebu/zcui.work.B2B/zebu.work</vHost:ZebuWorkPath>
  <vHost:DesignFeaturesPath>${PWD}/../zebu/designFeatures.B2B</vHost:DesignFeaturesPath>
  <vHost:ProxyVisor>
    <vHost:Name>pv0</vHost:Name>
    <vHost:IsServer>true</vHost:IsServer>
    <vHost:PortNumber>0</vHost:PortNumber>
    <vHost:ServerHostName></vHost:ServerHostName>
    <vHost:ServerSpecFile>${HOME}/VM.socket</vHost:ServerSpecFile>
    ...
    ...
    <vHost:ProxyXtor>
      <vHost:Name>pci0</vHost:Name>
      ...
  </vHost:ProxyVisor>
</vHost:Config>
```

```

...
...
<vHost:PCIE>
  <vHost:Type>DPI</vHost:Type>
  ...
  ...
  ...
</vHost:PCIE>
</vHost:ProxyXtor>
</vHost:ProxyVisor>
</vHost:Config>

```

In the config structure, the `ZebuWorkPath` and `DesignFeaturesPath` are mandatory. You need at least one `ProxyVisor` component to connect to VM. Every VM has its specified `ProxyVisor` component. There can be one/many `ProxyXtor` components registered with `ProxyVisor`.

ProxyEnv Schema Structure

Every component in the XML has a reference schema at `$VSA_VHS_ROOT/ Schemas/ <component>/schema.xml`.

Consider the following example of PCIe (`ProxyPCIEXtor`) component schema:

```

schema: "$ZEBU_VM_ROOT/Schemas/PCIe/schema.yaml" title:
  gtb_proxy_pcie_xtor alias: ProxyPCIEXtor description: ProxyPCIEXtor
  Schema type: object required: - Name properties:
Name:
label: Name
description: ProxyPCIEXtor Instance Name type: string advanced: - 'true'
  - 'false'
advanced_default: 'false' value: set: []
... .. PCIe:
label: PCIe

```

```

description: PCIe Xtor Configuration advanced: - 'true'
  - 'false' advanced_default: 'false'
value:
set: [] properties:
Type:
label: Type
description: Type of physical PCIe driver type: enum
default: Gen4 c_type: NA advanced:
  - 'true'
  - 'false' advanced_default: 'false' value:
set:
  - info: Zcei Transactor string: Gen4
    c_value: NA

```

```
- info: DPI Transactor string: DPI c_value: NA
- info: CXL Transactor string: CXL c_value: NA

- info: Functional PCIe Model string: Functional
  c_value: NA
- info: NVMe string: NVMe c_value: NA
```

Every “**properties**” in the schema translates to a config parameter tag of that component in the XML. Nested “**properties**” in schema is a child config parameter tag of that component. The “**type**” parameter of each property specifies the configuration value type.

For example the “**Name**” property is of type “**string**”. Each property has a parameter “**value**” and “**set**” and it is a child parameter for value. If “**set**” is described in the schema, it describes the possible values that the *config* parameter can take as specified under **value** > **set** > **string** (“string”). Sometime “**min**” and “**max**” allowed values are specified for **type** > **int** or **unsigned int**. “**default**” specifies the default value of the parameter. “**description**” describes the config parameter.

ProxyPCleXtor Component

This component virtualizes a PCIe, CXL, SATA Host, or UFS Host interfaces. The following methods might be called at runtime to perform additional configurations, inject additional commands, or react to responses from the DUT that cannot (or should not) be handled by the Virtual Machine.

- [enableClockControl \(\) Method](#)
- [cfgRd \(\) Method](#)
- [cfgWr Method](#)
- [memWr \(\) Method](#)
- [setDebugLevel \(\) Method](#)
- [setPreTrainingFct \(\) Method:](#)
- [setInitFct \(\) Method](#)
- [setPreDownstreamFct \(\) Method](#)
- [setPreCompletionFct \(\) Method](#)
- [setHotResetFct \(\) Method](#)
- [setPostHotResetFct \(\) Method](#)

- [setLinkResetFct \(\) Method](#)
- [setHotPlugFct \(\) Method](#)
- [setMessageHandler \(\) Method](#)
- [sendMessage\(\) Method](#)
- [lock \(\) / unlock \(\) Method](#)

enableClockControl () Method

This method dynamically enables and disables the clock control mode.

```
void enableClockControl(bool enable = true);
```

where, `enableClockControl(false)` switches to the free running mode.

cfgRd () Method

This method executes a Configuration Read operation.

```
uint32_t cfgRd ( uint8_t  busNum,  
    uint8_t  devNum,  
    uint8_t  fnNum,  
    uint16_t  addr,  
    uint8_t  len  
)
```

Where, 'len' must be 1, 2, or 4 (bytes).

If this method is successful, it returns the readback value. If this method fails, it returns 0xFFFFFFFF.

cfgWr Method

This method executes a configuration write operation.

```
bool cfgWr ( uint8_t  busNum,  
    uint8_t  devNum,  
    uint8_t  fnNum,  
    uint16_t  addr,  
    uint8_t  len,  
    uint32_t  data  
)
```

This method returns `true`, if successful.

memRd () Method

This method executes a Memory Read operation for eight bytes or less.

```
uint64_t memRd (    uint64_t  addr,
                   uint8_t   len
                   )
```

If this method is successful, it returns the readback value. If this method fails, it returns 0xFFFFFFFFFFFFFFFF.

This method executes an arbitrary-length Memory Read operation as follows:

```
bool memRd (
    uint64_t  addr,
    uint16_t  len,
    void* data
)
```

This method returns `true`, if successful.

memWr () Method

This method executes a Memory Write operation for 8 bytes or less.

```
void memWr (
    uint64_t  addr,
    uint8_t   len,
    uint64_t  data
)
```

This method executes a Memory Write operation for an arbitrary-length.

```
void memWr (
    uint64_t  addr,
    uint16_t  len,
    void * data
)
```

setDebugLevel () Method

This method sets the debug verbosity level of the Proxy transactor and of the underlying transactor. The least significant decimal digit sets the debug level of the Virtual Host Solution “*proxy*” layer that is located on top of the ZeBu transactor. The second least significant decimal digit sets the debug level of the underlying ZeBu transactor.

For example, setting the debug level to 13, sets the debug level of the “*proxy*” layer to 3 and of the ZeBu transactor to 1.

```
virtual void setDebugLevel ( unsigned int  lvl )
```

where,

- `lvl`: Debug level, $(0..3) + 10 \cdot (0..3)$.

setPreTrainingFct () Method:

This method registers a function that is called just before link training is initiated. It enables additional sideband testbench and DUT configuration operations (such as signal forces and memory load) to be performed. The link training starts when this method returns `true`.

```
void setPreTrainingFct(bool (*fct)(void *user), void *user);
```

This method returns `true`, if successful.

setInitFct () Method

This method registers a function that is called before the VM connection is established thereby allowing additional configuration operations to be performed. Commands from the VM are accepted as soon as this function returns.

```
void setInitFct ( bool(*fct)(ProxyPCIExtor *proxy, void *user),  
                void * user  
                )
```

This method returns `true`, if successful.

setPreDownstreamFct () Method

Register a function that is called just before a downstream `CfgRd`, `CfgWr`, `MemRd`, `MemWr`, `IoRd`, or `IoWr` operation is started, only if the registered address matches.

If `0xFFFFFFFFFFFFFFFF` is registered, it is triggered for all down stream transactions. If `busNum` is 0, it is a `Mem` operation. If `wData` is `NULL`, it is a `READ` operation.

```
void setPreDownstreamFct(void (*fct)(ProxyPCIExtor *proxy, uint8_t  
    busNum, uint8_t devNum, uint8_t fnNum, uint64_t addr, uint16_t len, void  
    *wData, void *user), uint64_t addr, void *user);
```

setPreCompletionFct () Method

This method registers a non-blocking function that is called before a downstream completion for an upstream Memory Read operation is sent. The content of the response might be modified, or the length might be shortened.

If the length is set to `0xFFFF`, an `UNSUPPORTED` response is sent.

If 'delay' is set to a non-zero value, the response is delayed by the specified number of PCIe clock cycles at minimum (but might be delayed for longer).

If 'reorder' is set to a non-zero value, the response is delayed until the specified number of (other) downstream completions have been sent

'numPendingCpl' is the current number of Cpl TLP currently delayed or reordered.

This callback function **MUST NOT BLOCK** the execution thread. If the callback function returns `false`, the response is not sent at all.

```
typedef struct CompletionContext_s {
    uint64_t      addr;
    uint16_t      reqId;
    bool          atomicOp;
    uint8_t       AT;
    uint32_t      PASID;
    unsigned int  numPending;
} CompletionContext_t;
void setPreCompletionFct(bool (*fct)(ProxyPCleXtor *proxy, const
CompletionContext_t &context, uint16_t &len, uint8_t *data,
                                uint32_t &delayCpl, uint8_t &reorderCpl,
void *user), void *user = NULL);
```

setHotResetFct () Method

This method registers a function that is called before hot reset is initiated and allows for sideband DUT forces. Hot reset is triggered immediately after this method. This method must return `true`, if successful.

```
void setHotResetFct(bool (*fct)(void *user), void *user = NULL);.
```

setPostHotResetFct () Method

This method registers a function that is called after `HotReset`. `HotReset` is triggered before issuing transactor reset. This method must return `true`, if successful.

```
void setPostHotResetFct(bool (*fct)(void *user), void *user = NULL);.
```

setLinkResetFct () Method

This method registers a function that is called before hot reset is initiated and allows for sideband DUT forces. `HotReset` is triggered immediately after this method. This method must return `true` if successful.

```
void setLinkResetFct(bool (*fct)(void *user), void *user = NULL);.
```

setHotPlugFct () Method

This method is a Hot Plug Callback.

```
void setHotPlugFct(bool (*fct)(void *user), void *user = NULL);
```

setMessageHandler () Method

This method registers a message handling callback function that is called whenever a message TLP is received.

```
void setMessageHandler(void (*fct)(const Message_t &msg, void *user),  
void *user = NULL);
```

The callback function is called with the following `struct` and describes the message that was received:

```
typedef struct Message_s {  
    uint16_t    reqId;  
    uint16_t    tag;  
    uint8_t     route;  
    uint8_t     code;  
    union {  
        uint64_t    addr;  
        uint16_t    byId  
    } routing;  
    uint32_t    bytes;  
    const char* data;  
} Message_t;
```

sendMessage() Method

This method sends a message TLP to the design.

```
bool sendMessage(const Message_t &msg);
```

lock () / unlock () Method

These methods lock/release the transactor for/ from exclusive local use. This prevents operations coming from the Virtual Machine from interfering with a sequence of local accesses.

Note:

Only the thread that locked the transactor for exclusive use can unlock it.

```
void unlock ( )
```

ProxyUSBXtor Component

This component virtualizes a USB Host interface. The following methods might be called at runtime to perform additional configuration, inject additional commands, or react to responses from the DUT that cannot (or should not) be handled by the Virtual Machine.

The following functions are used to define the function of the proxy transactor for USB:

- [lock \(\) Method](#)
- [unlock \(\) Method](#)
- [setInitFct \(\) Method](#)
- [setDebugLevel \(\) Method](#)

lock () Method

This method locks the transactor for exclusive use by preventing the Virtual Machine and other threads from executing commands.

```
void lock ( )
```

Note:

Locking the transactor may cause timeouts on the Virtual Machine.

unlock () Method

This method releases the transactor from exclusive use.

```
void unlock ( )
```

Note:

Only the thread that locked the transactor for exclusive use can unlock it.

setInitFct () Method

This method registers a function that is called before the Virtual Machine connection is established thereby allowing additional configuration operations to be performed. Commands from the VM are accepted as soon as this function returns.

```
void setInitFct (
bool(*fct)(ProxyUSBXtor *proxy, void *user) ,
void * user
)
```

This method returns `true`, if successful.

setDebugLevel () Method

This method sets the log level of detail for *USBProxyXtor*.

```
virtual void setDebugLevel ( unsigned int lvl )
```

where,

- `lvl`: Debug level, 0-4.

ProxyEthXtor Component

This component virtualizes a NIC interface. The following methods might be called at runtime to perform additional configurations, inject additional commands, or react to responses from the DUT that cannot (or should not) be handled by the Virtual Machine.

This proxy transactor requires ENET Transactor.

This section explains the following topics:

- [setDebugLevel\(\) Method](#)
- [enableTx\(\) Method](#)
- [enableRx\(\) Method](#)
- [isTxEnabled\(\) Method](#)
- [isRxEnabled\(\) Method](#)
- [flushFrames\(\) Method](#)
- [getFrameCounts\(\) Method](#)
- [printStats\(\) Method](#)
- [ProxyEthXtor Component](#)

setDebugLevel() Method

The following is the syntax of the `setDebugLevel()` method:

```
void setDebugLevel(unsigned int level)
```

This method sets the debug verbosity level for this instance of the `ProxyEthXtor`. Ensure that the following is true for this method:

```
0 <= level <= 5
```

enableTx() Method

The following is the syntax of the `enableTx()` method:

```
bool enableTx(bool enable)
```

This function enables the transmission of ETH frame to the DUT. By default, frame transmission is enabled.

enableRx() Method

The following is the syntax of the `enableRx()` method:

```
bool enableRx(bool enable)
```

This function enables the reception of ETH frame from the DUT. By default, frame reception is enabled. When frame reception is disabled, frames are still sunk, but they are not forwarded to the Virtualization Solution.

isTxEnabled() Method

The following is the syntax of the `isTxEnabled()` method:

```
bool isTxEnabled()
```

This method returns `true` if the transmission of ETH frame is enabled.

isRxEnabled() Method

The following is the syntax of the `isRxEnabled()` method:

```
bool isRxEnabled()
```

This method returns `TRUE` if the reception of ETH frame is enabled.

flushFrames() Method

The following is the syntax of the `flushFrames()` method:

```
unsigned int flushFrames(unsigned in maxFrames)
```

This method flushes frame that is still pending for transmission to the DUT, up to the specified number of frames.

Also, this method returns the number of frames actually flushed.

getFrameCounts() Method

The following is the syntax of the *getFrameCounts()* method:

```
bool getFrameCounts(unsigned int &tx, unsigned int &rx)
```

This method enables you to update the `tx` and `rx` arguments with the current running total of frames sent and received.

printStats() Method

Syntax

```
void printStats(FILE *fp = stdout)
```

This method prints the current running transmit and receive statistics (number of frames and number of byte counts for a given transactor interface).

Examples

Create a method or function to get the following statistics in your testbench: **Arguments:** interface (`eth0/eth1`) **Return Values:** standard output of transactor frames send / receive packets)

```
void getStatistics(const char *interface)
{
    ProxyEthXtor *eth0 = env->getProxyEthXtor(interface);
    eth0->printStats(stdout);
}
```

Call this method in `zRci` parameters method as shown

```
void* zRci_param (const std::string& key, const std::string &value)
{
    std::cerr << " simz_param: key = " << key << " value = " << value <<
    std::endl;
    if (key == "printStats" )
    {
        const char* c = value.c_str();
        getStatistics(c);
        return NULL;
    }
}
```

Use this key at runtime (zRci prompt) to get the Frames Sent and Received packets on the transactor.

```
zRci % testbench -param print_stats eth0
```

The log with current running transmit and receive statistics is printed as follows:

```
simz_param: key = print_stats value = eth0
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372662807
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372673227
=====
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372676547
      Statistics      Frames      Bytes
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372678547
=====
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372681427
      Tx -> to Zebu :      128      17638
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372683857
      Rx <- from Zebu :      128      18150
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372685797
-----
INFO : Log      [eth0      [ ProxyEthXtor    ] ] 4164.372687567
```

printEthStats() Method

Syntax

```
void printEthStats(boolean)
```

This method prints all the statistics for the transmitter and receiver of a port.

Examples

Add the following function in the testbench:

```
void getStatistics(const char *interface) {
    ProxyEthXtor *eth = env->getProxyEthXtor(interface);
    if (eth == NULL) {
        fprintf(stderr, "ERROR: There is no %s Interface
        Available\n", interface);
    }
    else {
        eth->printEthStats(false);
    }
}
```

Call this method in zRci parameters method as shown

```
void* zRci_param (const std::string& key, const std::string &value)
{
    std::cerr << " simz_param: key = " << key << " value = " << value <<
    std::endl;
```

```
if (key == "print_stats" )
{
    const char* c = value.c_str();
    getStatistics(c);
    return NULL;
}
}
```

Use this key at runtime (zRci prompt) to get the Frames Sent and Received packets on the transactor.

```
zRci % testbench -param print_stats eth0
```

The log with current running transmit and receive statistics is printed as follows:

```
zRci % testbench -param print_stats eth0
simz_param: key = print_stats value = eth0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] Print Dashboard:
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txPktNb
151
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txByteNb
18610
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txBadPktNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txBadByteNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txRuntPktNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txRuntByteNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txFirstTS
66590451
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] txLastTS
1412287087
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] DeltaTxTS
1345696636
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] RateTx
0.020005
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxPktNb
151
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxByteNb
18610
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxBadPktNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxBadByteNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxCRCErrorPktNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxRuntPktNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxRuntByteNb
0
```

```
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxFirstTS
66590451
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxLastTS
1412287087
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] DeltaRxTS
1345696636
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] RateRx
0.020005
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxPauseNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] rxPfcNb
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] PFC PAUSE COUNTERS
INFO on clk_1562.5::
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] PauseCounter
0X0

INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] Delayed DPI related
INFO:::
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] idleDPICount
0
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] transmitTimeSw(in
ms) 2260787
INFO      : hw_top_svs.enet_i[0].e_inst [xtor_enet_svs] transmitTimeHw(in
ms) 2260787
```